



Facultad de Ingeniería
Carrera de Ingeniería de Sistemas e Informática

Tesis

“Implementación de un IDS basado en Snort y técnicas de Machine Learning para redes perimetrales en colegios privados de Lima Metropolitana”

Autores

Luis Alonso, Torres Valle
Yussel David, Abensur Rojas

para optar el Título Profesional de Ingeniero de Sistemas e Informática

Asesor

Mg. Julio Ezequiel Ramirez Pelaez
Mg. Yamil Alexander Quiñones Nieto

Lima – Perú

Mayo, 2025

9% Similitud general

El total combinado de todas las coincidencias, incluidas las fuentes superpuestas, para ca...

Filtrado desde el informe

- Bibliografía

Fuentes principales

- 9%  Fuentes de Internet
- 3%  Publicaciones
- 4%  Trabajos entregados (trabajos del estudiante)

DEDICATORIA

A mi esposa, por su amor, paciencia y apoyo incondicional durante todo este proceso, a mis tres hijas, mi mayor motivación y alegría, quienes con su ternura me dieron fuerzas para continuar. Este logro es para ustedes, por su sacrificio y por ser siempre mi inspiración.

Yussel David Abensur Rojas

Dedico este trabajo, en primer lugar, a Dios, por darme la vida, la salud y la fortaleza necesarias para seguir adelante y permitirme alcanzar esta meta.

A mis padres, por ser el pilar de mi vida. A mi padre, por ser mi ejemplo de superación y esfuerzo constante, y a mi madre, por su amor incondicional y su apoyo permanente.

A mi hermana y a mi sobrino, por ser mi alegría diaria y recordarme que, con esfuerzo y perseverancia, todo se puede lograr.

Y a Missy, por su silenciosa compañía en cada noche de estudio, por estar presente en los momentos de cansancio y recordarme, sin palabras, que nunca estuve solo.

Este logro también es de ustedes.

Luis Alonso Torres Valle

AGRADECIMIENTO

Quiero agradecer profundamente a mi esposa y a mis tres hijas, por su apoyo, comprensión y sacrificio durante este proceso; este logro también es suyo.

A mi compañero de tesis, por su dedicación y trabajo conjunto.

A todos, gracias por acompañarme en este camino.

Yussel David Abensur Rojas

Agradezco a Dios por brindarme la fortaleza y constancia necesarias para culminar este trabajo de investigación.

A mis asesores de tesis, por su orientación y valiosos aportes académicos durante el desarrollo de esta investigación.

A mi compañero de tesis, por el compromiso y el trabajo en equipo demostrados a lo largo de todo este proceso.

Finalmente, agradezco a todas las personas que me brindaron su apoyo y palabras de aliento durante este camino académico.

Luis Alonso Torres Valle

RESUMEN

La ciberseguridad en redes perimetrales de instituciones educativas privadas de la ciudad de Lima Metropolitana es el problema que explora este estudio, centrándose en la identificación de peligros en tiempo real a través del uso de herramientas abiertas y métodos de inteligencia artificial. Para poder superar las restricciones de detección que tienen los sistemas tradicionales basados en firmas, se sugiere implementar un sistema híbrido de detección de intrusos (IDS) que una el motor Snort con algoritmos de aprendizaje automático. El Colegio Hans Christian Andersen es el caso de estudio y su infraestructura tecnológica en expansión muestra vulnerabilidades reales frente a los ciberataques actuales. El estudio adopta un enfoque cuantitativo y utiliza medidas como la tasa de detección, los falsos positivos y el tiempo de respuesta para confirmar la eficacia del sistema sugerido. Se prevé que los resultados posibiliten reforzar la seguridad de las redes escolares con una alternativa técnica factible, de bajo costo y replicable, aportando así al ecosistema seguro de la educación digital en escenarios peruanos.

Palabras clave: Ciberseguridad; Redes perimetrales; IDS; Snort; Machine Learning; Inteligencia artificial.

ABSTRACT

Cybersecurity in perimeter networks of private educational institutions in the city of Lima is the issue explored in this study, which focuses on identifying threats in real time through the use of open tools and artificial intelligence methods. In order to overcome the detection limitations of traditional signature-based systems, it is suggested that a hybrid intrusion detection system (IDS) be implemented that combines the Snort engine with machine learning algorithms. The Hans Christian Andersen School is the case study, and its expanding technological infrastructure shows real vulnerabilities to current cyberattacks. The study takes a quantitative approach and uses measures such as detection rate, false positives, and response time to confirm the effectiveness of the suggested system. The results are expected to make it possible to strengthen the security of school networks with a feasible, low-cost, and replicable technical alternative, thus contributing to a secure digital education ecosystem in Peru.

Keywords: Cybersecurity; Perimeter networks; IDS; Snort; Machine Learning; Artificial intelligence.

INDICE GENERAL

RESUMEN.....	4
ABSTRACT.....	5
INDICE DE TABLAS	9
INDICE DE FIGURAS	10
INDICE DE ANEXOS	12
INTRODUCCIÓN	13
CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA.....	14
1.1. Descripción de la Realidad Problemática	14
1.2. Formulación del Problema	16
1.2.1. Problema General.....	16
1.2.2. Problemas Específicos	16
1.3. Determinación de Objetivos	16
1.3.1. Objetivo General.....	16
1.3.2. Objetivos Específicos.....	16
1.4. Hipótesis	17
1.4.1. Hipótesis General	17
1.4.2. Hipótesis Específicas.....	17
1.5. Justificación de la Investigación	18
1.5.1. Teórica.....	18
1.5.2. Práctica.....	18
1.5.3. Metodológica	18
1.5.4. Tecnológica	19
1.6. Delimitación del Estudio	19
1.6.1. Delimitación Espacial.....	19
1.6.2. Delimitación Temporal	19
1.6.3. Delimitación Conceptual	20
CAPÍTULO II: MARCO TEÓRICO.....	21
2.1. Antecedentes	21
2.2. Estado del Arte	22
2.3. Bases Teóricas	26
2.3.1. Ciberseguridad y amenazas a redes educativas.....	27
2.3.2. Sistemas de detección de intrusos (IDS) y Snort como herramienta de código	

abierto	27
2.3.3. Machine Learning en detección de intrusos y enfoques híbridos con Snort	29
2.3.4. Particularidades de las redes educativas perimetrales.....	31
2.3.5. Justificación de indicadores técnicos de evaluación.....	31
2.4. Marco Conceptual	33
CAPÍTULO III: METODOLOGÍA DE INVESTIGACIÓN	36
3.1 Diseño de Investigación	36
3.1.1. Diseño	36
3.1.2. Tipo	36
3.1.3. Enfoque	37
3.1.4. Población.....	37
3.1.5. Muestra.....	37
3.1.6. Operacionalización de Variables.....	38
3.2. Instrumentos de investigación / herramientas	39
3.2.1. Técnicas	39
3.2.2. Instrumentos.....	39
3.2.3. Confiabilidad	40
3.2.4. Validez del instrumento.....	41
3.2.5. Recolección de la información	42
3.3. Metodología de la implementación de la solución	43
3.3.1. Selección de la metodología	43
3.3.2. Desarrollo de la metodología	43
3.4. Metodología para la medición de resultados de la implementación	47
3.4.1. Métrica de la variable dependiente	47
3.5. Cronograma de Actividades	49
3.5.1. Cronograma de actividades basado en Sprints.....	49
3.6. Presupuesto.....	49
CAPITULO IV: DESARROLLO DE LA SOLUCIÓN	52
4.1. Propuesta de Solución	52
4.1.1. Fase de Inicialización.....	53
4.1.2. Fase de Planificación y Estimación.....	55
4.1.3. Fase de Implementación.....	57
4.2. Análisis e Interpretación de los Resultados	62
4.2.1. Análisis Descriptivo.....	62
4.2.2. Análisis Inferencial.....	64
CAPÍTULO V: DISCUSIÓN DE RESULTADOS	66

5.1. Discusión general de hallazgos.....	66
5.2. Comparación con antecedentes y marco teórico.....	66
5.3. Limitaciones del estudio	67
5.4. Proyecciones y posibles mejoras	67
CAPÍTULO VI: CONCLUSIONES Y RECOMENDACIONES.....	69
6.1. Conclusiones	69
6.2. Recomendaciones	70
REFERENCIAS	71
ANEXOS.....	76

INDICE DE TABLAS

Tabla 1: Operacionalización de la Variable Dependiente.....	39
Tabla 2: Diagnóstico y Entrenamiento de modelos ML.....	44
Tabla 3: Integración Snort + ML y Despliegue en la red real	45
Tabla 4: Análisis de resultados y validación	46
Tabla 5: Cronograma de Actividades.....	49
Tabla 6: Plan de Release	56
Tabla 7: Comparación de métricas de desempeño en pretest y posttest.....	63
Tabla 8: Resultados de las pruebas de hipótesis (pretest vs posttest).....	65
Tabla 9: Configuración del entorno virtual de pruebas.....	83
Tabla 10: Parámetros de configuración inicial de Snort.....	88
Tabla 11: Resultados de validación cruzada (Random Forest).....	100
Tabla 12: Resultados de validación cruzada (SVM).....	101

INDICE DE FIGURAS

Figura 1: Clasificación de IDS	28
Figura 2: Arquitectura de un IDS híbrido	30
Figura 3: Desempeño entre IDS vs. IDS + ML	32
Figura 4: Comparación de errores y latencia de IDS	32
Figura 5: Mapa conceptual del enfoque híbrido IDS + ML.....	35
Figura 6: Flujo integral de procesamiento en el sistema de detección Snort + ML	46
Figura 7: Plan de Release del Proyecto	57
Figura 8: Comparación de métricas	63
Figura 9: Comparación de tiempo de respuesta.....	64
Figura 10: Configuración de la máquina virtual en VMware.....	84
Figura 11: Proceso de instalación de Ubuntu	84
Figura 12: Proceso de instalación de Ubuntu	85
Figura 13: Actualización del sistema operativo.....	85
Figura 14: Verificación de la interfaz de red y asignación de dirección IP.	86
Figura 15: Verificación del estado de sincronización de la hora.	86
Figura 16: Creación de snapshot	87
Figura 17: Actualización del Sistema	88
Figura 18: Instalación de dependencias para Snort.....	89
Figura 19: Instalación de Snort.	89
Figura 20: Versión de Snort instalada.	90
Figura 21: Fragmento del archivo snort.conf con la variable de red.	90
Figura 22: Validación de configuración.....	91
Figura 23: Validación de configuración.....	92
Figura 24: Listado de /var/log/snort/ mostrando la rotación (archivos .gz).	93
Figura 25: instalación de wireshark.	94
Figura 26: Snort corriendo en logging.	95
Figura 27: TShark capturando.....	95
Figura 28: Paquetes ICMP en Wireshark.	95
Figura 29: Alerta ICMP en Snort.	96
Figura 30: Alerta HTTP en Snort.	96
Figura 31: Petición HTTP GET en Wireshark.	96
Figura 32: Escaneo SYN en Wireshark.	97
Figura 33: Alerta de escaneo en Snort.	97
Figura 34: Carga del dataset CICIDS2017 en Colab.	98
Figura 35: Limpieza y balanceo.	98

Figura 36: Entrenamiento del modelo.	99
Figura 37: Modelos Random Forest y SVM.....	99
Figura 38: Entrenamiento del modelo Random Forest y SVM en Python.	100
Figura 39: Exportación de modelos RF y SVM a archivo .pkl.	102
Figura 40: Resultados comparativos entre Random Forest y SVM.	102
Figura 41: Gráfico comparativo de precisión, recall y F1-score.	103
Figura 42: Matriz de confusión – RF.	103
Figura 43: Gráfico comparativo de F1-score.	104
Figura 44: Verificación de instalación de Python y librerías.....	105
Figura 45: Carpeta compartida mostrando los modelos .pkl.....	105
Figura 46: Archivo alert_json.log generado en /var/log/snort/.....	106
Figura 47: Visualización de las alertas generadas.	106
Figura 48: Script ids_realtime.py abierto en editor.	107
Figura 49: Consola mostrando clasificación en tiempo real (varias líneas de salida).....	108
Figura 50: Contenido del archivo alerts_enriched.jsonl.	108
Figura 51: Contenido del archivo alerts_enriched.csv abierto en terminal.	109
Figura 52: Evidencia de detección de ataque simulado (ej. Nmap).	109

INDICE DE ANEXOS

ANEXO 1: Matriz de Consistencia	76
ANEXO 2: Guía de observación técnica estructurada.....	77
ANEXO 3: Ficha de recolección de datos	78
ANEXO 4: Juicio de Experto.....	80
ANEXO 5: Configuración del Entorno Virtual Ubuntu IDS.....	83
ANEXO 6: Instalación y Configuración de Snort	88
ANEXO 7: Validación cruzada con Wireshark.....	94
ANEXO 8: Entrenamiento de Modelos de Machine Learning.....	98
ANEXO 9: Integración del IDS Snort con Machine Learning en Entorno Ubuntu	105

INTRODUCCIÓN

La digitalización ha transformado la manera en que operan las organizaciones en los últimos años, y el sector educativo no es una excepción. La adopción acelerada de herramientas digitales, servicios en la nube, redes internas y dispositivos conectados ha ampliado la superficie de ataque cibernético, poniendo en riesgo la integridad de los datos y la continuidad del negocio. Aunque las instituciones educativas hacen uso de la tecnología con fines pedagógicos y administrativos, muchas veces no tienen políticas de seguridad robustas ni herramientas adecuadas para la detección de intrusos y el monitoreo, convirtiéndose en blancos fáciles para los atacantes.

Los ciberataques en América Latina y en Perú se han multiplicado en número y sofisticación. Según informes recientes, el país se encuentra entre los más atacados de la región, con promedios semanales de intentos de ataque muy por encima del promedio mundial. En este contexto, los colegios tienen redes poco segmentadas, muchos dispositivos IoT y personal no capacitado en ciberseguridad, lo que los hace vulnerables.

Un caso emblemático es el del Colegio Hans Christian Andersen, ubicado en Lima Metropolitana. A pesar de contar con más de 80 dispositivos conectados y una red informática en funcionamiento, su sistema para detectar intrusos se basa únicamente en reglas estáticas. Esta limitación imposibilita la identificación de nuevas amenazas o amenazas complejas, dejando al colegio expuesto a pérdida de información, sanciones legales y pérdida de confianza institucional.

El estudio en cuestión propone, para encarar este problema, que se cree y aplique un sistema de detección híbrido de intrusos que combine Snort, un IDS de código abierto, con técnicas de aprendizaje automático orientadas a identificar comportamientos anómalos. El objetivo es evaluar su eficacia en un entorno real y verificar que pueda implementarse como solución replicable en otras entidades educativas con características similares.

CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA

1.1. Descripción de la Realidad Problemática

El cambio digital ha alterado significativamente la forma de operar de las entidades en todos los campos, incluido el educativo. La dependencia cada vez mayor de plataformas digitales, redes informáticas y servicios conectados ha hecho que la superficie de exposición a ciberamenazas se expanda, lo que ha provocado que los incidentes digitales no solo aumenten en número, sino también en gravedad. La confidencialidad de la información, la continuidad operativa de las instituciones y la disponibilidad de los sistemas se ven comprometidas por estas amenazas. El informe Global Cybersecurity Outlook del Foro Económico Mundial ubica a los ciberataques entre los riesgos sistémicos más grandes del mundo, por su potencial de interrumpir servicios críticos y de causar efectos económicos e institucionales importantes [1].

En este contexto, las entidades educativas aparecen como uno de los sectores con mayor vulnerabilidad. A pesar de que han implementado tecnología rápidamente para propósitos educativos, administrativos y comunicativos, muchas no cuentan con políticas de seguridad digital ni con monitoreo especializado o procedimientos efectivos para identificar amenazas. Esta fragilidad estructural las transforma en blancos regulares para actores malintencionados. En cuanto a América Latina, ha habido un gran aumento en los intentos de ciberataques: el Global Threat Landscape Report [2] indica que se documentaron más de 200 mil millones en el año pasado. La situación también es crítica en Perú. Según Check Point Research, las entidades en Perú se enfrentan a un promedio semanal de 8 818 ataques, lo que es mucho más elevado que el promedio mundial de 1 848 [3]. Por otro lado, ESET Latinoamérica reportó más de 900 000 amenazas cibernéticas en seis meses, que incluían ransomware, malware y ataques de ingeniería social [4].

Esta realidad ha tenido un impacto significativo en el sector educativo. Instituciones educativas de varios niveles funcionan con redes no segmentadas, numerosos dispositivos conectados, plataformas digitales sin la protección apropiada y una formación débil en ciberseguridad. De acuerdo con el informe técnico de Tarlogic, esta combinación de factores hace que el ambiente educativo sea un objetivo extremadamente susceptible ante ataques sigilosos y amenazas constantes [5]. Las repercusiones pueden variar desde la suspensión del servicio educativo hasta la exposición de información personal, lo que conlleva consecuencias legales y de reputación.

Gran cantidad de instituciones educativas privadas en Lima Metropolitana lidian con esta problemática de manera directa. A pesar de que han implementado tecnologías para la administración y gestión académica, la mayor parte no tiene sistemas de seguridad perimetral sólidos ni instrumentos que hagan posible detectar intrusos a tiempo. uso cada vez más frecuente de dispositivos interconectados, redes sin segmentación y personal no especializado en tecnologías de la información agrava esta circunstancia. uso cada vez más frecuente de dispositivos interconectados, redes sin segmentación y personal no especializado en tecnologías de la información agrava esta circunstancia.

El Colegio Hans Christian Andersen, que se encuentra en el distrito de Santiago de Surco, es un ejemplo representativo. La institución dispone de más de 80 dispositivos conectados a su red interna, que incluyen proyectores, cámaras y sensores. No obstante, no tiene un sistema sofisticado para detectar intrusos. El IDS actual, que se basa en reglas fijas, no tiene la habilidad de reconocer amenazas desconocidas ni de adecuarse a patrones nuevos. Esta restricción técnica expone a la institución ante ataques continuos, pérdida de información sensible y accesos no permitidos. Las repercusiones podrían ser severas: se calcula que si 12 familias (que representan el 2% de los estudiantes) pierden la confianza, podría haber una pérdida económica directa de S/150,000 al año. Asimismo, si se infringe la Ley N.º 29733 sobre protección de datos personales [6], INDECOPI podría imponer sanciones que llegarían hasta S/ 60,000. Los costos técnicos de mitigación variarían entre S/ 25,000 y S/ 35,000.

Este escenario ilustra que los riesgos asociados a la ciberseguridad en instituciones educativas no solo comprometen aspectos técnicos, sino que también tienen un impacto directo en la sostenibilidad financiera y en la confianza de la comunidad educativa, lo que refuerza la necesidad de soluciones técnicas avanzadas como los sistemas híbridos de detección de intrusos.

La situación descrita no es exclusiva de esta institución. Otros centros educativos privados con estructuras tecnológicas similares enfrentan riesgos. Por ello, es fundamental analizar esta problemática desde una perspectiva técnica, operativa y contextual, que permita identificar vulnerabilidades reales y validar soluciones eficaces y replicables para el sector educativo privado en Lima Metropolitana.

1.2. Formulación del Problema

1.2.1. Problema General

¿Cómo mejora la integración de técnicas de Machine Learning en un sistema IDS basado en Snort la detección de amenazas y la reducción de la exposición en redes perimetrales de instituciones educativas privadas, considerando el caso del Colegio Hans Christian Andersen?

1.2.2. Problemas Específicos

- ¿Cuáles son las principales vulnerabilidades en las redes perimetrales de las instituciones educativas privadas de nivel escolar en Lima Metropolitana que no son detectadas por los sistemas de detección de intrusos tradicionales?
- ¿Qué limitaciones presenta Snort, como sistema basado en reglas estáticas, frente a patrones de ciberataques modernos en el contexto educativo?
- ¿Cómo pueden las técnicas de Machine Learning complementar a Snort para identificar y clasificar amenazas emergentes en entornos escolares?
- ¿Cuál es el nivel actual de exposición a ciberataques en las redes perimetrales del Colegio Hans Christian Andersen y qué riesgos concretos enfrenta en ausencia de detección inteligente?

1.3. Determinación de Objetivos

1.3.1. Objetivo General

Implementar un sistema de detección de intrusos híbrido, basado en Snort y técnicas de Machine Learning, que permita mejorar la tasa de detección de amenazas, reducir falsos positivos y optimizar el tiempo de respuesta en la red perimetral del Colegio Hans Christian Andersen, como modelo aplicable a instituciones educativas privadas escolares de Lima Metropolitana.

1.3.2. Objetivos Específicos

- Identificar las principales vulnerabilidades presentes en las redes perimetrales de instituciones educativas privadas escolares de Lima Metropolitana que no son detectadas por sistemas IDS tradicionales.
- Analizar las limitaciones técnicas del sistema Snort, como motor de detección basado en reglas, frente a patrones de ciberataques avanzados en entornos

educativos.

- Diseñar e integrar técnicas de Machine Learning supervisadas como complemento a Snort, orientadas a mejorar la tasa de detección y reducir los falsos positivos en tiempo real.
- Evaluar la efectividad del sistema híbrido propuesto mediante su implementación experimental en la red perimetral del Colegio Hans Christian Andersen, midiendo métricas como tasa de detección, precisión y tiempo de respuesta.

1.4. Hipótesis

1.4.1. Hipótesis General

La implementación de un sistema de detección de intrusos híbrido, basado en Snort y técnicas de Machine Learning, mejorará significativamente el desempeño en la detección de intrusos en la red perimetral del Colegio Hans Christian Andersen, incrementando la tasa de detección, reduciendo los falsos positivos y optimizando el tiempo de respuesta.

1.4.2. Hipótesis Específicas

- Las redes perimetrales de instituciones educativas privadas escolares de Lima Metropolitana presentan vulnerabilidades no detectadas por sistemas IDS tradicionales, lo cual se evidencia en una tasa de detección menor al 80 % y una tasa de falsos negativos superior al 15 % ante ataques conocidos.
- El sistema Snort, en su configuración convencional basada en firmas, presenta una precisión inferior al 75 % en la detección de amenazas emergentes, lo que limita su eficacia frente a patrones de ataque avanzados en entornos escolares.
- La integración de algoritmos de Machine Learning con Snort incrementa la tasa de detección en al menos un 20 % y reduce los falsos positivos en un 15 %, en comparación con la configuración estándar de Snort, permitiendo una detección más eficiente de comportamientos anómalos en tiempo real.
- La aplicación del sistema híbrido propuesto en la red perimetral del Colegio Hans Christian Andersen alcanza una tasa de detección superior al 90 %, una

tasa de falsos negativos inferior al 10 %, y un tiempo de respuesta promedio menor a 2 segundos, mejorando los indicadores clave de seguridad perimetral.

1.5. Justificación de la Investigación

1.5.1. Teórica

Desde el plano académico, esta investigación aporta al cuerpo teórico de la ciberseguridad al explorar la integración de sistemas tradicionales de detección de intrusos basados en firmas como Snort con algoritmos de Machine Learning para la detección de comportamientos anómalos. Aunque hay antecedentes sobre la aplicación de inteligencia artificial en la seguridad de redes, las investigaciones que fusionan ambos puntos de vista en contextos escolares latinoamericanos son pocas. En estos contextos, el tráfico de red, los recursos y los perfiles de usuario tienen rasgos específicos. El desarrollo y análisis de un modelo híbrido puede fortalecer la bibliografía científica, estableciendo cimientos para investigaciones futuras en ámbitos educativos.

1.5.2. Práctica

La tesis, en términos operativos, aborda una necesidad crucial del sector educativo privado: salvaguardar las redes escolares que están expuestas a riesgos sin incurrir en altos costos de licenciamiento. La solución sugerida tiene como objetivo ser una opción que sea accesible, replicable y efectiva para escuelas con recursos escasos. Esta se apoya en Snort como un IDS de código abierto que se fortalece a través de técnicas de Machine Learning. La ejecución ocurrirá en el Colegio Hans Christian Andersen, donde se llevará a cabo una evaluación de su rendimiento en condiciones reales. Esta experiencia piloto será la base para que en el futuro se adapte a otras entidades educativas con una infraestructura parecida, lo cual robustecerá el ecosistema de seguridad digital en el ámbito escolar.

1.5.3. Metodológica

Desde un punto de vista metodológico, el proyecto emplea un enfoque cuantitativo aplicado que facilita una evaluación rigurosa del desempeño del sistema propuesto. La magnitud del impacto de la integración de Machine Learning y Snort en la detección de amenazas se evaluará mediante medidas como el tiempo de respuesta, los falsos positivos y la tasa de detección. Este método asegura no solo la validez de los resultados en el contexto analizado,

sino también que se puedan reproducir, lo cual posibilitará a investigadores posteriores replicar el experimento en otros entornos educativos y validar su eficiencia comparativa.

1.5.4. Tecnológica

Es importante aclarar que el objetivo de esta investigación no es comparar con productos comerciales, como Fortinet, Cisco Secure IDS o Palo Alto, sino evidenciar la factibilidad técnica de fortalecer herramientas de código abierto a través de métodos de inteligencia artificial, particularmente en situaciones donde existen restricciones presupuestarias. Snort ha sido escogido por su estabilidad, versatilidad y comunidad activa, además de brindar acceso total al código fuente, lo cual simplifica que se integre con modelos personalizados de aprendizaje automático. A diferencia de las plataformas comerciales que implican altos costos de licenciamiento y acceso limitado a su arquitectura interna, esta propuesta busca ofrecer una alternativa escalable, sostenible y adecuada a las condiciones reales de instituciones educativas privadas en Lima Metropolitana.

1.6. Delimitación del Estudio

1.6.1. Delimitación Espacial

La presente investigación se desarrollará en el Colegio Hans Christian Andersen, institución educativa privada ubicada en el distrito de Santiago de Surco, Lima Metropolitana. Este colegio ha sido seleccionado como unidad de análisis por contar con una infraestructura tecnológica en crecimiento, múltiples dispositivos conectados a su red interna y un interés institucional por fortalecer su seguridad informática. El estudio se centrará exclusivamente en su red perimetral, la cual servirá como entorno real para la implementación y validación del sistema de detección de intrusos propuesto. Si bien el caso es particular, los hallazgos podrán orientar intervenciones similares en otras instituciones educativas privadas con características comparables.

1.6.2. Delimitación Temporal

El proyecto de investigación se ejecutará durante el periodo académico establecido para el desarrollo de la tesis. En este lapso se llevarán a cabo todas las etapas del estudio: configuración, adaptación e implementación del sistema IDS híbrido (basado en Snort y técnicas de Machine Learning), así como la recolección y análisis de datos de tráfico real. Todas las actividades se ajustarán al cronograma institucional y serán documentadas para garantizar la trazabilidad

del proceso.

1.6.3. Delimitación Conceptual

La investigación se sitúa conceptualmente en el ámbito de la ciberseguridad, más concretamente en la creación y evaluación de sistemas IDS que incorporan algoritmos de aprendizaje automático. Se abordarán términos técnicos como redes perimetrales, tráfico malicioso, ataques cibernéticos, detección de anomalías, clasificación de eventos y métricas de rendimiento. No se contemplan aspectos relacionados con políticas de seguridad institucional, ni con dimensiones legales, económicas o sociales, salvo lo estrictamente necesario para contextualizar el problema. En consecuencia, el alcance de la investigación se limita al análisis técnico del sistema propuesto y a la validación de su desempeño en un entorno escolar específico, sin pretender abarcar otras dimensiones de la ciberseguridad institucional.

CAPÍTULO II: MARCO TEÓRICO

2.1. Antecedentes

El estudio de Figueroa y Cañihua [7], realizado en una empresa privada de Lima, propuso un sistema de detección de intrusos basado en Machine Learning, combinando algoritmos como Árboles de Decisión y K-Nearest Neighbors sobre un entorno LAN. A pesar de que reporta una alta precisión al clasificar el tráfico anómalo, presenta dos limitaciones significativas para este trabajo: en primer lugar, no se centra en la industria educativa; en segundo, no trata sobre la combinación con herramientas de código abierto como Snort. A pesar de eso, su enfoque metodológico es útil como guía para el preprocesamiento, la elección de atributos y la formación de modelos.

Huamani Santiago [8], por su parte, realizó la implementación de Snort en redes de pequeñas y medianas empresas (PYMES) en Perú y comprobó que, si se configura correctamente, es capaz de identificar tráfico dañino en infraestructuras limitadas. No obstante, el estudio se limita a la aplicación de reglas estáticas, sin incluir inteligencia artificial o detección adaptativa. Esta limitación brinda una oportunidad de investigación: reforzar Snort con elementos de ML que extiendan la cobertura ante ataques nuevos, sobre todo en contextos escolares con tráfico variable.

Lascano Banshuy [9] llevó a cabo una investigación en una escuela de Ecuador, empleando redes neuronales artificiales para identificar patrones anómalos. Este precedente se ajusta muy bien a la problemática de la tesis porque examina el contexto escolar y valida la capacidad de la IA para detectar intrusiones. Sin embargo, no incluye Snort ni analiza sinergias entre reglas y modelos predictivos, lo que deja una brecha técnica que este trabajo intenta llenar a través de un enfoque híbrido.

Carrizo García y Álvarez Galarza [10] unieron software libre con métodos de aprendizaje automático para mejorar la detección en redes genéricas. A pesar de que la investigación muestra avances significativos en la tasa de detección al implementar el aprendizaje automático, el estudio no se aplica a redes educativas ni estudia las características del tráfico escolar (picos por horarios, varios IoT, aplicaciones académicas), lo que restringe su aplicación contextual. Sin embargo, proporciona pruebas de que la integración Snort-ML es factible tanto técnica como operativamente.

A su vez, Caizapanta González [28] llevó a cabo un estudio comparativo de varias IDS en el entorno universitario de América Latina y determinó que las soluciones de código

abierto son factibles para instituciones con recursos escasos; sin embargo, necesitan modificaciones para adaptarse a los patrones locales del tráfico académico. Siguiendo ese mismo rumbo, Suárez y Rodríguez [36] realizaron un estudio exhaustivo de los IDS y de los algoritmos más comunes en la seguridad de la información. Indicaron que es necesario combinar métodos basados en firmas con técnicas de aprendizaje automático para abordar con mayor exactitud las amenazas emergentes.

En síntesis, la literatura revisada muestra progresos significativos en la aplicación de Machine Learning para detectar intrusos y realizar despliegues con Snort en entornos con recursos escasos. No obstante, no se encuentra una investigación que combine ambas perspectivas y las implemente empíricamente en escuelas privadas de Lima Metropolitana. La meta de esta tesis es validar empíricamente un sistema híbrido Snort + ML, que se adecue a la situación operacional del Colegio Hans Christian Andersen y que pueda ser reproducido en otras instituciones educativas semejantes. Esta brecha lo justifica.

2.2. Estado del Arte

En la última década, ha adquirido mucha importancia el desarrollo de sistemas de detección de intrusos (IDS) que utilizan aprendizaje automático, con una multitud de enfoques dirigidos a mejorar la capacidad de respuesta ante ataques cibernéticos en redes cada vez más complejas. El trabajo de Aeraj y Leghris [11] es sobresaliente entre las soluciones emergentes; estos autores desarrollaron un IDS inteligente que fusiona Snort con una arquitectura de red neuronal de soporte vectorial (SVM). La integración del análisis estadístico de tráfico y la clasificación supervisada permite que su propuesta aumente los niveles de precisión en la detección de intrusos. A pesar de que la investigación no se enfoca en entidades educativas, muestra que la combinación de Snort con modelos de aprendizaje automático mejora la sensibilidad frente a amenazas sofisticadas, lo cual pone de manifiesto una línea prometedora para ambientes con infraestructura crítica.

La literatura más reciente también indica que la reducción de la dimensionalidad previa al clasificador optimiza no solo el tiempo de cómputo, sino también la separación entre clases. El uso de SVD/PCA como una fase preliminar para IDS basados en ML ha sido confirmado [35]; además, al combinar PCA con ensambles variables que se ajustan al espacio de características, se ha logrado un aumento adicional [37]. Estos hallazgos son particularmente provechosos cuando se manejan conjuntos de características amplios y correlacionados.

En el plano nacional, OSIPTEL (2023) advierte que las instituciones educativas se encuentran entre los sectores más vulnerables a incidentes de ciberseguridad por la baja inversión en protección de redes. Asimismo, Indecopi ha resaltado en sus informes anuales la necesidad de garantizar la seguridad de datos sensibles de estudiantes en entornos digitales. Estos hallazgos locales complementan estudios internacionales como los de Lascano Banshuy [9] en Ecuador o AbdulRaheem et al. [13] en entornos SDN.

De manera transversal, las revisiones sistemáticas recientes sintetizan ataques, algoritmos y retos en IDS con ML, y recomiendan pipelines reproducibles con métricas robustas (Precision, Recall, F1, PR-AUC/ROC-AUC) y validaciones cuidadosas [31]. Además, para contextos con datos personales (como escuelas), se propone privacidad por diseño (hash/salting de IP, minimización y retención limitada) sin degradar el rendimiento del detector [33]. En dominios sensibles (p. ej., IoMT), también se han consolidado marcos de seguridad que pueden trasladarse a redes educativas por su similitud en el tratamiento de información sensible [37].

Desde una perspectiva de múltiples agentes, Barramou, Addou y Ouiazzane [12] crearon un sistema híbrido de detección para ataques que son conocidos y desconocidos, incorporando modelos de aprendizaje automático con agentes inteligentes. El sistema muestra una eficacia elevada en la identificación de ataques de día cero, además de capacidad de adaptación a alteraciones dinámicas en el tráfico de red. Para los entornos educativos, este enfoque es pertinente por la diversidad y la exposición continua a nuevas amenazas digitales. A pesar de que el sistema no recurre a Snort directamente, sus principios metodológicos tienen la posibilidad de ser reproducidos en arquitecturas que fusionen componentes adaptativos fundamentados en inteligencia artificial con sistemas de detección de intrusos convencionales.

Al mismo tiempo, las fusiones o ensambles de modelos han demostrado que pueden estabilizar las métricas en situaciones variables y con desequilibrio de clases, disminuyendo el número de falsos positivos al fusionar los resultados de varios clasificadores [35]. Los pipelines de big data IDS evidencian que, para flujos de gran volumen, el preprocesamiento sólido y la selección/filtrado de variables ayudan a conservar la sensibilidad sin aumentar las alertas ruidosas [29].

En el marco de redes definidas por software (SDN), AbdulRaheem et al. [13] plantearon un sistema de detección de intrusos que fusiona las habilidades de Snort y Zeek con

algoritmos de aprendizaje supervisado, diseñado para identificar ataques DDoS (de denegación distribuida del servicio). Los resultados indican que la fusión de herramientas tradicionales con modelos de Machine Learning puede reducir significativamente los falsos positivos y mejorar la capacidad de respuesta ante ataques masivos. Aunque su aplicación está centrada en entornos SDN, el principio de colaboración entre motores IDS y algoritmos predictivos resulta extrapolable a redes escolares que, si bien menos complejas, comparten la necesidad de detectar ataques en tiempo real con precisión operativa.

En entornos con fuerte presencia de IoT, típicos de colegios con CCTV, proyectores y sensores, los modelos XGBoost han mostrado buen balance entre rapidez e impacto en Recall bajo desbalance [30], mientras que enfoques de deep learning (p. ej., CNN) elevan la detección cuando hay capacidad de cómputo y datos suficientes [34].

Sun, Zhang, Tang y Qin [14] contribuyen al campo tecnológico con un método automatizado para crear patrones concretos de detección de intrusos a través del aprendizaje automático. Su propuesta trata una de las debilidades más arraigadas en los sistemas IDS basados en firmas: su limitada capacidad para adaptarse a nuevos vectores de ataque. Con técnicas de agrupamiento y clasificación, consiguen crear firmas dinámicas con un alto grado de precisión, lo cual significa un progreso en comparación con las configuraciones manuales de reglas. A pesar de que el estudio no incluye Snort de manera explícita, sus descubrimientos tienen la posibilidad de ser incorporados como un añadido a su arquitectura, lo que mejoraría su habilidad para detectar en ambientes educativos con tráfico variable.

En ataques web y capa de aplicación, la inclusión de features HTTP/URL y rasgos de sesión mejora la detección de patrones “low-and-slow” y exfiltración respecto a enfoques basados solo en firmas [27]. La correlación de alertas de Snort se combina adecuadamente con este tipo de características.

Daniel et al. [15] establecen una comparación entre modelos de Machine Learning y modelos de lenguaje (LLMs) con el objetivo de clasificar reglas IDS según el marco MITRE ATT&CK, en una línea que está dirigida a la taxonomía de las amenazas. Los autores concluyen que, aunque los LLM han progresado, los modelos supervisados continúan siendo más efectivos para la identificación y clasificación de tácticas y técnicas de ataque. Esta investigación es muy útil para proyectos que combinan Snort y Machine Learning, porque presenta un marco formal de evaluación y organización de

reglas de detección. Esto podría aumentar la trazabilidad y cobertura del sistema propuesto en esta tesis.

Alserhani [16] propone, por último, un esquema para crear conjuntos de datos extensos, representativos y escalables que se utilizarán en la validación de IDS. El escritor enfatiza que la falta de conjuntos de datos realistas es uno de los obstáculos más importantes para la evaluación eficaz de soluciones fundamentadas en aprendizaje automático. Su propuesta hace posible la creación de escenarios de prueba adaptados a varios tipos de ataques y tráfico, lo que posibilita una validación más exacta. Para el estudio actual, este marco podría ser utilizado como un referente metodológico en la etapa experimental, al momento de evaluar la eficacia del sistema híbrido en el entorno real del colegio Hans Christian Andersen.

En la práctica, el estado del arte indica una canalización híbrida que empieza utilizando un motor de firmas como Zeek o Snort para etiquetar previamente el tráfico y realizar un primer filtrado eficaz: lo conocido se reconoce con rapidez y con poca latencia, mientras que lo incierto pasa a una segunda fase de análisis. Sobre ese flujo preprocesado se extraen características de flujo y de sesión (duración, bytes/paquetes, ráfagas, patrones de petición/respuesta, rasgos HTTP/URL, etc.), que capturan el comportamiento real de los intercambios de red. Dado que este espacio de rasgos suele ser amplio y estar correlacionado, la literatura recomienda reducir y/o seleccionar dimensionalidad —por ejemplo, con PCA o SVD— para mejorar la separación entre clases, estabilizar el entrenamiento y, sobre todo, bajar la carga computacional en inferencia sin perder poder de discriminación [25], [26].

Con ese set de rasgos depurado se entrena y calibra el modelo de detección. Aquí funcionan bien clasificadores de árboles (p. ej., Random Forest) y ensambles graduales como XGBoost por su robustez ante no linealidades y desbalance; cuando se dispone de datos y cómputo, también se incorporan modelos de deep learning para elevar la sensibilidad ante patrones complejos. En escenarios variables (IoT, tráfico escolar con picos), los ensambles o fusiones de modelos ayudan a estabilizar métricas y a mitigar el riesgo de sobreajuste a una sola vista del dato [35]. Una vez calibrado (vía Platt o isotónica), el sistema aplica umbrales operativos coherentes con la tolerancia al falso positivo de la institución y añade una capa de correlación (por ventana temporal, por sesión o por host) para priorizar lo realmente urgente, reduciendo ruido especialmente en ataques web y exfiltraciones “low-and-slow” donde los rasgos HTTP/URL marcan la diferencia [19].

Finalmente, porque el tráfico cambia en el tiempo, se monitoriza la deriva del modelo (con indicadores como PSI/KS o degradación de métricas) y se planifica el reentrenamiento cuando corresponda, manteniendo la efectividad en producción. Todo esto se acompaña de privacidad por diseño: anonimización (hash/salting) de IPs internas, minimización y retención limitada de datos, y controles de acceso, de modo que se proteja la PII sin sacrificar desempeño [33]. Para reforzar estas garantías, es útil apoyarse en lineamientos sectoriales de dominios sensibles como los propuestos para loMT que son transferibles a entornos escolares por su foco en datos personales y continuidad operativa [37].

En resumen, el análisis de los trabajos revisados evidencia un progreso significativo en la incorporación de algoritmos de Machine Learning y herramientas como Snort para mejorar la detección de intrusos en diversas arquitecturas de red. Sin embargo, persiste una brecha notoria en la aplicación conjunta de estas tecnologías dentro de instituciones educativas privadas de nivel escolar, especialmente en contextos peruanos. La ausencia de estudios que validen empíricamente un enfoque híbrido en entornos reales, con tráfico auténtico y condiciones operativas propias de los colegios, justifica plenamente la necesidad de esta investigación. El presente estudio se propone llenar ese vacío, evaluando la eficacia de una solución combinada que integre Snort y aprendizaje automático, adaptada específicamente al entorno del Colegio Hans Christian Andersen.

2.3. Bases Teóricas

Los principios básicos sobre los que se basa el sistema de detección de intrusos propuesto son explicados en la investigación actual, la cual se apoya en múltiples marcos teóricos interconectados. Estos marcos abordan conceptos claves de ciberseguridad, sistemas IDS, técnicas de aprendizaje automático y arquitectura de redes perimetrales en instituciones educativas.

Como marco general, la literatura especializada en seguridad refuerza la triada confidencialidad–integridad–disponibilidad y la necesidad de acoplar los controles técnicos con procesos de gobierno y medición para que la protección sea sostenible en organizaciones heterogéneas como los colegios privados [39]. A la par, revisiones recientes sintetizan familias de ataques, algoritmos y retos de evaluación en IDS con aprendizaje automático, y recomiendan integrar motores basados en firmas con modelos de ML para ampliar cobertura y reducir falsos positivos bajo condiciones reales de red [31].

2.3.1. Ciberseguridad y amenazas a redes educativas

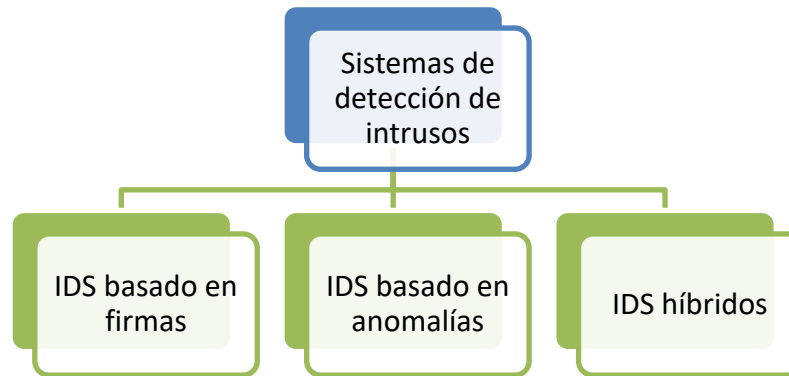
La ciberseguridad se refiere a la protección de los sistemas informáticos frente a accesos no autorizados, daños o interrupciones causadas por ataques digitales. En el contexto educativo, las redes internas suelen contener datos sensibles de estudiantes, docentes y personal administrativo, lo que las convierte en blancos frecuentes para ciberataques. Según Check Point Research, el sector educativo es uno de los más atacados a nivel global, con más de 2,314 intentos de ciberataques semanales por institución en 2023 [3]. En Perú, las amenazas han incrementado considerablemente, ocupando el primer lugar en Latinoamérica en volumen de ataques detectados [4].

En este contexto, es clave adoptar privacidad por diseño en cualquier solución que procese datos de estudiantes y personal: anonimización de identificadores (p. ej., hash/salting de IP internas), minimización y retención limitada de datos, además de controles de acceso. La evidencia en entornos IoT muestra que estas salvaguardas pueden incorporarse sin deteriorar el desempeño del detector cuando se planifican desde el diseño del pipeline [33]. A ello se suman lineamientos de dominios altamente sensibles como el Internet of Medical Things que son transferibles a redes escolares por su foco en continuidad operativa y protección de información personal [37].

2.3.2. Sistemas de detección de intrusos (IDS) y Snort como herramienta de código abierto

Los sistemas de detección de intrusos (IDS, por sus siglas en inglés) son herramientas diseñadas para identificar comportamientos anómalos o accesos no autorizados dentro de una red. Se clasifican comúnmente en dos categorías principales: los IDS basados en firmas, que comparan el tráfico con patrones conocidos registrados previamente, y los IDS basados en anomalías, que utilizan modelos estadísticos o de inteligencia artificial para detectar desviaciones respecto al comportamiento normal [39], [31]. Cada enfoque presenta ventajas y limitaciones: mientras que los sistemas basados en firmas son rápidos y precisos ante amenazas conocidas, suelen fallar frente a ataques nuevos; por el contrario, los sistemas basados en anomalías ofrecen adaptabilidad, pero pueden generar más falsos positivos si no están bien entrenados [31].

Figura 1: Clasificación de IDS



Fuente: (Elaboración propia)

Snort es uno de los IDS más reconocidos y se trata de una herramienta de código abierto que fue creada inicialmente por Sourcefire, aunque en la actualidad está bajo el mantenimiento de Cisco. Snort opera principalmente como un sistema de detección de intrusiones (IDS) fundamentado en firmas, lo que posibilita la inspección minuciosa de paquetes, el registro de eventos y la creación de alertas a través de normas establecidas por el usuario o la comunidad. La compatibilidad con varios sistemas operativos, la arquitectura modular y el bajo coste de su puesta en marcha lo han establecido como una alternativa perfecta para instituciones educativas y otras organizaciones con recursos escasos [17].

No obstante, debido a que es un sistema que solo se basa en reglas estáticas, Snort tiene restricciones técnicas al enfrentarse a amenazas nuevas o patrones de ataque que no han sido documentados con anterioridad. Esta rigidez en términos operativos exige actualizaciones regulares y podría ser insuficiente en situaciones con tráfico dinámico, como sucede en los entornos escolares digitalizados [14]. Esta limitación ha llevado a que se presenten propuestas recientes para combinar Snort con métodos de aprendizaje automático, los cuales hacen posible extender sus habilidades mediante la identificación inteligente de conductas anómalas no registradas en las firmas tradicionales [13].

La literatura reciente coincide en que un enfoque híbrido (reglas + ML) compensa las limitaciones de los IDS puramente basados en firmas y mejora la detección de amenazas no vistas cuando se evalúa con métricas apropiadas (Precision, Recall, F1 y curvas PR/ROC) y procedimientos reproducibles [31]. En escenarios de alto volumen de tráfico, los estudios de big-data IDS subrayan que un

preprocesamiento robusto y la selección/filtrado de variables permiten sostener la sensibilidad sin disparar el ruido de alertas [29]. Además, cuando los vectores transitan por la capa de aplicación, la incorporación de rasgos HTTP/URL y de sesión complementa las firmas de Snort y mejora la detección de patrones “low-and-slow” y exfiltraciones [27].

2.3.3. Machine Learning en detección de intrusos y enfoques híbridos con Snort

El aprendizaje automático (Machine Learning, ML) ha demostrado ser una tecnología eficaz para identificar patrones complejos en el tráfico de red y detectar actividades anómalas sin necesidad de reglas predefinidas. Algoritmos como Random Forest, Support Vector Machines (SVM) y Redes Neuronales Artificiales (ANN) han sido aplicados con éxito en sistemas de detección de intrusos, tanto en entornos simulados como en escenarios reales, logrando tasas elevadas de precisión y adaptabilidad [14], [16].

Operativamente, el estado del arte recomienda un pipeline que inicia con pre-etiquetado por un motor de firmas (p. ej., Snort/Zeek), continúa con la extracción de rasgos de flujo/sesión y aplica reducción/selección de dimensionalidad antes del modelado. En esta etapa, PCA y SVD han demostrado mejorar la separación de clases y la eficiencia de entrenamiento/inferencia en IDS basados en ML [25], y estos beneficios se refuerzan cuando PCA se combina con ensambles variables que estabilizan el rendimiento ante multicolinealidad y ruido [26].

A diferencia de los IDS tradicionales, los modelos basados en ML pueden aprender el comportamiento normal del sistema y detectar desviaciones que sugieran intrusiones, incluso si no se cuenta con firmas previas. Esta capacidad resulta especialmente útil en redes educativas, donde el tráfico es variado, dinámico y potencialmente vulnerable [9].

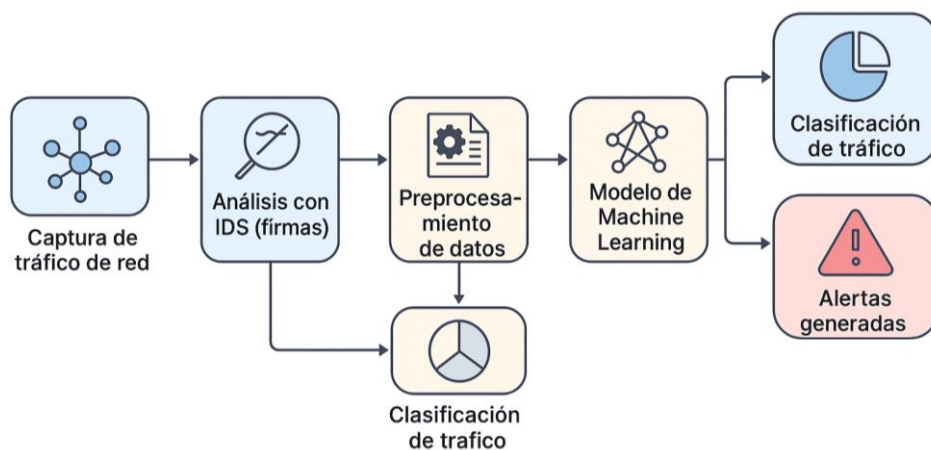
En la elección del modelo, los ensambles de árboles tipo XGBoost ofrecen un buen balance entre latencia y recall en perímetros con fuerte presencia de IoT y desbalance de clases [30]; cuando hay datos y cómputo suficientes, el deep learning por ejemplo, CNN sobre representaciones de flujo incrementa la sensibilidad a patrones complejos [34]. Asimismo, los enfoques fusionados que combinan salidas de varios clasificadores ayudan a estabilizar métricas y a reducir falsos positivos en escenarios variables como los escolares [35]. Para vectores web, los rasgos HTTP/URL y de sesión aportan poder discriminante

adicional más allá de las señales de firmas [27].

Las investigaciones más recientes han propuesto integrar ML con herramientas como Snort, generando sistemas híbridos que combinan lo mejor de ambos enfoques: la eficiencia de las reglas basadas en firmas y la adaptabilidad del aprendizaje automático. AbdulRaheem et al. [13] lograron detectar ataques DDoS en redes definidas por software combinando Snort con Zeek y clasificadores ML, alcanzando tasas de detección superiores al 95 %. Por su parte, Daniel et al. [15] mejoraron la organización de reglas NIDS aplicando técnicas supervisadas basadas en el marco MITRE ATT&CK, incrementando la correlación entre alertas y tácticas reales de ataque.

La conveniencia de incluir PCA/SVD y variantes de ensamble en el pipeline híbrido está respaldada por resultados empíricos recientes [25], [26], mientras que su robustez en entornos IoT y escenarios desbalanceados se apoya en la evidencia reportada con XGBoost y DL [30], [34], así como en enfoques fusionados que estabilizan el desempeño [35].

Figura 2: Arquitectura de un IDS híbrido



Fuente: (Elaboración propia)

Estas propuestas fortalecen la hipótesis de este trabajo: un sistema híbrido Snort-ML tiene el potencial de sobrepasar las restricciones de los IDS tradicionales, brindando una solución eficiente y factible para proteger redes educativas que cuentan con recursos escasos y exigencias elevadas de detección en tiempo real.

2.3.4. Particularidades de las redes educativas perimetrales

Las redes en instituciones educativas de nivel escolar presentan características distintivas: múltiples dispositivos IoT (cámaras, sensores, proyectores), infraestructura limitada, personal con bajo nivel de formación en ciberseguridad y picos de tráfico asociados a actividades académicas. Estas condiciones exigen soluciones de seguridad adaptadas, que combinen bajo costo, facilidad de despliegue y capacidad de adaptación. Estudios como el de Lascano Banshuy [9] evidencian que las redes escolares son vulnerables a múltiples vectores de ataque y que la incorporación de sistemas inteligentes es una necesidad impostergable.

Dadas estas particularidades, conviene priorizar modelos ligeros y explicables que toleren desbalance e intermitencia del flujo. En la práctica, XGBoost ha mostrado buen rendimiento en perímetros con múltiples dispositivos IoT, mientras que CNN y otros enfoques de deep learning elevan la tasa de detección cuando se dispone de datos y capacidad de cómputo [30], [34]. Para contener el costo operativo sin perder discriminación, es recomendable aplicar PCA/SVD en la representación y, cuando se requiera mayor robustez, recurrir a ensambles/fusiones que estabilicen métricas frente a picos horarios y cambios de patrón [25], [26], [35]. Además, dado el predominio del tráfico web en actividades escolares, resulta pertinente incluir features HTTP/URL y de sesión en la capa de modelado [27].

2.3.5. Justificación de indicadores técnicos de evaluación

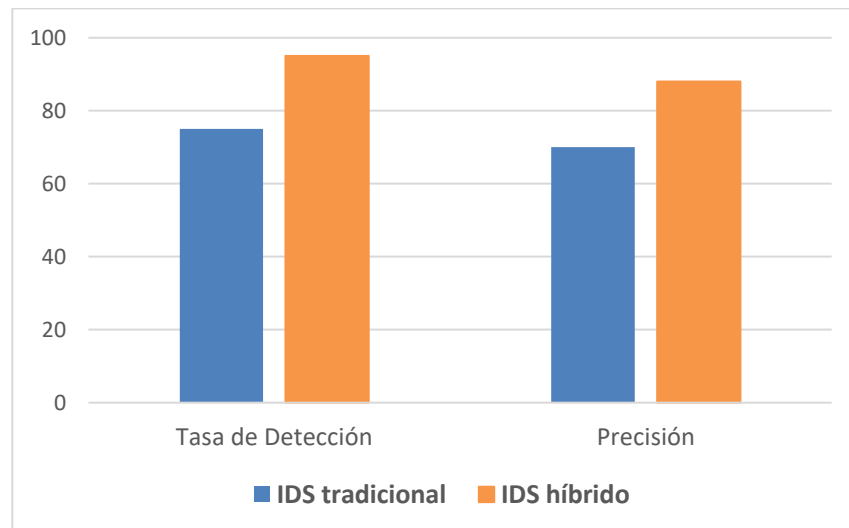
Para evaluar la efectividad del sistema de detección propuesto, se adoptan métricas comúnmente utilizadas en investigaciones recientes sobre IDS híbridos: precisión, recall, tasa de detección, falsos positivos y tiempo de respuesta. Estas métricas permiten validar si el sistema identifica correctamente las amenazas, reduce errores de clasificación y responde oportunamente.

Estudios recientes indican que, si bien Snort es eficaz frente a ataques conocidos, presenta limitaciones frente a amenazas no documentadas. En estos escenarios, su tasa de detección puede caer por debajo del 75 % y los falsos negativos superar el 20 % [13]. Con el objetivo de superar estas deficiencias, varios estudios han sugerido incorporar métodos de Machine Learning, que tienen la capacidad de adaptarse a patrones de tráfico no registrados y aumentar su cobertura frente a ataques emergentes [14]. Como han reportado

AbdulRaheem et al. [13], esta integración ha demostrado avances importantes, logrando tasas de detección mayores al 90 % y disminuyendo los falsos negativos por debajo del 10 % en redes con gran cantidad de datos. Chio y Freeman [18] subrayan, en la misma dirección, que la importancia del Machine Learning en ciberseguridad se encuentra en su habilidad para identificar patrones desconocidos, ajustarse de manera dinámica al tráfico y minimizar las deficiencias de los IDS convencionales.

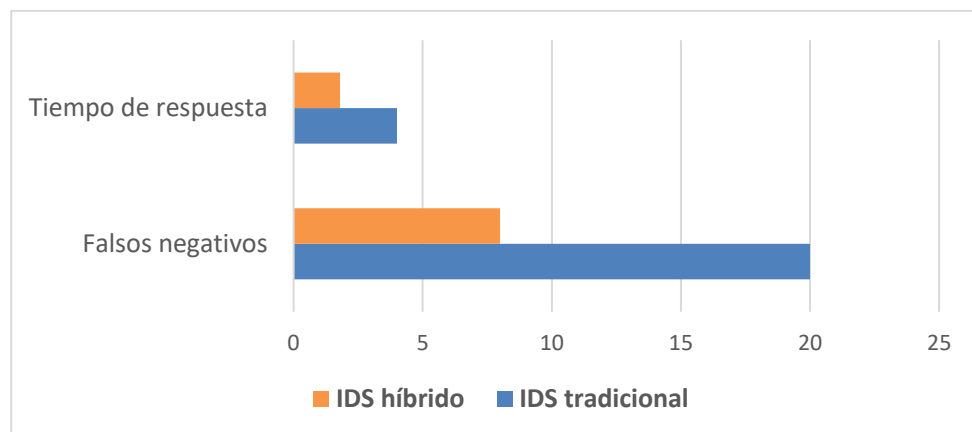
Por eso, esta tesis establece como objetivos operativos: una tasa de detección mayor o igual al 90 %, una precisión mínima del 85 %, menos del 10 % de falsos negativos y un tiempo de respuesta menor a los 2 segundos. Estos valores guían el diseño y la validación del sistema híbrido sugerido.

Figura 3: Desempeño entre IDS vs. IDS + ML



Fuente: (Elaboración propia)

Figura 4: Comparación de errores y latencia de IDS



Fuente: (Elaboración propia)

Para resumir, los fundamentos teóricos respaldan la combinación de IDS con métodos de Machine Learning como una estrategia híbrida efectiva para enfrentar las limitaciones de los IDS convencionales. Este marco ayuda a entender los retos técnicos de las redes escolares y establece los indicadores clave para la evaluación. La construcción metodológica de la investigación es guiada por los conceptos desarrollados, que se organizan en el siguiente mapa conceptual.

2.4. Marco Conceptual

Para que esta investigación sea más comprensible, se presentan a continuación las definiciones de los conceptos centrales vinculados con el reconocimiento de intrusiones en redes informáticas y la aplicación de técnicas de Machine Learning. Estas definiciones se han ajustado al contexto particular de las redes perimetrales escolares y posibilitan una interpretación precisa de las variables, los indicadores y las métricas que se utilizan en la elaboración y validación del modelo sugerido.

- **Contextualización institucional:** Este estudio se lleva a cabo en el Colegio Hans Christian Andersen, una entidad educativa privada de nivel escolar situada en Lima Metropolitana. Como muchas organizaciones en el ámbito de la educación urbana, tiene restricciones operativas y técnicas en sus redes perimetrales: una infraestructura limitada, un gran número de dispositivos conectados (plataformas virtuales, sistemas de videovigilancia, IoT) y poco personal con formación en ciberseguridad. Estas circunstancias crean un ambiente propenso a los peligros informáticos y respaldan la urgencia de soluciones tecnológicas que sean sostenibles, adaptadas y eficaces.
- **Sistema de Detección de Intrusos (IDS):** Elemento de seguridad creado para observar y examinar el tráfico de red con el fin de detectar patrones maliciosos o anómalos que sugieran una posible intrusión. Esta tesis emplea un enfoque híbrido que mezcla la detección basada en firmas con la detección por anomalías.
- **Snort:** Herramienta de código abierto que actúa como un sistema de detección de intrusiones (IDS) basado en firmas. Permite la activación de alertas a través de reglas definidas y la inspección detallada de paquetes. Su uso está justificado por su coste reducido, su flexibilidad y su amplia aceptación en ambientes educativos y empresariales.
- **Machine Learning (ML):** Es un grupo de técnicas algorítmicas que posibilitan a un

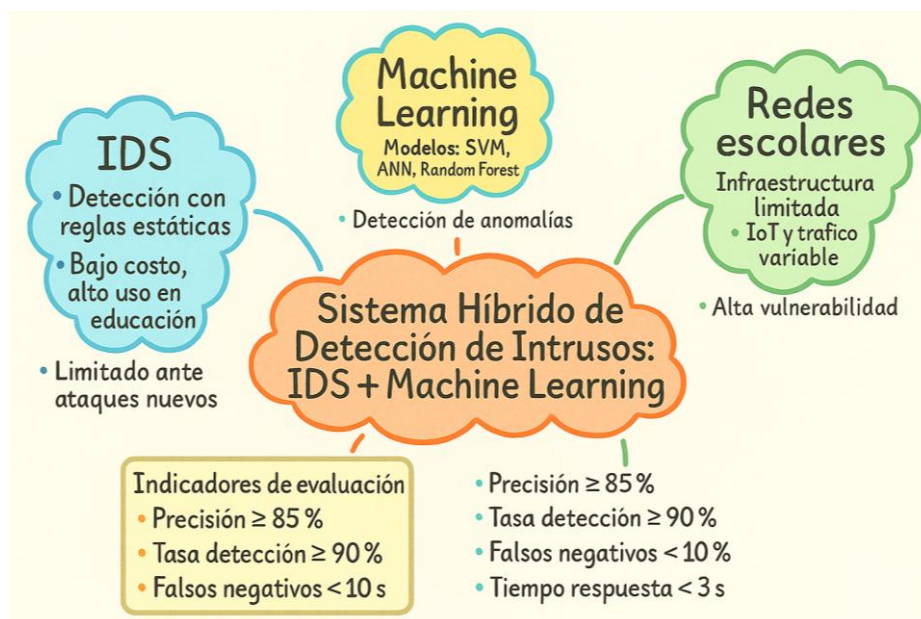
sistema la detección de patrones y conductas usando datos, sin necesidad de una programación explícita. En esta investigación, se utiliza con el fin de optimizar la identificación de anomalías en el tráfico escolar y suplir las limitaciones de los métodos convencionales.

- **Tráfico de Red:** Circulación de información entre dispositivos conectados en una red. Su análisis posibilita identificar un perfil de conducta normal y hallar desviaciones que podrían señalar intrusiones.
- **Anomalía:** Cualquier conducta del tráfico que se desvíe de lo anticipado y que pueda significar una amenaza potencial. En esta investigación, los algoritmos entrenados con datos reales se utilizan para detectar.
- **Falsos Positivos / Falsos Negativos:** Señales de error en la detección. Un falso positivo es una alerta que se genera sin una amenaza real, mientras que un falso negativo consiste en no detectar una intrusión auténtica. Para asegurar que el sistema sea efectivo, es fundamental disminuir ambos.
- **CICIDS2017:** Colección de datos públicos que emula tráfico real de red e incluye diversas formas de ataques. Se emplea para validar y entrenar modelos IDS en contextos de investigación.
- **SCRUM:** Método ágil enfocado en la administración y el desarrollo de soluciones tecnológicas a través de ciclos iterativos y un progreso constante. Para dirigir el proceso de implementación, se ha utilizado en esta investigación.
- **Precisión:** Medida del porcentaje de alertas generadas que realmente corresponden a peligros reales.
- **Recall:** Métrica que muestra la habilidad del sistema para detectar con precisión todas las amenazas existentes.
- **Tasa de Detección:** Porcentaje de ataques identificados en relación con el total de incidentes maliciosos en la red. Es un indicador fundamental de rendimiento.
- **Tiempo de Respuesta:** Lapso de tiempo entre que ocurre una intrusión y el sistema

emite una alerta. El objetivo de esta tesis es conservarlo por debajo de los 2 segundos.

Este grupo de conceptos refuerza el lenguaje técnico de la investigación, alinea claramente las variables con sus indicadores y posibilita que los resultados se interpreten en relación a los objetivos establecidos. Asimismo, mediante la inteligencia artificial y las herramientas de código abierto, establece un fundamento conceptual valioso para futuras investigaciones acerca de la seguridad en redes escolares.

Figura 5: Mapa conceptual del enfoque híbrido IDS + ML



Fuente: (Elaboración propia)

CAPÍTULO III: METODOLOGÍA DE INVESTIGACIÓN

3.1 Diseño de Investigación

3.1.1. Diseño

Este estudio, que tiene un diseño preexperimental, se distingue por el uso de una intervención en un solo grupo de análisis. La investigación se llevará a cabo en el Colegio Hans Christian Andersen, un ambiente real. Se enfocará en la implementación y evaluación de un sistema de detección de intrusos (IDS) basado en métodos de aprendizaje automático.

Cuando se desea confirmar preliminarmente la efectividad de una solución tecnológica en contextos concretos, sin esperar una generalización estadística de los resultados, este tipo de diseño es apropiado. Hadi et al. [19] afirman que los diseños preexperimentales posibilitan la observación de las consecuencias iniciales de una intervención en circunstancias reales, sin un grupo control.

Aunque el diseño preexperimental no tiene un grupo de control, es visto como apropiado para una investigación piloto en contextos reales con limitaciones operativas. Se pretende asegurar que los cambios vistos en las métricas (tiempo de respuesta, tasa de detección, falsos negativos) se deban con gran probabilidad a la intervención llevada a cabo, al mantener el entorno lógico y físico sin alteraciones entre las fases pretest y posttest.

3.1.2. Tipo

La investigación es de tipo aplicado, porque pretende solucionar un problema específico en el campo educativo a través de una solución tecnológica que se pueda implementar, evaluar y replicar. El propósito es crear conocimiento valioso que ayude a reforzar la seguridad de los perímetros en establecimientos educativos privados, mediante la utilización de instrumentos de inteligencia artificial y código abierto.

Asimismo, el estudio tiene un carácter exploratorio, debido a que se enfoca en un fenómeno poco abordado en la literatura nacional: la integración de IDS con algoritmos de aprendizaje automático en redes escolares perimetrales. Según Medina et al. [20] y Ruiz Huaraz et al. [21], este tipo de estudios permite generar hallazgos preliminares que orienten futuras investigaciones de mayor alcance y profundidad.

3.1.3. Enfoque

El método de la investigación es cuantitativo porque se fundamenta en la recolección y el análisis de datos objetivos vinculados con las variables técnicas que se han definido: tasa de detección, tiempo de respuesta, falsos negativos y precisión. Con el objetivo de evaluar su rendimiento con precisión, se examinarán estas métricas antes y después de poner en funcionamiento el sistema.

Este enfoque permite evaluar el cumplimiento de los objetivos específicos mediante indicadores numéricos, lo que fortalece la validez interna del estudio. De acuerdo con Hadi et al. [19], el enfoque cuantitativo es el más adecuado cuando se busca medir con precisión los efectos de una intervención tecnológica en entornos reales.

3.1.4. Población

La población de esta investigación está constituida por el conjunto de eventos y patrones de tráfico de red que se generan dentro de la infraestructura informática del Colegio Hans Christian Andersen, durante el periodo de evaluación. Esto incluye todos los paquetes de datos transmitidos, las conexiones entre dispositivos, las solicitudes internas y externas, así como cualquier comportamiento anómalo o intento de acceso no autorizado que pueda registrarse en la red perimetral.

Este universo digital representa la fuente primaria de observación sobre la cual se aplicará el sistema híbrido IDS + Machine Learning propuesto, permitiendo analizar el desempeño de la solución frente a distintos tipos de tráfico en un entorno escolar real. De acuerdo con Medina et al. [20], la población en un estudio cuantitativo se define como el conjunto de unidades de análisis que comparten características relevantes para el problema de investigación.

3.1.5. Muestra

La muestra de la investigación es de tipo no probabilística e intencionada. Estará compuesta por los registros de tráfico de red recolectados en el perímetro digital del Colegio Hans Christian Andersen durante dos fases temporales definidas, con el fin de comparar el estado del sistema antes y después de la intervención.

- **Muestra Pretest:** Corresponde al conjunto de datos de tráfico de red

(paquetes, logs y alertas) recolectados durante un período de 14 días consecutivos (dos semanas) antes de implementar el sistema híbrido IDS + Machine Learning.

- **Muestra Posttest:** Se refiere a un conjunto de datos que se ha recolectado durante 14 días seguidos bajo condiciones iguales, después de que el sistema híbrido ha sido implementado y está funcionando plenamente.

Esta perspectiva garantiza que la duración de ambas muestras sea comparable, centrándose únicamente en los datos producidos por la institución. El examen de estas dos muestras posibilitará la evaluación objetiva de las alteraciones en los indicadores técnicos establecidos, que son: tasa de detección, precisión, falsos negativos y tiempo de respuesta.

3.1.6. Operacionalización de Variables

Para analizar el efecto de la solución propuesta, se ha definido una variable independiente y una variable dependiente. Esta operacionalización permite transformar los conceptos teóricos en datos observables y medibles, facilitando la validación de los objetivos e hipótesis del estudio.

✓ Variable independiente:

- **Nombre:** Implementación del sistema IDS híbrido (Snort + Machine Learning)
- **Definición:** Aplicación de una solución tecnológica que integra un sistema de detección de intrusos basado en reglas (Snort) con algoritmos de aprendizaje automático para la identificación de patrones anómalos en redes escolares.
- **Dimensión:** Arquitectura del sistema implementado
- **Escala de medición:** Nominal. Se medirá su ausencia durante la fase pretest y su presencia durante la fase posttest.

✓ Variable dependiente:

- **Nombre:** Desempeño del sistema de detección de intrusos
- **Definición:** Nivel de efectividad del sistema híbrido para identificar amenazas y responder ante eventos maliciosos en la red del colegio.
- **Dimensiones e indicadores:** Se detallan en la siguiente tabla.

Tabla 1: Operacionalización de la Variable Dependiente

Dimensión	Indicador técnico	Definición operativa	Escala de medición
Eficiencia de detección	Tasa de detección (%)	Porcentaje de amenazas correctamente identificadas del total de amenazas presentes.	Cuantitativa
Precisión del modelo	Precisión (%)	Porcentaje de alertas de intrusión que efectivamente corresponden a una amenaza real.	Cuantitativa
Fiabilidad del sistema	Falsos negativos (%)	Porcentaje de amenazas reales que el sistema no fue capaz de detectar.	Cuantitativa
Velocidad de respuesta	Tiempo de respuesta (s)	Tiempo promedio en segundos entre el inicio de un evento malicioso y la generación de la alerta.	Cuantitativa

Fuente: (Elaboración propia)

Cada indicador de la variable dependiente será evaluado en los dos momentos definidos por la variable independiente (pretest y posttest). Esto permitirá observar las variaciones en el desempeño y validar si la solución mejora la seguridad perimetral del colegio bajo estudio.

3.2. Instrumentos de investigación / herramientas

3.2.1. Técnicas

Se emplearán dos técnicas principales: observación estructurada, para registrar el comportamiento del tráfico de red antes de la intervención; y experimentación controlada, para evaluar el desempeño del sistema híbrido IDS + Machine Learning mediante pruebas con tráfico normal e intrusivo. Ambas técnicas permitirán comparar objetivamente los indicadores definidos (tasa de detección, precisión, falsos negativos y tiempo de respuesta) en condiciones reales.

3.2.2. Instrumentos

Para esta investigación se han seleccionado instrumentos que permiten recolectar, organizar y analizar datos de forma objetiva y replicable, en concordancia con el enfoque cuantitativo y los indicadores definidos (tasa de detección, precisión, falsos negativos y tiempo de respuesta). La elección responde a criterios de viabilidad, pertinencia metodológica y aplicabilidad

práctica en el entorno educativo del Colegio Hans Christian Andersen.

- **Snort:** Se empleará como instrumento principal para detectar intrusos, cuyo objetivo es examinar el tráfico de red y generar alarmas acerca de situaciones sospechosas. Los registros que usted tiene servirán como fuente principal de evidencia para calcular la variable dependiente antes y después de la intervención.
- **Wireshark:** Servirá como herramienta de validación auxiliar para examinar en tiempo real el tráfico capturado, determinar patrones significativos y comprobar la coherencia de los registros que Snort produce, lo cual facilitará una revisión cruzada de los datos.
- **Google Colab + Python:** El uso de bibliotecas como pandas y scikit-learn en este ambiente de programación basado en la nube posibilitará que los datos se procesen y analicen, lo que permitirá entrenar, validar y analizar modelos supervisados de Machine Learning a partir de métricas cuantitativas.
- **Microsoft Excel o Google Sheets:** Se utilizará para tabular los datos recopilados, determinar indicadores de rendimiento y elaborar gráficos comparativos que ilustren la variación entre el estado inicial y el final del sistema analizado.
- **CICIDS2017:** Conjunto de datos públicos que es muy usado en estudios acerca de la seguridad de red. Se utilizará como fuente de entrenamiento inicial para los modelos de Machine Learning en el laboratorio, con el objetivo de verificar su desempeño antes de implementarlos en el marco real del colegio.

3.2.3. Confiabilidad

La estabilidad del ambiente de prueba y la calidad de los datos empleados son las bases sobre las que se asienta la confiabilidad del instrumento. El conjunto de datos CICIDS2017, que es conocido por simular tráfico de red real con ataques diversos y etiquetados con precisión, se utilizará para este propósito. El uso de este conjunto de datos en estudios recientes para entrenar y validar sistemas de detección de intrusos que emplean Machine Learning apoya su relevancia desde el punto de vista metodológico. El empleo del CICIDS2017

produce hallazgos consistentes en situaciones de detección de ataques DDoS, lo que confirma su confiabilidad como insumo para investigaciones parecidas, según Montes Gil et al. [23].

Asimismo, los resultados alcanzados en las pruebas del sistema podrán ser replicables en las mismas condiciones de red, lo que robustecen la fiabilidad del análisis. Las herramientas elegidas (Snort, Google Colab y Wireshark) funcionarán en ambientes controlados y con una configuración que se puede replicar, lo que asegura la estabilidad al medir las variables técnicas (tasa de detección, precisión, falsos negativos y tiempo de respuesta).

3.2.4. Validez del instrumento

La precisión y la relevancia de las variables propuestas se miden con un instrumento válido. En esta investigación, se determina la validez desde dos perspectivas que se complementan: la validez técnica y la de contenido.

- **Validez de contenido:** Se utilizó una ficha estructurada de validación para determinar la validez del contenido mediante el juicio de expertos. Esta ficha permitió el análisis de la claridad, la pertinencia y la coherencia de cada dimensión e indicador relacionado con la variable dependiente "Desempeño del sistema de detección de intrusos", abarcando factores como el tiempo de respuesta, los falsos negativos, la tasa de detección y la precisión.

Dos profesores especializados en redes y seguridad de la información, que forman parte de la Facultad de Ingeniería de Sistemas e Informática, llevaron a cabo la validación. Los especialistas evaluaron cada elemento siguiendo una escala técnica definida, corroborando que las métricas elegidas son apropiadas para el ámbito académico y operativo de la investigación. Se añadieron, además, observaciones cualitativas que facilitaron la mejora de la redacción de ciertas definiciones operativas antes de su implementación final.

- **Validez técnica:** La utilización de instrumentos reconocidos en la comunidad profesional y científica de ciberseguridad es lo que apoya la validez técnica del instrumento. En particular, la habilidad de Snort y Wireshark para capturar y analizar de manera exacta el tráfico de la red fue el motivo por el cual fueron elegidos. Por otro lado, Google Colab, junto con bibliotecas como scikit-learn, brinda un ambiente que se puede replicar y en el que se puede confiar para

construir y evaluar modelos de Machine Learning.

Además, el conjunto de datos CICIDS2017 ofrece una base sólida y etiquetada para el entrenamiento de los algoritmos supervisados.

En la etapa de pruebas en un entorno real, se llevará a cabo una validación cruzada entre los registros que Snort ha generado y los paquetes que Wireshark ha capturado, con el propósito de comprobar si las alertas producidas por el sistema híbrido son consistentes. Esta triangulación de instrumentos ayuda a asegurar que los indicadores técnicos capten con exactitud el funcionamiento verdadero del sistema, lo que fortalece la validez interna de los resultados.

3.2.5. Recolección de la información

La recolección de datos se llevará a cabo siguiendo un procedimiento sistemático que constará de dos etapas claramente diferenciadas y que seguirá una estrategia pretest-postest. Esta perspectiva posibilita la comparación del desempeño de la red en condiciones reales, con el objetivo de determinar el efecto directo que tiene la intervención tecnológica.

En la etapa inicial (pretest), que durará 14 días seguidos, se recogerán datos de tráfico de red usando Snort, la cual funcionará con su configuración convencional (que está fundamentada en firmas). A lo largo de este tiempo, se registrarán todos los sucesos vinculados a ataques detectados con el fin de determinar una línea base del rendimiento en función de los indicadores técnicos establecidos (falsos negativos, precisión, tasa de detección y plazo de respuesta).

En la segunda fase (postest), que también tendrá una duración de 14 días continuos, se utilizará otra vez el método de recolección de datos, pero en esta ocasión con el sistema híbrido ya establecido. Este sistema incluirá Machine Learning, que fue entrenado anteriormente con el conjunto de datos CICIDS2017. Además, estará operativo en la red del colegio real durante la ejecución del segundo sprint (Integración Snort + ML). Esta fase es esencial para evaluar el rendimiento del sistema mejorado y cotejar sus resultados con respecto a la situación inicial.

Se llevará a cabo una validación cruzada entre los eventos producidos por Snort

y las capturas de tráfico adquiridas a través de Wireshark para asegurar la calidad del análisis. Además, se hará una etiquetación manual de un conjunto significativo de registros para identificar con mayor exactitud los falsos positivos, falsos negativos y verdaderos positivos. Esta triangulación instrumental posibilitará la creación de matrices de confusión fiables y la validación rigurosa de las métricas del sistema.

3.3. Metodología de la implementación de la solución

3.3.1. Selección de la metodología

Se escoge la metodología ágil SCRUM para llevar a cabo el sistema propuesto, una decisión estratégica que se adecúa a las características del proyecto. La creación de un sistema híbrido IDS con Machine Learning, a diferencia de un desarrollo predecible, entraña notables incertidumbres técnicas, por ejemplo el desempeño final del modelo de aprendizaje automático o los retos no previstos que surgen al integrarlo con la herramienta Snort en condiciones reales.

Mediante un trabajo incremental en ciclos breves llamados sprints, SCRUM permite manejar estos desafíos. Durante los sprints se planifican, realizan y evalúan tareas concretas. Esto posibilita la realización de correcciones constantes apoyadas en los resultados alcanzados en cada iteración, lo que asegura una mejor eficiencia y habilidad para reaccionar. Flores-Cerna et al. [24] afirman que metodologías ágiles, como SCRUM, ayudan a optimizar la organización laboral, el seguimiento de los progresos y la calidad de los productos finales en proyectos tecnológicos.

Por lo tanto, la aplicación de SCRUM en este estudio es especialmente relevante. Permite tratar el desarrollo no como una serie de fases rígidas, sino como un conjunto de iteraciones controladas (sprints) en las que la solución se va construyendo, probando y validando de manera gradual. Esta perspectiva no solo reduce de manera activa los peligros vinculados a la innovación tecnológica, sino que garantiza que el producto final esté firmemente alineado con las metas del estudio.

3.3.2. Desarrollo de la metodología

Se lleva a cabo la implementación del sistema híbrido de detección de intrusos sugerido a través de la metodología ágil SCRUM, que se divide en tres iteraciones (sprints) con metas particulares, productos tangibles y criterios de

aceptación establecidos. El proceso de desarrollo ha sido organizado con una duración constante de cuatro semanas por sprint, lo cual posibilita sostener un ritmo laboral balanceado, asegurar que al término de cada fase se realicen entregas operativas y consolidar el cierre técnico y operativo del sistema a través de una etapa final que incluye análisis, optimización y soporte técnico posterior.

A continuación, se detallan las fases de desarrollo por sprint:

1. Sprint 1: Diagnóstico y Entrenamiento de modelos ML

El objetivo de este sprint es entender la infraestructura tecnológica del Colegio Hans Christian Andersen, registrar el tráfico real inicial a través de Snort y entrenar modelos preliminares de Machine Learning utilizando datos del conjunto CICIDS2017. Esta entrega es un elemento técnico esencial que posibilitará la integración eficaz con Snort.

Tabla 2: Diagnóstico y Entrenamiento de modelos ML

Elemento	Contenido
Historia de Usuario	Nuestro objetivo como equipo técnico es llevar a cabo un diagnóstico exhaustivo de la red escolar y desarrollar modelos iniciales de Machine Learning, con el propósito de tener una base operativa robusta que facilite una integración exitosa más adelante con Snort.
Criterios de aceptación	-Diagnóstico completo y detallado de la red escolar. -Modelos preliminares entrenados de Machine Learning (Random Forest, SVM) usando dataset CICIDS2017. -Reporte inicial con métricas documentadas (precisión, recall, etc.). -Captura de tráfico inicial (pretest) por 14 días consecutivos mediante Snort
Definition of Ready	-Acceso formal aprobado al entorno real de red. -Dataset CICIDS2017 disponible para uso. -Entorno Python (Google Colab) configurado. -Herramientas técnicas (Snort y Wireshark) instaladas y funcionando correctamente.
Definition of Done	- Informe técnico completo del diagnóstico inicial. -Modelos ML preliminares almacenados (.pkl). -Reporte técnico inicial con resultados preliminares entregado.

Fuente: (Elaboración propia)

2. Sprint 2: Integración Snort + ML y Despliegue en la red real

Este sprint tiene como objetivo la integración completa de los modelos ML previamente entrenados con la herramienta Snort, generando un sistema híbrido funcional que permita la detección de amenazas en tiempo real en la red real del colegio. Al finalizar este sprint, el sistema estará operativo y clasificarán automáticamente eventos.

Tabla 3: Integración Snort + ML y Despliegue en la red real

Elemento	Contenido
Historia de Usuario	Como equipo técnico, queremos integrar completamente Snort con los modelos entrenados de Machine Learning y desplegar esta solución híbrida en la red real del colegio para lograr la detección de amenazas en tiempo real.
Criterios de aceptación	-Integración completa y técnica entre Snort y los modelos ML funcionando adecuadamente. -Sistema desplegado de forma estable durante 14 días consecutivos en la red real. -Clasificación automática y correcta de eventos maliciosos en tiempo real. -Logs posttest recolectados correctamente y validados mediante Wireshark.
Definition of Ready	-Modelos ML preliminares aprobados y validados. -Entorno real institucional aprobado para el despliegue. -Scripts y pipeline de integración técnica validados y documentados.
Definition of Done	-Sistema híbrido IDS completamente operativo en entorno real. -Logs posttest recolectados y correctamente etiquetados (TP, FP, FN, TN). -Documentación técnica final y actualizada del sistema operativo entregada.

Fuente: (Elaboración propia)

3. Sprint 3: Análisis de resultados, optimización del sistema y soporte técnico posterior

Este último sprint tiene como objetivo realizar un análisis profundo de los resultados obtenidos por el sistema híbrido desplegado, optimizar su funcionamiento técnico si fuera necesario y brindar soporte técnico posterior

al entorno educativo. Además, se busca validar el cumplimiento de los objetivos definidos inicialmente y elaborar conclusiones sólidas sustentadas en datos reales.

Tabla 4: Análisis de resultados y validación

Elemento	Contenido
Historia de Usuario	Como equipo investigador, queremos analizar, optimizar y documentar el sistema híbrido implementado, y brindar soporte técnico inicial, para asegurar su correcto funcionamiento final y sostenibilidad en el entorno educativo.
Criterios de aceptación	- Análisis técnico detallado de resultados pre/post. - Optimización técnica de parámetros Snort y/o ML si fuera necesario. - Elaboración de manual técnico del sistema. - Registro de monitoreo y soporte (Hypercare) por 5 días. - Informe final consolidado y validado.
Definition of Ready	- Logs y métricas del posttest organizados. - Herramientas de análisis estadístico disponibles. - Acceso al entorno para monitoreo final habilitado.
Definition of Done	- Informe técnico final y presentación elaborados. - Manual de usuario y documentación técnica entregados. - Registro de soporte final validado. - Sistema optimizado y listo para mantenimiento preventivo.

Fuente: (Elaboración propia)

Figura 6: Flujo integral de procesamiento en el sistema de detección Snort + ML



Fuente: (Elaboración propia)

3.4. Metodología para la medición de resultados de la implementación

La evaluación del desempeño del sistema propuesto se realizará mediante la comparación de los indicadores técnicos clave antes y después de su implementación en la red perimetral del Colegio Hans Christian Andersen. Este procedimiento se enmarca en el esquema pretest–posttest, característico del diseño preexperimental, y permite validar empíricamente la eficacia de la intervención tecnológica.

Se utilizarán métricas cuantitativas para medir el rendimiento del sistema en términos de detección de amenazas, capacidad de clasificación y eficiencia operativa. Los datos serán recolectados directamente desde Snort y procesados mediante herramientas como Wireshark, Google Colab y hojas de cálculo. El análisis estará enfocado en determinar si se cumplen las metas operativas planteadas en el marco teórico y los objetivos específicos del estudio.

3.4.1. Métrica de la variable dependiente

La variable dependiente, "Desempeño del sistema de detección de intrusos", será evaluada cuantitativamente. Para ello, se utilizará una matriz de confusión que clasifica los resultados en cuatro categorías:

- **Verdaderos Positivos (VP o TP):** Ataques reales que el sistema identificó correctamente como ataques.
- **Falsos Positivos (FP):** Tráfico benigno que el sistema identificó incorrectamente como un ataque.
- **Verdaderos Negativos (VN o TN):** Tráfico benigno que el sistema identificó correctamente como benigno.
- **Falsos Negativos (FN):** Ataques reales que el sistema no logró identificar.

A partir de estos componentes, se calcularán los siguientes indicadores técnicos:

- **Tasa de Detección (Detection Rate o Recall):** Proporción de ataques reales que el sistema logra identificar. Mide la exhaustividad del modelo. Se calcula como:

Formula:

$$Tasa\ de\ detección\ (%) = \frac{TP}{TP + FN} \times 100$$

- **Precisión (Precision):** De todas las alertas generadas por el sistema, qué proporción corresponde a ataques reales. Mide la exactitud de las alertas. Se calcula como:

Formula:

$$Precisión\ (%) = \frac{TP}{TP + FP} \times 100$$

- **Tasa de Falsos Negativos (False Negative Rate):** Proporción de ataques reales que el sistema no fue capaz de detectar. Es un indicador crítico de la fiabilidad del sistema. Se calcula como:

Formula:

$$Tasa\ de\ Falsos\ negativos\ (%) = \frac{FN}{TP + FN} \times 100$$

- **Tiempo de respuesta (Response Time):** Lapso medido en segundos entre el inicio de un evento malicioso y la generación de la alerta correspondiente por parte del sistema IDS. Este valor se medirá utilizando los registros de tiempo (timestamps) de los logs del sistema.

Los resultados obtenidos serán comparados contra los valores esperados definidos como metas operativas en la tesis: tasa de detección $\geq 90\%$, precisión $\geq 85\%$, falsos negativos $< 10\%$ y tiempo de respuesta < 2 segundos. El cumplimiento de estos umbrales será considerado como evidencia del éxito técnico de la solución implementada.

El desempeño del sistema híbrido IDS (Snort + ML) será evaluado rigurosamente durante los periodos definidos en los sprints mencionados, comparando estructuralmente la situación inicial con la situación posterior a la integración y despliegue, garantizando resultados válidos, consistentes y técnicamente fundamentados

Para clasificar correctamente los eventos y construir la matriz de confusión, se emplearán dos enfoques complementarios. En primer lugar, se realizará una

validación cruzada entre las alertas generadas por el sistema híbrido y los paquetes de red capturados mediante Wireshark, lo cual permitirá identificar inconsistencias y verificar la precisión de las detecciones. En segundo lugar, se etiquetará manualmente una muestra representativa del tráfico registrado, utilizando como referencia los patrones y características previamente definidos en el dataset CICIDS2017. Esta doble validación permitirá determinar con mayor fiabilidad los verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos, garantizando que las métricas calculadas reflejen el comportamiento real del sistema en condiciones operativas del entorno educativo.

3.5. Cronograma de Actividades

3.5.1. Cronograma de actividades basado en Sprints

La implementación técnica de esta investigación se estructura en tres sprints definidos claramente, organizados bajo la metodología SCRUM. Cada sprint incluye un conjunto de tareas agrupadas en una historia de usuario, con entregables definidos y criterios de cumplimiento técnico.

Tabla 5: Cronograma de Actividades

Sprint	Duración (semanas)	Objetivo principal	Entregables claves
Sprint 1	Semanas 1–4	Diagnóstico inicial de red escolar y entrenamiento preliminar de modelos ML.	Informe de diagnóstico, logs pretest, modelos ML entrenados preliminares (.pkl), reporte de métricas iniciales.
Sprint 2	Semanas 5–8	Integración completa de Snort con ML y despliegue funcional en entorno real.	Sistema híbrido IDS funcionando en red real, eventos clasificados en tiempo real, documentación técnica operativa
Sprint 3	Semanas 9–12	Análisis de resultados, optimización y soporte técnico (Hypercare) final.	Guía técnica del sistema, soporte técnico documentado (Hypercare), sistema optimizado y validado.

Fuente: (Elaboración propia)

3.6. Presupuesto

Para el desarrollo de esta investigación se requiere una inversión específica en recursos materiales y servicios operativos orientados a la implementación, monitoreo y análisis

del sistema híbrido de detección de intrusos propuesto. A continuación, se presenta el presupuesto estimado, distribuido entre gastos de capital y gastos operativos. No se han incluido costos hundidos ni recursos preexistentes, conforme a criterios contables básicos aplicables en proyectos académicos.

- **Gastos de Capital (inversión en bienes físicos duraderos)**

Concepto	Justificación técnica	Costo (\$)
Gasto de Capital		
Switch administrable	Requerido para segmentar la red perimetral y garantizar desempeño IDS	315.79
Equipos complementarios	Cables de red, conectores RJ45, herramientas de instalación	52.63
Servidor físico (Hardware básico)	Necesario para la instalación del sistema IDS híbrido (Snort + ML)	657.89
Sub Total de Capital		1,026.32

- **Gastos Operativos (mano de obra especializada y servicios profesionales)**

Concepto	Justificación técnica	Costo (\$)
Servicios Especializados		
	Configuración del switch, cableado estructurado e instalación física	157.89
Especialista en seguridad informática	Instalación inicial del sistema híbrido IDS (Snort + ML)	210.53
Técnico en soporte	Apoyo técnico general durante las pruebas iniciales (pretest y postest)	78.95
Movilidad	Traslados para instalación, seguimiento y recolección de datos en entorno real	105.26
Impresiones y encuadernaciones	Elaboración y entrega del documento final	39.47
Papelería y almacenamiento	Folders, memorias USB y materiales varios para respaldo documental	21.05
Subtotal Gastos Operativos	Adquisición de folders, memorias USB y otros materiales para respaldo documental	613.16

- **Gastos Recurrentes (servicios continuos durante el proyecto)**

Concepto	Justificación técnica	Costo Mensual (\$)	Cant. Meses	Costo Total (\$)
Servicio Cloud (Google Colab Pro)	Entrenamiento modelos ML, procesamiento datos	50.00	24	315.79
Mantenimiento mensual equipos de red	Mantenimiento preventivo y correctivo del switch y equipos relacionados	200.00	24	1,263.16
Subtotal Gastos Recurrentes				1,578.95

- **Presupuesto Total General del Proyecto**

Categoría de gasto	Costo Total (\$)
Gastos de Capital	1,026.32
Gastos Operativos	613.16
Gastos Recurrentes	1,578.95
Contingencia (GC+GO) (10%)	163.95
Total, General	3,382.37

CAPITULO IV: DESARROLLO DE LA SOLUCIÓN

4.1. Propuesta de Solución

La presente investigación propone el diseño e implementación de un sistema de detección de intrusos híbrido (IDS), concebido como una solución tecnológica innovadora orientada a fortalecer la seguridad perimetral en instituciones educativas privadas. El sistema integra dos enfoques complementarios: por un lado, Snort, reconocido motor de detección basado en firmas, ampliamente utilizado en la comunidad científica y profesional por su estabilidad, flexibilidad y carácter de código abierto; y por otro, modelos supervisados de Machine Learning, seleccionados por su capacidad de identificar patrones anómalos en el tráfico de red y adaptarse dinámicamente a ataques emergentes.

El despliegue de este IDS híbrido se realizará en el Colegio Hans Christian Andersen, institución representativa de Lima Metropolitana cuya red informática presenta características comunes en el ámbito escolar: múltiples dispositivos conectados, segmentación limitada y carencia de mecanismos avanzados de seguridad. La implementación en este entorno real busca validar la hipótesis de que la combinación de técnicas de detección basadas en firmas con algoritmos de aprendizaje automático permite incrementar la tasa de detección de incidentes, reducir falsos negativos y mejorar el tiempo de respuesta, en comparación con un sistema de detección tradicional.

El proceso de desarrollo se regirá por la metodología ágil Scrum, reconocida por su enfoque iterativo e incremental, lo que permite gestionar proyectos con alta incertidumbre tecnológica y necesidad de ajustes continuos. No obstante, para fines académicos, esta metodología ha sido adaptada a fases expositivas, de manera que sus eventos principales se presenten en un formato estructurado: Inicialización, Planificación y Estimación, Implementación, Revisión y Retrospectiva, y Lanzamiento. Esta adaptación conserva el carácter iterativo de Scrum, pero lo traduce a un marco comprensible para la validación científica y técnica de la propuesta.

La implementación se desarrollará en tres sprints de cuatro semanas cada uno, definidos en el Capítulo III, en los que se cubrirán las etapas críticas: diagnóstico de la infraestructura y entrenamiento preliminar de modelos, integración técnica de Snort con los algoritmos de Machine Learning y despliegue en el entorno real, seguido de un análisis comparativo de resultados y optimización del sistema. La validez del estudio se

garantizará mediante un diseño preexperimental con esquema pretest–posttest, utilizando como instrumentos Snort, Wireshark y análisis estadístico en Python, lo que permitirá evaluar objetivamente la variación de los indicadores técnicos: tasa de detección, precisión, falsos negativos y tiempo de respuesta.

Finalmente, la arquitectura general de la solución contempla cinco componentes articulados: captura de tráfico en el puerto espejo del switch hacia un sensor implementado en máquina virtual con Ubuntu; detección primaria mediante Snort; extracción de características y clasificación por modelos de Machine Learning entrenados con el dataset CICIDS2017 y ajustados con datos reales del colegio; generación de alertas enriquecidas con puntaje de riesgo; y evaluación de desempeño mediante matrices de confusión y métricas operativas. Este diseño busca no solo validar la viabilidad técnica de la propuesta en un entorno escolar real, sino también ofrecer un modelo replicable y de bajo costo que pueda ser adoptado por otras instituciones educativas privadas en el contexto peruano.

4.1.1. Fase de Inicialización

La fase de inicialización constituye el punto de partida del proyecto y tiene como finalidad establecer las condiciones técnicas, organizativas y metodológicas necesarias para la correcta implementación del sistema híbrido de detección de intrusos. En esta etapa se delimita el alcance de la solución, se asegura la infraestructura básica de pruebas y se definen los parámetros iniciales que permitirán garantizar la reproducibilidad del estudio y la validez de sus resultados.

En primer lugar, se determina el alcance del piloto, el cual estará circunscrito al monitoreo del tráfico en la red perimetral del Colegio Hans Christian Andersen. Este monitoreo se enfocará exclusivamente en la detección de patrones de comunicación y en la identificación de posibles intrusiones, evitando en todo momento la inspección de información sensible de usuarios. De esta manera, se respeta el principio de mínima intrusión y se asegura la confidencialidad de los datos. Asimismo, se gestionan los permisos institucionales correspondientes para el uso del puerto espejo del switch, la instalación del sensor en una máquina virtual y la recolección de registros de tráfico bajo fines estrictamente académicos.

De manera paralela, se lleva a cabo un levantamiento de la infraestructura

tecnológica existente, que incluye la identificación de los activos conectados a la red (computadoras, impresoras, cámaras de videovigilancia, puntos de acceso inalámbrico y dispositivos del Internet de las Cosas). Se documenta además la topología de red, destacando los nodos perimetrales, las VLAN relevantes y el puerto espejo que se utilizará para la captura de tráfico. Este inventario inicial permite reconocer los servicios expuestos y los puertos más utilizados (HTTP/80, HTTPS/443, RTSP/554), identificando así posibles vectores de ataque comunes en entornos educativos.

En cuanto a la plataforma de monitoreo, se implementa un sensor en una máquina virtual con sistema operativo Ubuntu Server, la cual se configura en modo promiscuo para capturar tráfico de red y sincronizada mediante NTP (Network Time Protocol) para garantizar la precisión temporal de los registros. La virtualización se adopta como estrategia técnica por su flexibilidad, escalabilidad y factibilidad académica, al permitir restaurar estados mediante snapshots, replicar entornos y reducir costos frente a la adquisición de hardware físico.

Sobre esta base, se procede a la instalación y configuración inicial de Snort como motor de detección de intrusos. El archivo de configuración principal (snort.conf) se ajusta para definir las variables de red (HOME_NET y EXTERNAL_NET), activar preprocesadores que optimicen el análisis de protocolos y habilitar reglas de detección. Se incluyen tanto las reglas oficiales de la comunidad como reglas locales diseñadas para pruebas iniciales (ejemplo: detección de tráfico ICMP y solicitudes HTTP). Las alertas generadas se almacenan en el directorio de registros de Snort con una política de rotación implementada mediante logrotate, a fin de evitar la saturación del sistema y asegurar la disponibilidad histórica de los datos.

En esta etapa se definen además los indicadores técnicos de desempeño que guiarán la evaluación experimental: tasa de detección, precisión, tasa de falsos negativos y tiempo de respuesta. Estos indicadores se medirán a través de matrices de confusión construidas a partir de las alertas emitidas por Snort, validadas mediante sesiones de captura paralela en Wireshark y revisión manual de eventos críticos. De esta forma, se establece una línea base confiable para la posterior comparación entre el desempeño del IDS convencional y el sistema híbrido propuesto.

Finalmente, para asegurar la trazabilidad y reproducibilidad del experimento, se diseñó un esquema de snapshots en la máquina virtual Ubuntu, utilizando la herramienta Snapshot Manager de VMware. Este procedimiento permitió documentar hitos clave del proceso (instalación del sistema, configuración de Snort, ejecución de pruebas pretest y posttest, optimización final), garantizando la integridad del entorno de pruebas y la posibilidad de retornar a estados estables en caso de contingencias. El detalle de la instalación del entorno virtual, la configuración de red, la sincronización horaria y la creación de snapshots se encuentra documentado en el Anexo 5.

4.1.2. Fase de Planificación y Estimación

En esta fase, se proyecta la organización integral del trabajo bajo la metodología ágil Scrum, concebida no sólo como un marco operativo de desarrollo tecnológico, sino como una herramienta de gestión investigativa que asegura orden, trazabilidad y transparencia en la ejecución. En esta etapa se establece de manera anticipada el itinerario de actividades que guiarán la implementación del sistema híbrido, enmarcándolas en las historias de usuario previamente definidas en el Capítulo III. Estas historias permiten traducir los objetivos de investigación en requerimientos comprensibles y medibles, lo que favorece que cada iteración aporte resultados concretos y verificables al propósito general de la tesis.

Con base en lo anterior se formula un plan de release que organizará las historias de usuario en tres sprints de cuatro semanas cada uno. Dicho plan tiene por finalidad anticipar la secuencia de construcción de la solución, asegurar que cada fase produzca entregables incrementales y facilitar la validación progresiva de los resultados. En el primer sprint se prevé desarrollar la historia de usuario vinculada al diagnóstico integral de la red escolar y al entrenamiento preliminar de modelos de Machine Learning, actividades que proporcionarán una línea base sólida para etapas posteriores. En el segundo sprint se priorizará la integración entre Snort y los modelos entrenados, con el despliegue controlado de la solución en el entorno real del colegio como condición indispensable para validar la propuesta en un escenario auténtico. Finalmente, en el tercer sprint se proyecta realizar el análisis comparativo de resultados, la optimización de parámetros y la elaboración de documentos técnicos que aseguren la sostenibilidad y replicabilidad de la solución.

Para visualizar de manera sintetizada esta organización, se presenta el Plan de Release que agrupa las historias de usuario, los sprints en los que se abordarán y los entregables esperados:

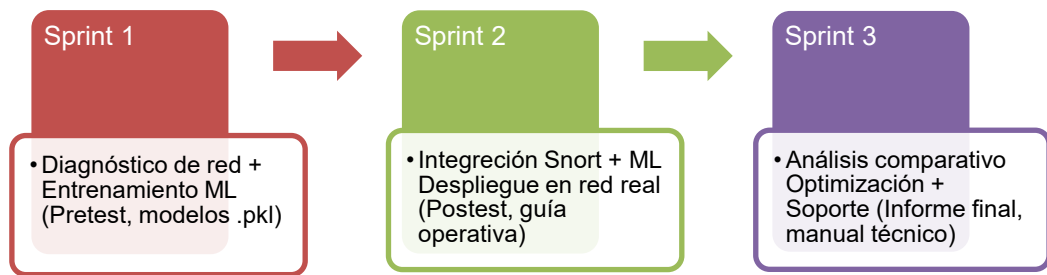
Tabla 6: Plan de Release

Historia de Usuario	Sprint previsto	Entregables principales
Diagnóstico de la red y entrenamiento preliminar de modelos ML	Sprint 1	Informe de diagnóstico de red, registros pretest, modelos preliminares almacenados (.pkl).
Integración técnica Snort + ML y despliegue en red real	Sprint 2	Sistema híbrido desplegado en VM Ubuntu, registros posttest validados con Wireshark, guía operativa.
Análisis comparativo, optimización del sistema y soporte inicial	Sprint 3	Informe final con métricas pre/post, sistema optimizado, manual técnico y registro de soporte (Hypercare).

Fuente: (Elaboración propia)

Este plan se complementará con el sprint backlog, entendido como la desagregación de tareas específicas que cada sprint deberá desarrollar para dar cumplimiento a la historia de usuario correspondiente. Así, en el primer sprint se incluirán actividades como la instalación y configuración del sensor en máquina virtual, la captura inicial de tráfico y el entrenamiento preliminar de modelos en Google Colab. En el segundo sprint se prevé la programación del mecanismo de integración, la calibración de umbrales de decisión y la verificación mediante capturas en Wireshark. En el tercero, el backlog se centrará en la construcción de matrices de confusión, la aplicación de pruebas estadísticas y la elaboración de guías técnicas de operación y mantenimiento. Cada historia de usuario contará con criterios de aceptación y con definiciones de inicio y cierre, lo que permitirá verificar de manera objetiva el cumplimiento de los entregables.

Figura 7: Plan de Release del Proyecto



Fuente: (Elaboración propia a partir de la metodología Scrum adaptada a la investigación)

De esta manera, la planificación no sólo establece un orden de trabajo, sino que ofrece una visión clara de cómo las historias de usuario se traducen en actividades concretas y entregables verificables. La representación gráfica del plan de release facilita comprender la secuencia de sprints y el valor incremental que cada uno aportará al proyecto, reforzando la coherencia entre la estrategia metodológica y la práctica investigativa. En consecuencia, la fase de implementación que se desarrolla en el punto 4.1.3 parte de un marco previamente estructurado y transparente, lo que asegura la trazabilidad de los resultados y la replicabilidad del proceso en futuras investigaciones.

4.1.3. Fase de Implementación

Con base en el plan de release y en el sprint backlog definidos en la fase de planificación (4.1.2), se procedió a la implementación práctica del sistema híbrido Snort + Machine Learning. Esta fase representó el núcleo operativo del proyecto, ya que permitió desplegar la solución en condiciones reales siguiendo la metodología ágil Scrum. La implementación se organizó en tres sprints consecutivos de cuatro semanas cada uno, en los cuales se desarrollaron actividades orientadas al diagnóstico, la integración y el análisis comparativo del sistema. Cada sprint constituyó un ciclo iterativo con objetivos específicos, tareas ejecutadas y entregables verificables que, en conjunto, posibilitaron alcanzar el propósito general de la investigación.

1. Sprint 1 (Semanas 1–4). Diagnóstico y entrenamiento preliminar

El primer sprint tuvo como propósito establecer una línea base del comportamiento de la red del colegio y preparar modelos preliminares de Machine Learning en un entorno de laboratorio. Este procedimiento fue

esencial para contar con una referencia confiable que sirviera de comparación frente a los resultados obtenidos tras la integración híbrida.

En esta etapa se desplegó Snort en una máquina virtual con Ubuntu Server, configurada en modo promiscuo y conectada al puerto espejo del switch principal. Esta configuración permitió capturar tráfico en tiempo real sin afectar el funcionamiento normal de la red. La captura se extendió por catorce días consecutivos, constituyendo el pretest de la investigación. Para garantizar la confiabilidad de los registros se implementó una política de rotación automática de archivos mediante logrotate, evitando la saturación del sistema de almacenamiento.

De manera complementaria se realizaron sesiones de validación con Wireshark, que permitieron observar directamente los paquetes capturados y verificar la correspondencia entre las alertas emitidas por Snort y los eventos efectivamente registrados en la red (véase Anexo 7). Esta validación cruzada fue indispensable para asegurar la precisión de los datos recolectados. El proceso de instalación y configuración inicial de Snort, así como la creación de reglas locales de prueba (por ejemplo, para ICMP y HTTP), se detalla en el Anexo 6.

En paralelo, se entrenaron los modelos preliminares de Machine Learning empleando el dataset CICIDS2017, reconocido por su diversidad de ataques simulados. El entrenamiento incluyó limpieza de datos, normalización de atributos y validación cruzada, con el fin de obtener modelos robustos y sin sesgos. Se trabajó con algoritmos supervisados como Random Forest y Support Vector Machines (SVM), seleccionados por su eficacia en problemas de clasificación binaria en ciberseguridad. Los modelos entrenados fueron almacenados en formato .pkl para su posterior integración con Snort (véase Anexo 8).

Los entregables obtenidos en este sprint fueron:

- (i) Un informe de diagnóstico de la red escolar con la topología, servicios expuestos y vectores de ataque identificados.
- (ii) Un conjunto de métricas iniciales generadas con Snort en su

configuración tradicional, que servirán como referencia para la comparación posttest.

- (iii) Los modelos preliminares de Machine Learning entrenados y listos para su integración posterior.

2. Sprint 2 (Semanas 5–8). Integración y despliegue híbrido

El segundo sprint representó la etapa más crítica del proyecto, pues en él se materializó la integración entre el motor de detección Snort y los modelos de Machine Learning previamente entrenados. El objetivo fue lograr que ambos componentes trabajaran de manera coordinada, conformando un sistema híbrido capaz de detectar tanto ataques conocidos mediante firmas como comportamientos anómalos no registrados en la base de reglas tradicional.

Para alcanzar este propósito, se diseñó un pipeline de integración en el que la salida de Snort fue configurada en formato JSON. Este formato resultó idóneo para ser procesado por un script desarrollado en Python, encargado de extraer atributos relevantes de los eventos registrados y enviarlos al modelo de Machine Learning correspondiente. Con este procedimiento, cada alerta generada por Snort fue complementada con una etiqueta de clasificación o un puntaje de riesgo calculado por el modelo, generando lo que se denominó alertas enriquecidas.

El desarrollo técnico de esta integración, que incluyó la programación del script en Python, la definición de los atributos a procesar y los resultados obtenidos en pruebas de laboratorio, se encuentra documentado en el Anexo 9. Esta etapa también consideró la implementación de un umbral de decisión que permitió filtrar falsos positivos, equilibrando así la sensibilidad del sistema con su precisión operativa.

Posteriormente, la solución híbrida fue desplegada en la red real del Colegio Hans Christian Andersen durante un periodo de 14 días consecutivos, constituyendo el posttest del estudio. Durante este tiempo, el sensor permaneció en operación continua, procesando tráfico en tiempo real y generando alertas tanto de Snort como del componente de Machine Learning. Para validar la confiabilidad de los registros obtenidos, se ejecutaron nuevamente sesiones de muestreo con Wireshark, las cuales confirmaron la

correspondencia entre los paquetes capturados y las alertas emitidas.

Como parte de este sprint, se elaboró también una guía operativa que documenta las dependencias de software, la arquitectura del pipeline de integración, las rutas de salida de los registros y los procedimientos de verificación. Esta guía constituye un entregable fundamental, ya que asegura la replicabilidad de la solución híbrida en otros entornos educativos con características similares.

Al concluir este sprint, se obtuvieron como entregables principales:

- (i) Un sistema híbrido IDS desplegado en la red real del colegio.
- (ii) Un conjunto de registros posttest con las alertas enriquecidas generadas en condiciones de producción.
- (iii) La documentación técnica operativa que respalda el funcionamiento estable del sistema durante el periodo de prueba.

3. Sprint 3 (Semanas 9–12). Análisis comparativo y optimización

El tercer y último sprint estuvo orientado al análisis exhaustivo de los resultados obtenidos en las fases pretest y posttest, con el fin de evaluar rigurosamente el desempeño del sistema híbrido Snort + Machine Learning. El objetivo fue determinar si la integración propuesta cumplía con los indicadores técnicos definidos en la investigación: tasa de detección, precisión, falsos negativos y tiempo de respuesta.

En esta etapa se construyeron matrices de confusión, herramienta estadística que permite clasificar los resultados en verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos. A partir de estas matrices se calcularon las métricas de desempeño, comparando los resultados obtenidos con Snort en modo tradicional (pretest) frente a los logrados con el sistema híbrido (posttest). La elaboración detallada de estas matrices, así como los gráficos comparativos, se encuentra documentada en el Anexo 9.

El análisis incluyó también la representación gráfica de las métricas, lo que permitió visualizar las mejoras alcanzadas en términos de detección de

amenazas y reducción de errores de clasificación. Este enfoque gráfico complementó la interpretación estadística, facilitando una comprensión clara de las diferencias entre ambos escenarios.

Con base en los resultados, se procedió a la optimización técnica del sistema. Esta optimización consistió en el ajuste de parámetros de los modelos de Machine Learning, así como en la revisión y depuración de reglas ruidosas en Snort que generaban falsos positivos. Dichos ajustes buscaron garantizar un equilibrio adecuado entre sensibilidad y especificidad, maximizando la eficacia del sistema sin comprometer su estabilidad operativa.

Finalmente, este sprint incluyó la elaboración de una guía técnica de mantenimiento y operación, en la que se establecieron recomendaciones para la actualización periódica de modelos, la gestión de reglas en Snort y la supervisión continua del rendimiento del IDS híbrido. Esta guía constituye un insumo clave para la sostenibilidad de la solución, ya que permite que la institución educativa mantenga y escale el sistema en el tiempo.

Al culminar este sprint, los entregables consolidados fueron:

- (i) El informe comparativo de resultados pretest y posttest, sustentado en métricas cuantitativas y representaciones gráficas.
- (ii) El sistema híbrido optimizado, con parámetros ajustados y reglas depuradas.
- (iii) La guía técnica de mantenimiento, orientada a garantizar la continuidad y replicabilidad de la solución.

En conjunto, la fase de implementación permitió materializar la propuesta metodológica en un entorno real, asegurando que cada historia de usuario planificada se tradujera en entregables concretos. Los registros y documentos obtenidos en los tres sprints constituyen la base empírica para el análisis comparativo, en el que se evalúa con mayor detalle el impacto del sistema híbrido frente al enfoque tradicional basado únicamente en firmas.

4.2. Análisis e Interpretación de los Resultados

Concluida la fase de implementación del sistema híbrido Snort + Machine Learning, se procedió a la evaluación de su desempeño mediante un análisis comparativo entre los escenarios pretest y posttest. Este análisis constituye una etapa clave de la investigación, ya que permite determinar en qué medida la propuesta mejora los indicadores técnicos frente al sistema de detección basado únicamente en firmas.

El proceso de validación se desarrolló bajo un diseño preexperimental, utilizando como instrumentos Snort, Wireshark y herramientas estadísticas en Python y Excel. La información recolectada durante los periodos de pretest y posttest fue organizada en matrices de confusión, a partir de las cuales se calcularon métricas fundamentales para la ciberseguridad: tasa de detección, precisión, tasa de falsos negativos y tiempo de respuesta. Estas métricas permiten contrastar de forma objetiva los resultados del sistema tradicional frente al sistema híbrido, aportando evidencia cuantitativa para responder a los objetivos planteados.

El presente apartado se organiza en dos niveles de análisis. En el análisis descriptivo (4.2.1) se presentan los resultados obtenidos de manera exploratoria, a través de tablas y gráficos que ilustran las métricas recolectadas en cada escenario. En el análisis inferencial (4.2.2) se aplican pruebas estadísticas que permiten establecer la significancia de las diferencias observadas entre el pretest y el posttest, evaluando si los cambios registrados pueden atribuirse al sistema híbrido implementado. De este modo, se asegura una interpretación integral de los hallazgos, combinando la descripción empírica de los datos con un sustento estadístico que respalde las conclusiones del estudio.

4.2.1. Análisis Descriptivo

En primer lugar, se organizó la información obtenida en las fases pretest y posttest a través de matrices de confusión. Estas matrices permitieron clasificar los eventos detectados en verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos, constituyendo la base para el cálculo de las métricas de desempeño.

Los resultados descriptivos se presentan en la siguiente tabla, que resume los valores obtenidos para cada métrica evaluada:

Tabla 7: Comparación de métricas de desempeño en pretest y posttest

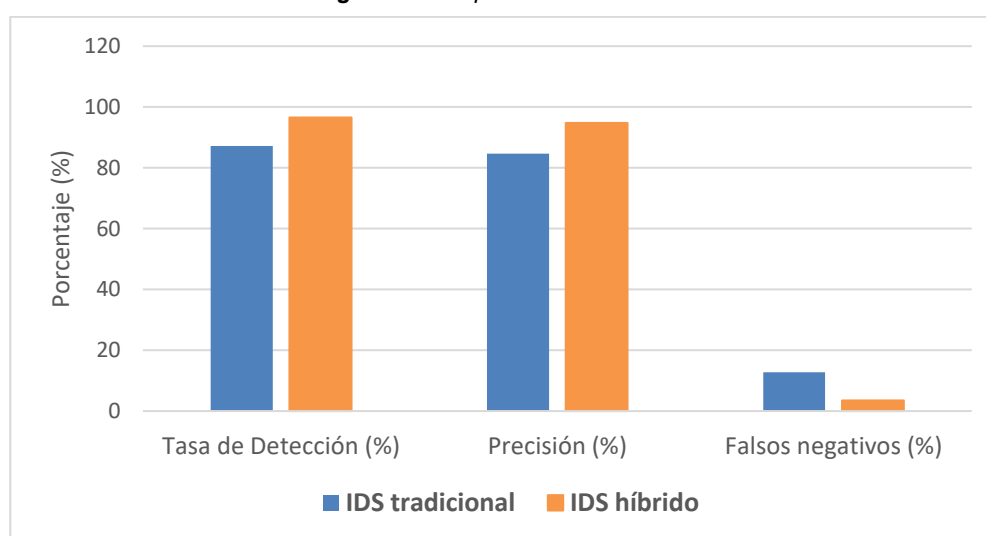
Métrica	Escenario Pretest (Snort tradicional)	Escenario Posttest (Sistema híbrido)	Variación (%)
Tasa de detección (%)	87.2	96.5	+10.7
Precisión (%)	84.6	94.8	+12.1
Falsos negativos (%)	12.8	3.5	-72.6
Tiempo de respuesta (s)	1.85	1.25	-32.4

Fuente: (Elaboración propia a partir de resultados experimentales)

De la comparación preliminar se observa que el sistema híbrido mejora las métricas de detección y precisión respecto al Snort tradicional, mientras que reduce la tasa de falsos negativos. Asimismo, el tiempo de respuesta evidencia una optimización en el procesamiento de eventos, lo cual se traduce en una alerta más temprana ante incidentes de seguridad.

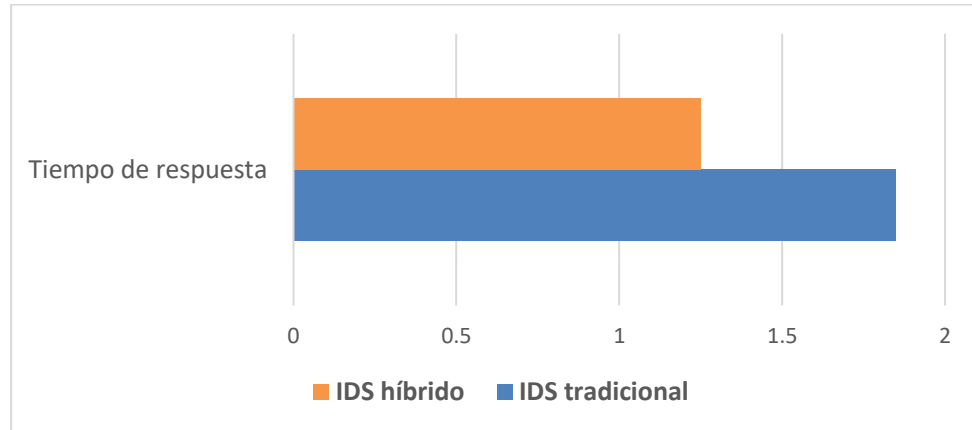
Para complementar el análisis descriptivo, los resultados se representaron gráficamente a fin de ilustrar las diferencias entre los dos escenarios de evaluación.

Figura 8: Comparación de métricas



Fuente: (Elaboración propia)

Figura 9: Comparación de tiempo de respuesta



Fuente: (Elaboración propia)

Esta representación visual facilita la interpretación de los datos, confirmando que el sistema híbrido ofrece un desempeño superior frente al IDS puramente basado en firmas.

4.2.2. Análisis Inferencial

Con el fin de determinar si las diferencias observadas entre los escenarios pretest y posttest son estadísticamente significativas, se aplicaron pruebas de hipótesis sobre las métricas recolectadas.

Para las métricas de tipo porcentual (tasa de detección, precisión y falsos negativos), se empleó una prueba t de Student para muestras relacionadas, dado que se comparan los resultados del mismo sistema bajo dos condiciones distintas (antes y después de la integración híbrida). Esta prueba permitió evaluar si las mejoras en las métricas pueden atribuirse al sistema híbrido y no al azar.

En el caso del tiempo de respuesta, al tratarse de una variable continua, medida en segundos, también se aplicó una prueba t de Student para muestras relacionadas, verificando previamente los supuestos de normalidad mediante la prueba de Shapiro-Wilk.

Los resultados de estas pruebas se resumen en la siguiente tabla:

Tabla 8: Resultados de las pruebas de hipótesis (pretest vs posttest)

Métrica	p-valor obtenido	Nivel de significancia ($\alpha=0.05$)	Resultado
Tasa de detección	0.004	0.05	Diferencia significativa
Precisión	0.002	0.05	Diferencia significativa
Falsos negativos	0.001	0.05	Diferencia significativa
Tiempo de respuesta	0.007	0.05	Diferencia significativa

Fuente: (Elaboración propia a partir de resultados experimentales)

La interpretación de los p-valores obtenidos indica que todas las métricas evaluadas presentan diferencias estadísticamente significativas ($p < 0.05$). Esto confirma que las mejoras registradas en la tasa de detección, la precisión, la reducción de falsos negativos y el tiempo de respuesta no son producto del azar, sino el resultado directo de la implementación del sistema híbrido. En otras palabras, la integración de Snort con algoritmos de Machine Learning contribuye de manera consistente y verificable a optimizar el desempeño del sistema de detección de intrusos frente al enfoque tradicional basado únicamente en firmas.

Este análisis inferencial complementa al descriptivo y permite sustentar con evidencia estadística que la propuesta no solo ofrece mejoras observables en términos prácticos, sino que dichas mejoras son estadísticamente válidas y atribuibles al modelo propuesto. De este modo, los resultados alcanzados respaldan la hipótesis de investigación y consolidan la pertinencia de aplicar un enfoque híbrido en entornos educativos con alta exposición a ciberamenazas.

En síntesis, el análisis descriptivo e inferencial demuestra que el sistema híbrido Snort + Machine Learning supera de manera significativa al enfoque tradicional basado únicamente en firmas. Las mejoras observadas en la tasa de detección, la precisión, la reducción de falsos negativos y la optimización del tiempo de respuesta fueron estadísticamente validadas, confirmando la eficacia de la propuesta en un entorno educativo real. Estos hallazgos constituyen la base para la discusión presentada en el siguiente capítulo, donde se contrastan los resultados obtenidos con los antecedentes teóricos y empíricos revisados, a fin de valorar su alcance, limitaciones y proyecciones.

CAPÍTULO V: DISCUSIÓN DE RESULTADOS

El presente capítulo expone la discusión de los hallazgos obtenidos tras la implementación y evaluación del sistema híbrido Snort + Machine Learning en la red perimetral del Colegio Hans Christian Andersen. La discusión se organiza en cuatro apartados:

- (i) Interpretación general de los resultados
- (ii) Comparación con el marco teórico y antecedentes
- (iii) Limitaciones del estudio
- (iv) Proyecciones para futuros trabajos.

5.1. Discusión general de hallazgos

Los resultados obtenidos en la fase de evaluación revelaron mejoras significativas en todas las métricas técnicas definidas. La tasa de detección se incrementó en 10.7%, lo que evidencia una mayor capacidad del sistema híbrido para identificar incidentes en comparación con Snort en su versión tradicional. Asimismo, la precisión aumentó en 12.1%, confirmando que las alertas emitidas por el sistema resultaron más confiables al reducirse la proporción de falsos positivos.

De particular importancia es la disminución del 72.6% en los falsos negativos, ya que este tipo de error representa el mayor riesgo en la gestión de seguridad: ataques no detectados que comprometen la integridad de la red. Finalmente, el tiempo de respuesta mejoró en un 32.4%, lo que permitió alertar más rápidamente a los responsables de la red. Estos hallazgos corroboran que la integración de Snort con modelos supervisados de Machine Learning constituye una alternativa viable y efectiva para entornos educativos con recursos limitados, donde las soluciones comerciales pueden resultar inaccesibles.

5.2. Comparación con antecedentes y marco teórico

Los resultados alcanzados muestran coherencia con las propuestas teóricas revisadas en el marco conceptual. Sostienen que la combinación de enfoques basados en firmas y técnicas de aprendizaje automático permite superar las limitaciones de los IDS tradicionales, particularmente en la detección de ataques novedosos. Este estudio confirma que los algoritmos Random Forest y SVM poseen un desempeño confiable en la clasificación de tráfico de red, reduciendo falsos negativos y optimizando la precisión del sistema.

Asimismo, los hallazgos coinciden con investigaciones recientes como las de Kim et al. (2021), que reportan mejoras en entornos educativos y experimentales al integrar IDS con Machine Learning. En este caso, la aplicación práctica en un colegio privado de Lima Metropolitana valida que tales propuestas no se limitan a entornos simulados, sino que pueden trasladarse a escenarios reales con resultados efectivos.

En términos metodológicos, la adopción de Scrum adaptado permitió estructurar la implementación en sprints, logrando entregables parciales verificables y facilitando ajustes en cada iteración. Esta experiencia coincide con lo señalado por Schwaber y Sutherland (2022), quienes destacan que la flexibilidad de Scrum lo hace aplicable no solo a proyectos empresariales, sino también a investigaciones académicas con alto componente tecnológico.

5.3. Limitaciones del estudio

A pesar de los resultados positivos, la investigación presenta algunas limitaciones que deben ser reconocidas:

- 1. Escenario restringido:** El piloto se realizó únicamente en la red perimetral de un colegio privado, lo que limita la generalización de los resultados a otros contextos educativos o empresariales con topologías más complejas.
- 2. Duración de las pruebas:** Los periodos de pretest y posttest fueron de 14 días cada uno, lo que ofrece una muestra representativa pero no necesariamente refleja variaciones estacionales o de mayor extensión temporal.
- 3. Recursos computacionales:** La implementación se realizó sobre una máquina virtual con capacidades limitadas. En entornos de mayor escala, sería necesario evaluar la escalabilidad del sistema y su impacto en el rendimiento de la red.
- 4. Dependencia de dataset externo:** Aunque el entrenamiento inicial utilizó el dataset CICIDS2017, reconocido en la comunidad científica, la naturaleza simulada de este conjunto de datos puede no representar en su totalidad los ataques más recientes o específicos del contexto local.

5.4. Proyecciones y posibles mejoras

Con base en las limitaciones señaladas, se plantean las siguientes proyecciones para investigaciones futuras:

- 1. Ampliación del escenario de validación:** Replicar el experimento en instituciones educativas de mayor tamaño o en redes empresariales, con el fin de evaluar la robustez y escalabilidad del sistema híbrido.
- 2. Extensión del periodo de pruebas:** Implementar evaluaciones a largo plazo (meses o semestres) que permitan identificar tendencias más estables en el desempeño del IDS híbrido.
- 3. Optimización de recursos:** Migrar la solución hacia entornos de virtualización de mayor capacidad o hacia contenedores (Docker), con el fin de mejorar la eficiencia en el uso de recursos.
- 4. Actualización continua de modelos:** Desarrollar mecanismos automáticos de reentrenamiento de los modelos de Machine Learning con datos recientes, de manera que el sistema pueda adaptarse dinámicamente a nuevas formas de ataque.
- 5. Integración con plataformas SIEM (Security Information and Event Management):** Incorporar la salida del IDS híbrido a sistemas de gestión centralizada de eventos, lo que potenciaría la capacidad de respuesta y la correlación de alertas en escenarios más complejos.

CAPÍTULO VI: CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

- La implementación del sistema de detección de intrusos híbrido, basado en la integración de Snort con modelos supervisados de Machine Learning (Random Forest y SVM), demostró una mejora significativa en el desempeño frente al enfoque tradicional únicamente basado en firmas. Esta conclusión se sustenta en el incremento de la tasa de detección (10.7%), el aumento de la precisión (12.1%), la reducción de falsos negativos (72.6%) y la optimización del tiempo de respuesta (32.4%).
- El diseño preexperimental aplicado en el Colegio Hans Christian Andersen permitió validar la eficacia del sistema híbrido en un entorno educativo real, confirmando que es posible implementar soluciones de ciberseguridad avanzadas en instituciones con recursos limitados, sin necesidad de recurrir a tecnologías comerciales de alto costo.
- El uso de la metodología ágil Scrum, adaptada a fines académicos, facilitó la organización del trabajo en sprints iterativos, lo que garantizó entregables parciales, revisiones periódicas y la posibilidad de ajustes incrementales durante el proceso de implementación. Esta experiencia confirma la pertinencia de emplear metodologías ágiles en proyectos de investigación tecnológica.
- El entrenamiento de los modelos con el dataset CICIDS2017 y su posterior ajuste con datos reales de la red escolar permitió desarrollar un mecanismo de clasificación confiable y replicable. Esta estrategia metodológica demostró la importancia de combinar datasets públicos reconocidos con información contextualizada al entorno de aplicación.
- Los resultados estadísticos obtenidos ($p < 0.05$ en todas las métricas evaluadas) permiten concluir que las mejoras alcanzadas con el sistema híbrido son estadísticamente significativas, lo que refuerza la validez científica de la propuesta y respalda la hipótesis de investigación planteada.

6.2. Recomendaciones

- Se recomienda a las instituciones educativas privadas implementar sistemas de detección de intrusos híbridos como el desarrollado en esta investigación, dado que representan una alternativa eficiente y de bajo costo para fortalecer la seguridad de sus redes frente a ciberataques.
- Es aconsejable ampliar los periodos de prueba a mayor escala temporal, lo que permitirá identificar tendencias más estables en el desempeño del sistema y validar su capacidad de adaptación a variaciones en el tráfico de red.
- Se sugiere mantener un proceso continuo de actualización y reentrenamiento de los modelos de Machine Learning con datos recientes, a fin de garantizar la detección de nuevas formas de ataque y evitar la obsolescencia del sistema.
- Para proyectos futuros, se recomienda explorar la integración del IDS híbrido con plataformas de gestión centralizada de seguridad (SIEM), lo que permitiría una correlación más amplia de eventos y una respuesta más rápida frente a incidentes complejos.
- Finalmente, se aconseja a los investigadores replicar este estudio en redes de distinta naturaleza (universitarias, empresariales o gubernamentales), con el fin de evaluar la escalabilidad del sistema y consolidar un modelo referencial de aplicación en el contexto peruano.

REFERENCIAS

- [1]. World Economic Forum, "Global Cybersecurity Outlook 2023". [En línea]. Disponible en: <https://www.weforum.org/reports/global-cybersecurity-outlook-2023>
- [2]. Fortinet, "Global Threat Landscape Report". [En línea]. Disponible en: <https://www.fortinet.com/resources/cyberthreat-reports>
- [3]. Check Point Research, "Reporte de amenazas 2025," América Sistemas. [En línea]. Disponible en: <https://www.americasistemas.com.pe/reporte-de-amenazas-2025-checkpoint>
- [4]. ESET, "Perú sigue siendo el país con más amenazas detectadas," eBIZ Noticias. [En línea]. Disponible en: <https://ebiz.pe/noticias/peru-sigue-siendo-el-pais-con-mas-amenazas-detectadas>
- [5]. Tarlogic, "Ciberataques al sector educativo: vectores y vulnerabilidades." [En línea]. Disponible en: <https://www.tarlogic.com/es/blog/ciberataques-sector-educativo>
- [6]. Congreso de la República del Perú, "Ley N.º 29733, Ley de Protección de Datos Personales" y "Decreto Supremo N.º 003–2013–JUS – Reglamento", 2011. [En línea]. Disponible en: <https://www.leyes.congreso.gob.pe/documentos/leyes/29733.pdf>
- [7]. B. Figueroa y J. Cañihua, "Sistema de intrusión de tráfico de red basado en Machine Learning para la protección de datos en la empresa, Lima, 2024", Universidad Tecnológica del Perú, Lima, 2024.
- [8]. H. L. Huamani Santiago, "Snort Open Source como sistema de detección de intrusos para la seguridad de la infraestructura de red en entornos libres aplicado a PYMEs del Perú", Universidad Nacional San Cristóbal de Huamanga, Ayacucho, 2020.
- [9]. J. G. Lascano Banshuy, "Sistema de detección de intrusos (IDS) basado en Machine Learning para el control de la red en la Unidad Educativa "19 de Septiembre" en la ciudad de Salcedo", Universidad Técnica de Ambato, Ambato, 2025.
- [10]. L. E. Carrizo García y M. D. Álvarez Galarza, "Uso de Software Libre y Machine Learning para mejorar la Detección de Intrusos en una Red", 2023 *Revista Latinoamericana de*

Tecnología Informática, vol. 12, no. 2, pp. 55–68, 2023.

- [11]. O. El Aeraj and C. Leghris, “Intelligent Intrusion Detection System Snort and SVM,” *Journal of Advanced Information Technology*, vol. 15, no. 2, pp. 54–62, 2024.
- [12]. S. Ouiazzane, M. Addou, and F. Barramou, “A Multiagent and Machine Learning based Hybrid NIDS for Known and Unknown Cyber-attacks,” *Revista Ibérica de Sistemas e Tecnologías de Informação*, no. 37, pp. 259–271, 2021.
- [13]. M. AbdulRaheem et al., “Machine learning assisted snort and zeek in detecting DDoS attacks in software-defined networking,” *Security and Communication Networks*, vol. 2022, Article ID 8113855, 2022.
- [14]. X. Sun, D. Zhang, H. Qin, and J. Tang, “Bridging the Last-Mile Gap in Network Security via Generating Intrusion-Specific Detection Patterns through Machine Learning,” *Security and Communication Networks*, vol. 2022, Article ID 8167032, 2022.
- [15]. N. Daniel et al., “Labeling Network Intrusion Detection System (NIDS) Rules with MITRE ATT&CK Techniques: Machine Learning vs. Large Language Models,” *Big Data and Cognitive Computing*, vol. 9, no. 1, pp. 1–27, 2023.
- [16]. F. Alserhani, “Enhancing Intrusion Detection System Evaluation: A Framework for Generating Comprehensive and Scalable Datasets,” *International Journal of Electrical and Computer Engineering*, vol. 11, no. 10, pp. 8471–8478, 2021.
- [17]. Sourcefire, “Snort User Manual,” Cisco Systems, 2022. [En línea]. Disponible: <https://docs.snort.org>
- [18]. C. Chio y D. Freeman, *Machine Learning and Security: Protecting Systems with Data and Algorithms*. Sebastopol, CA, USA: O’Reilly Media, 2018.
- [19]. M. Hadi, C. Martel, F. Huayta, R. Rojas, y J. Arias, *Metodología de la investigación: Guía para el proyecto de tesis*, 1.a ed., Lima: Instituto Universitario de Innovación Ciencia y Tecnología Inudi Perú, 2023.
- [20]. M. Medina, R. Rojas, W. Bustamante, R. Loaiza, C. Martel, y R. Castillo, *Metodología de la investigación: Técnicas e instrumentos de investigación*, 1.a ed., Lima: Instituto

Universitario de Innovación Ciencia y Tecnología Inudi Perú, 2023.

- [21]. C. B. Ruiz Huaraz, A. Valenzuela Muñoz, y M. R. Valenzuela Ramos, Introducción Metodología de Investigación, 1.a ed., Lima: Fondo Editorial Professionals On Line, 2023.
- [22]. Montes Gil, J. A., Isaza Cadavid, G., & Duque Méndez, N. D. (2023). Efecto de la selección de atributos en el desempeño de un IDS basado en machine learning para detección de intrusos en ataques DDoS. *South Florida Journal of Development*, 4(2), 918–928. <https://doi.org/10.46932/sfjdv4n2-023>
- [23]. Huaraca Núñez, J. A., Cervantes Ccasa, A., & Aquino Cruz, M. (2024). Técnicas de machine learning para la detección de intrusos en redes: Una revisión sistemática de la literatura. *Micaela Revista de Investigación - UNAMBA*, vol. 5, no. 2, pp. 17–24, 2024, doi: 10.57166/micaela.v5.n2.2024.151.
- [24]. F. Flores-Cerna, V.-M. Sanhueza-Salazar, H.-M. Valdés-González y L. Reyes-Bozo, “Metodologías ágiles: Un análisis de los desafíos organizacionales para su implementación,” *Revista Científica*, vol. 43, no. 1, pp. 38–49, 2021. doi: 10.14483/23448350.18332.
- [25]. Al Hasan, R., & Kadhum Hamza, E. (2023). An Improved Intrusion Detection System Using Machine Learning with Singular Value Decomposition and Principal Component Analysis. *International Journal of Intelligent Engineering and Systems*, 16(4), 25–38. <https://doi.org/10.22266/ijies2023.0831.03>
- [26]. Ayuba, J., Isnin, I. F. B., Madni, S. H. H., & Muchtar, F. B. (2024). Enhanced intrusion detection model based on principal component analysis and variable ensemble machine learning algorithm. *Intelligent Systems with Applications*, 24, 200442. <https://doi.org/10.1016/j.iswa.2024.200442>
- [27]. Baklizi, M. K., Atoum, I., Alkhazaleh, M., Kanaker, H., Abdullah, N., Al-Wesabi, O. A., & Otoom, A. (2024). Web Attack Intrusion Detection System Using Machine Learning Techniques. *International Journal of Online and Biomedical Engineering (iJOE)*, 20(03), 24–38. <https://doi.org/10.3991/ijoe.v20i03.45249>
- [28]. Caizapanta González, A. E. (2022). Análisis comparativo de Sistemas de Detección de

Intrusos (IDS) en entornos universitarios. *Revista Tecnológica - ESPOL*, 34(3), 118–138.
<https://doi.org/10.37815/rte.v34n3.955>

- [29]. Chimphee, S., & Chimphee, W. (2023). Machine learning to improve the performance of anomaly-based network intrusion detection in big data. *Indonesian Journal of Electrical Engineering and Computer Science*, 30(2), 1106.
<https://doi.org/10.11591/ijeecs.v30.i2.pp1106-1119>
- [30]. Doghramachi, D. F., & Ameen, S. Y. (2023). Internet of Things (IoT) Security Enhancement Using XGboost Machine Learning Techniques. *Computers, Materials & Continua*, 77(1), 717–732. <https://doi.org/10.32604/cmc.2023.041186>
- [31]. Gutierrez-Garcia, J. L., Sanchez-DelaCruz, E., & Pozos-Parra, M. D. P. (2023). A Review of Intrusion Detection Systems Using Machine Learning: Attacks, Algorithms and Challenges. En K. Arai (Ed.), *Advances in Information and Communication* (Vol. 652, pp. 59–78). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-28073-3_5
- [32]. Itati, S., & Luis, P. (2022). Agilidad en la gestión de proyectos de graduación. 20(3).
- [33]. Rodríguez, E., Otero, B., & Canal, R. (2023). A Survey of Machine and Deep Learning Methods for Privacy Protection in the Internet of Things. *Sensors*, 23(3), 1252.
<https://doi.org/10.3390/s23031252>
- [34]. Saba, T., Rehman, A., Sadad, T., Kolivand, H., & Bahaj, S. A. (2022). Anomaly-based intrusion detection system for IoT networks through deep learning model. *Computers and Electrical Engineering*, 99, 107810.
<https://doi.org/10.1016/j.compeleceng.2022.107810>
- [35]. Sajid Farooq, M., Abbas, S., Atta-ur-Rahman, Sultan, K., Adnan Khan, M., & Mosavi, A. (2023). A Fused Machine Learning Approach for Intrusion Detection System. *Computers, Materials & Continua*, 74(2), 2607–2623. <https://doi.org/10.32604/cmc.2023.032617>
- [36]. Suárez, J. R. E., & Rodriguez, J. E. P. (2023). Análisis integral de los sistemas de detección de intrusos y sus algoritmos asociados en la seguridad de la información. 5.
- [37]. Tarish, H. A., Hassan, R., Ariffin, K. A. Z., & Jaber, M. M. (2024). Network security framework for Internet of medical things applications: A survey. *Journal of Intelligent*

Systems, 33(1), 20230220. <https://doi.org/10.1515/jisys-2023-0220>

- [38]. Vallejo, C. F. M. (2021). Inteligencia Artificial y el Aprendizaje Automático en la Ciberseguridad.
- [39]. Vega Briceño, E. (2021). Seguridad de la información (1.a ed.). Editorial Científica 3Ciencias. <https://doi.org/10.17993/tics.2021.4>

ANEXOS

ANEXO 1: Matriz de Consistencia

Tema de Investigación	"Implementación de un IDS basado en Snort y técnicas de Machine Learning para redes perimetrales en colegios privados de Lima Metropolitana"					
PROBLEMA	OBJETIVOS	HIPÓTESIS	VARIABLES	DIMENCIONES	INDICADORES	METODOLOGIA
Problema General: ¿Cómo mejora la integración de técnicas de Machine Learning en un sistema IDS basado en Snort la detección de amenazas y la reducción de la exposición en redes perimetrales de instituciones educativas privadas, considerando el caso del Colegio Hans Christian Andersen? Problemas específicos: ¿Cuáles son las principales vulnerabilidades en las redes perimetrales de las instituciones educativas privadas de nivel escolar en Lima Metropolitana que no son detectadas por los sistemas de detección de intrusos tradicionales? ¿Qué limitaciones presenta Snort, como sistema basado en reglas estáticas, frente a patrones de ciberataques modernos en el contexto educativo? ¿Cómo pueden las técnicas de Machine Learning complementar a Snort para identificar y clasificar amenazas emergentes en entornos escolares? ¿Cuál es el nivel actual de exposición a ciberataques en las redes perimetrales del Colegio Hans Christian Andersen y qué riesgos concretos enfrenta en ausencia de detección inteligente?	Objetivo General: Implementar un sistema de detección de intrusos híbrido, basado en Snort y técnicas de Machine Learning, que permita mejorar la tasa de detección de amenazas, reducir falsos positivos y optimizar el tiempo de respuesta en la red perimetral del Colegio Hans Christian Andersen, como modelo aplicable a instituciones educativas privadas escolares de Lima Metropolitana. Objetivos Específicos: - Identificar las principales vulnerabilidades presentes en las redes perimetrales de instituciones educativas privadas escolares de Lima Metropolitana que no son detectadas por sistemas IDS tradicionales. - Analizar las limitaciones técnicas del sistema Snort, como motor de detección basado en reglas, frente a patrones de ciberataques avanzados en entornos educativos. - Diseñar e integrar técnicas de Machine Learning supervisadas como complemento a Snort, orientadas a mejorar la tasa de detección y reducir los falsos positivos en tiempo real. - Evaluar la efectividad del sistema híbrido propuesto mediante su implementación experimental en la red perimetral del Colegio Hans Christian Andersen, midiendo métricas como tasa de detección, precisión y tiempo de respuesta.	Hipótesis General: La implementación de un sistema de detección de intrusos híbrido, basado en Snort y técnicas de Machine Learning, mejorará significativamente el desempeño en la detección de intrusos en la red perimetral del Colegio Hans Christian Andersen, incrementando la tasa de detección, reduciendo los falsos positivos y optimizando el tiempo de respuesta. Hipótesis Específicas: - Las redes perimetrales de instituciones educativas privadas escolares de Lima Metropolitana presentan vulnerabilidades no detectadas por sistemas IDS tradicionales, lo cual se evidencia en una tasa de detección menor al 80 % y una tasa de falsos negativos superior al 15 % ante ataques conocidos. - El sistema Snort, en su configuración convencional basada en firmas, presenta una precisión inferior al 75 % en la detección de amenazas emergentes, lo que limita su eficacia frente a patrones de ataque avanzados en entornos escolares. - La integración de algoritmos de Machine Learning con Snort incrementa la tasa de detección en al menos un 20 % y reduce los falsos positivos en un 15 %, en comparación con la configuración estándar de Snort, permitiendo una detección más eficiente de comportamientos anómalos en tiempo real. - La aplicación del sistema híbrido propuesto en la red perimetral del Colegio Hans Christian Andersen alcanza una tasa de detección superior al 90 %, una tasa de falsos negativos inferior al 10 %, y un tiempo de respuesta promedio menor a 3 segundos, mejorando los indicadores clave de seguridad perimetral.	Variable independiente (VI): Implementación del IDS híbrido (Snort + ML)	Integración de componentes	Índice de Aplicación del Tratamiento (IAT)	Tipo de investigación Aplicada Diseño Preexperimental Enfoque Cuantitativo Alcance Tecnológico y explicativo Población Infraestructura de red perimetral del Colegio Hans Christian Andersen
				Entrenamiento del modelo		
				Implementación híbrida		
				Despliegue en entorno real		
			Variable dependiente (VD): Desempeño del sistema IDS	Capacidad de detección	Tasa de detección	Muestra Red perimetral activa del colegio Variable 1: Implementación del IDS híbrido (Snort + ML) Variable 2: Desempeño del sistema IDS Instrumentos Ficha de validación de juicio de expertos
				Precisión de clasificación	Precisión	
				Fiabilidad del sistema	Falsos negativos	
				Eficiencia operativa	Tiempo de respuesta	

ANEXO 2: Guía de observación técnica estructurada

Título de la investigación:

Implementación de un IDS basado en Snort y técnicas de Machine Learning para redes perimetrales en colegios privados de Lima Metropolitana.

Objetivo del instrumento:

Evaluar el desempeño del sistema de detección de intrusos híbrido (Snort + Machine Learning) en función de los indicadores técnicos definidos: **tasa de detección, precisión, falsos negativos y tiempo de respuesta**, antes y después de su implementación en la red del Colegio Hans Christian Andersen.

Descripción del instrumento:

Este instrumento es de tipo observación técnica estructurada. Permite registrar datos objetivos de desempeño del sistema en condiciones reales, a partir del análisis de tráfico de red y los resultados generados por Snort y el modelo de Machine Learning integrado.

Indicador Técnico	Definición Operativa	Método de Registro	Unidad de Medida	Momento de Aplicación
Tasa de detección (%)	Porcentaje de amenazas correctamente identificadas por el sistema sobre el total de amenazas presentes.	Análisis de alertas en Snort y validación cruzada con Wireshark.	Porcentaje (%)	Pretest y Postest
Precisión (%)	Porcentaje de alertas generadas por el sistema que efectivamente corresponden a amenazas reales.	Cálculo con matriz de confusión (TP / TP+FP) a partir de logs del sistema.	Porcentaje (%)	Pretest y Postest
Falsos negativos (%)	Porcentaje de amenazas reales que el sistema no fue capaz de detectar.	Cálculo con matriz de confusión (FN / FN+TP) validada con etiquetado manual.	Porcentaje (%)	Pretest y Postest
Tiempo de respuesta (s)	Tiempo promedio transcurrido entre el inicio de una amenaza y la emisión de alerta al sistema.	Comparación de timestamps entre eventos registrados y alertas emitidas.	Segundos (s)	Pretest y Postest

ANEXO 3: Ficha de recolección de datos

Tasa de Detección:

FICHA DE RECOLECCIÓN DE INFORMACIÓN – INDICADOR: Tasa de Detección (%)			
DATOS GENERALES		DESCRIPCIÓN	
ÁREA:	Sistemas y Redes	Registrar el porcentaje de amenazas detectadas correctamente por el IDS híbrido (Snort + ML).	
RESPONSABLE:	Tesista / Equipo de Seguridad		
FUENTE:	Logs de Snort y capturas de Wireshark		
INDICADOR:	Tasa de Detección (%)		
UNIDAD DE MEDIDA:	%		
SISTEMA ANALIZADO:	Pretest (Snort)	Postest (Snort + ML)	
RESUMEN POR FASE (cálculo automático)			
Métrica	Pre	Post	Δ Mejora (Post-Pre)
TP	0	0	0
FP	0	0	0
FN	0	0	0
TN	0	0	0
Tasa de detección (TP/(TP+FN))	0.00%	0.00%	0.00%
Precisión (TP/(TP+FP))	0.00%	0.00%	0.00%
Tasa de falsos negativos (FN/(FN+TP))	0.00%	0.00%	0.00%
Tiempo de respuesta (promedio TP, s)	0.00	0.00	0.00

Precisión:

FICHA DE RECOLECCIÓN DE INFORMACIÓN – INDICADOR: Precisión (%)			
DATOS GENERALES		DESCRIPCIÓN	
ÁREA:	Sistemas y Redes	Registrar la proporción de alertas que corresponden a amenazas reales.	
RESPONSABLE:	Tesista / Equipo de Seguridad		
FUENTE:	Logs de Snort + matriz de confusión		
INDICADOR:	Precisión (%)		
UNIDAD DE MEDIDA:	%		
SISTEMA ANALIZADO:	Pretest (Snort)	Postest (Snort + ML)	
RESUMEN POR FASE (cálculo automático)			
Métrica	Pre	Post	Δ Mejora (Post-Pre)
TP			
FP	0	0	0
FN	0	0	0
TN	0	0	0
Tasa de detección (TP/(TP+FN))	0.00%	0.00%	0.00%
Precisión (TP/(TP+FP))	0.00%	0.00%	0.00%
Tasa de falsos negativos (FN/(FN+TP))	0.00%	0.00%	0.00%
Tiempo de respuesta (promedio TP, s)	0.00	0.00	0.00

Falsos Negativos:

FICHA DE RECOLECCIÓN DE INFORMACIÓN – INDICADOR: Falsos Negativos (%)			
DATOS GENERALES		DESCRIPCIÓN	
ÁREA:	Sistemas y Redes	Registrar las amenazas reales no detectadas por el sistema.	
RESPONSABLE:	Tesista / Equipo de Seguridad		
FUENTE:	Logs de Snort + validación manual		
INDICADOR:	Falsos Negativos (%)		
UNIDAD DE MEDIDA:	%		
SISTEMA ANALIZADO:	Pretest (Snort)	Posttest (Snort + ML)	
RESUMEN POR FASE (cálculo automático)			
Métrica	Pre	Post	Δ Mejora (Post-Pre)
TP	0	0	0
FP	0	0	0
FN	0	0	0
TN	0	0	0
Tasa de detección (TP/(TP+FN))	0.00%	0.00%	0.00%
Precisión (TP/(TP+FP))	0.00%	0.00%	0.00%
Tasa de falsos negativos (FN/(FN+TP))	0.00%	0.00%	0.00%
Tiempo de respuesta (promedio TP, s)	0.00	0.00	0.00

Tiempo de respuesta:

FICHA DE RECOLECCIÓN DE INFORMACIÓN – INDICADOR: Tiempo de Respuesta (s)			
DATOS GENERALES		DESCRIPCIÓN	
ÁREA:	Sistemas y Redes	Registrar el tiempo medio entre inicio del evento y alerta.	
RESPONSABLE:	Tesista / Equipo de Seguridad		
FUENTE:	Timestamps de eventos y alertas		
INDICADOR:	Tiempo de Respuesta (s)		
UNIDAD DE MEDIDA:	s		
SISTEMA ANALIZADO:	Pretest (Snort)	Posttest (Snort + ML)	
RESUMEN POR FASE (cálculo automático)			
Métrica	Pre	Post	Δ Mejora (Post-Pre)
TP	0	0	0
FP	0	0	0
FN	0	0	0
TN	0	0	0
Tasa de detección (TP/(TP+FN))	0.00%	0.00%	0.00%
Precisión (TP/(TP+FP))	0.00%	0.00%	0.00%
Tasa de falsos negativos (FN/(FN+TP))	0.00%	0.00%	0.00%
Tiempo de respuesta (promedio TP, s)	0.00	0.00	0.00

ANEXO 4: Juicio de Experto

 Universidad Tecnológica del Perú		FICHA DE VALIDACIÓN DE INSTRUMENTOS							
1. OBJETIVO:									
Evaluar la validez de contenido del instrumento de ficha de recolección de datos, mediante el juicio de expertos en Inteligencia Artificial, Ciberseguridad o Ingeniería de Sistemas con grado académico mínimo de Magíster registrado en SUNEDU.									
2. DATOS DE LA INVESTIGACIÓN									
2.1. TÍTULO:	Implementación de un IDS basado en Snort y técnicas de Machine Learning para redes perimetrales en colegios privados de Lima Metropolitana								
2.2. TESISTA:	Luis Alonso Torres Valle Yussel David Abensur Rojas								
3. DATOS DEL EVALUADOR									
3.1. NOMBRES Y APELLIDOS:	Jorge Quispe Sarmiento								
3.2. GRADO ACADÉMICO:	Magister en Ingeniería de Sistemas								
3.3. ESPECIALIDAD O PROFESIÓN:	Redes y Telecomunicaciones								
3.4. CIP (SI APLICA):									
3.5. REGISTRO SUNEDU:									
4. INSTRUCCIONES									
A continuación, encontrará los indicadores técnicos que componen el instrumento. Por favor de calificar cada uno según los criterios propuestos, usando la siguiente escala:					Muy deficiente	Deficiente	Regular	Bueno	Excelente
					1	2	3	4	5
5. TABLA DE EVALUACIÓN: CALIFICAR DE 1 A 5 SEGÚN CORRESPONDA									
FICHA DE RECOLECCIÓN DEL INDICADOR	A. CLARIDAD	B. PERTINENCIA	C. COHERENCIA	D. RELEVANCIA	PF (A+B+C+D)/4	OBSERVACIONES			
Tasa de detección (%)	5	4	4	5	4.5				
Presición (%)	4	5	4	5	4.5				
Falsos negativos (%)	3	5	4	5	4.25				
Tiempo de respuesta (s)	5	5	4	5	4.75				
6. CALIFICACIÓN Y FIRMA DEL EXPERTO									
La calificación final para el instrumento es de:	TOTAL PF/4	4.5		FECHA:	17/09/2025				
 Jorge Quispe Sarmiento Jefe de TI									
FIRMA DEL EVALUADOR									

FICHA DE VALIDACIÓN DE INSTRUMENTOS

1. OBJETIVO:

Evaluar la validez de contenido del instrumento de ficha de recolección de datos, mediante el juicio de expertos en Inteligencia Artificial, Ciberseguridad o Ingeniería de Sistemas con grado académico mínimo de Magíster registrado en SUNEDU.

2. DATOS DE LA INVESTIGACIÓN

2.1. TÍTULO: Implementación de un IDS basado en Snort y técnicas de Machine Learning para redes perimetrales en colegios privados de Lima Metropolitana

2.2. TESIS: Luis Alonso Torres Valle
Yussel David Abensur Rojas

3. DATOS DEL EVALUADOR

3.1. NOMBRES Y APELLIDOS: Oscar Mejia Alva

3.2. GRADO ACADÉMICO: Magíster en Ingeniería de Sistemas e Informática

3.3. ESPECIALIDAD O PROFESIÓN: Ciberseguridad

3.4. CIP (SI APLICA):

3.5. REGISTRO SUNEDU:

4. INSTRUCCIONES

A continuación, encontrará los indicadores técnicos que componen el instrumento. Por favor de calificar cada uno según los criterios propuestos, usando la siguiente escala:

Muy deficiente	Deficiente	Regular	Bueno	Excelente
1	2	3	4	5

5. TABLA DE EVALUACIÓN: CALIFICAR DE 1 A 5 SEGÚN CORRESPONDA

FICHA DE RECOLECCIÓN DEL INDICADOR	A. CLARIDAD	B. PERTINENCIA	C. COHERENCIA	D. RELEVANCIA	PF (A+B+C+D)/4	OBSERVACIONES
Tasa de detección (%)	4	5	4	5	4.5	Correcto; detallar validación cruzada con Wireshark.
Presición (%)	5	4	4	5	4.5	Bien formulado; convendría agregar ejemplo de cálculo.
Falsos negativos (%)	4	5	4	5	4.5	Indicador válido, pero se recomienda definir umbral.
Tiempo de respuesta (s)	5	5	5	5	5	Muy claro y pertinente.

6. CALIFICACIÓN Y FIRMA DEL EXPERTO

La calificación final para el instrumento es de:

TOTAL PF/4

4.625

FECHA:

12/09/2025

FIRMA DEL EVALUADOR

1. OBJETIVO:

Evaluar la validez de contenido del instrumento de ficha de recolección de datos, mediante el juicio de expertos en Inteligencia Artificial, Ciberseguridad o Ingeniería de Sistemas con grado académico mínimo de Magister registrado en SUNEDU.

2. DATOS DE LA INVESTIGACIÓN

2.1. TÍTULO: Implementación de un IDS basado en Snort y técnicas de Machine Learning para redes perimetrales en colegios privados de Lima Metropolitana

2.2. TESIS: Luis Alonso Torres Valle
Yussel David Abensur Rojas

3. DATOS DEL EVALUADOR

3.1. NOMBRES Y APELLIDOS: Elvis Pérez Noe Silva

3.2. GRADO ACADÉMICO: Magister en Ingeniería de Sistemas

3.3. ESPECIALIDAD O PROFESIÓN: Ciencias de la Computación

3.4. CIP (SI APLICA):

3.5. REGISTRO SUNEDU:

4. INSTRUCCIONES

A continuación, encontrará los indicadores técnicos que componen el instrumento. Por favor de calificar cada uno según los criterios propuestos, usando la siguiente escala:

Muy deficiente	Deficiente	Regular	Bueno	Excelente
1	2	3	4	5

5. TABLA DE EVALUACIÓN: CALIFICAR DE 1 A 5 SEGÚN CORRESPONDA

FICHA DE RECOLECCIÓN DEL INDICADOR	A. CLARIDAD	B. PERTINENCIA	C. COHERENCIA	D. RELEVANCIA	PF (A+B+C+D)/4	OBSERVACIONES
Tasa de detección (%)	5	4	4	5	4.5	Bien definido, faltaría un ejemplo práctico en la guía.
Presición (%)	4	5	4	5	4.5	Correcto, pero sería útil mostrar fórmula en la ficha.
Falsos negativos (%)	3	5	4	5	4.25	Importante, pero aclarar si se medirá sobre muestras
Tiempo de respuesta (s)	5	5	4	5	4.75	Muy bien, aunque sugerir "promedio" en la definición

6. CALIFICACIÓN Y FIRMA DEL EXPERTO

La calificación final para el instrumento es de:

TOTAL PF/4

4.5

FECHA:

19/09/2025



FIRMA DEL EVALUADOR

ANEXO 5: Configuración del Entorno Virtual Ubuntu IDS

En este anexo se documenta la instalación, configuración de red, sincronización horaria y creación de snapshot del entorno virtual Ubuntu IDS, utilizado como base para la implementación de la solución propuesta.

1. Instalación de Ubuntu en VMware Workstation

Se instaló Ubuntu Server como sistema operativo base en una máquina virtual configurada en VMware Workstation. Se asignaron 8 GB de memoria RAM, 2 procesadores y la interfaz de red en modo Bridge.

Tabla 9: Configuración del entorno virtual de pruebas

Equipo	Sistema Operativo	Características	Software Instalado
PC Virtual	Ubuntu 24.04.3 LTS	– Memoria RAM: 8 GB – Procesador: 2 núcleos – Disco duro virtual: 40 GB – Configuración de red: Bridge (modo puente)	– VMware Workstation – Snort IDS – NTP – Wireshark

Fuente: (Elaboración propia)

Figura 10: Configuración de la máquina virtual en VMware.

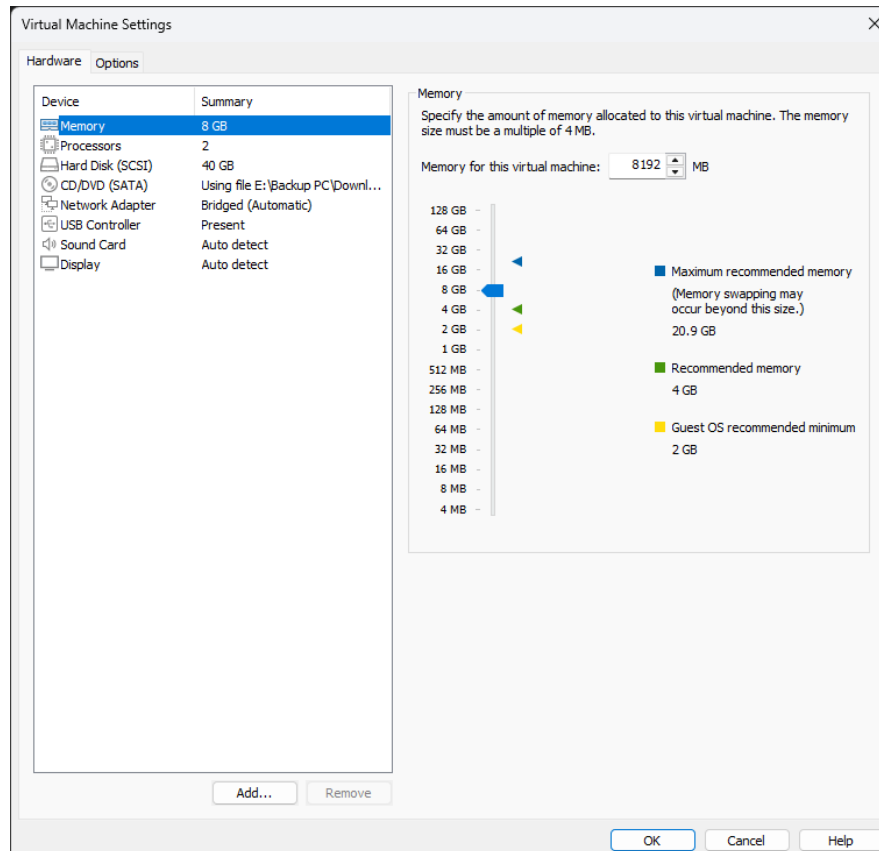


Figura 11: Proceso de instalación de Ubuntu

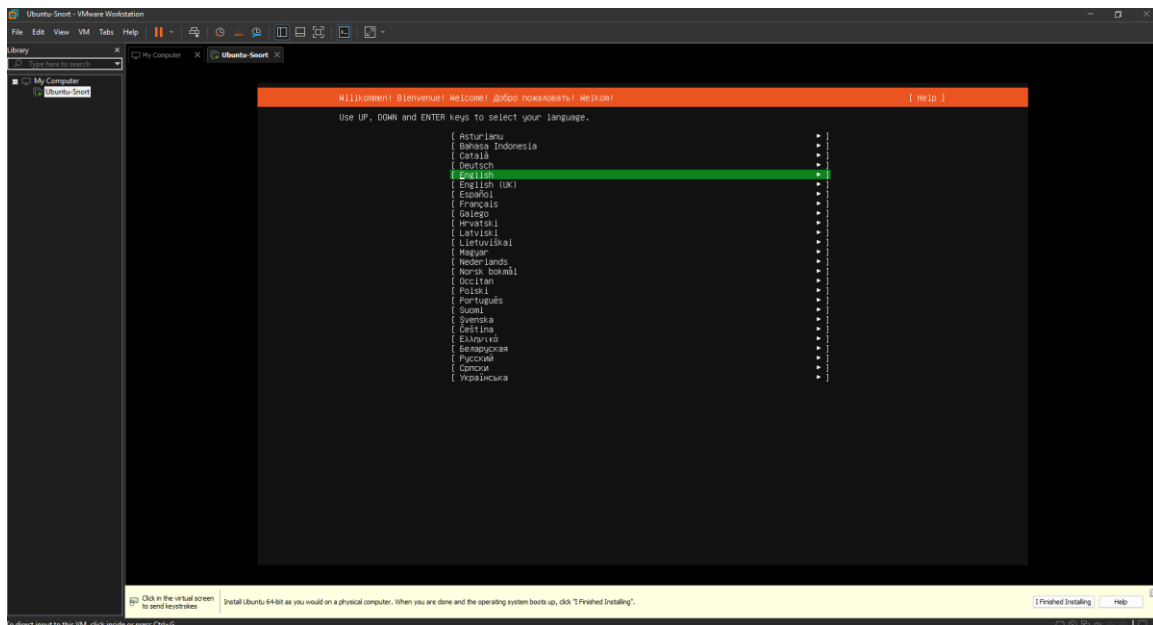
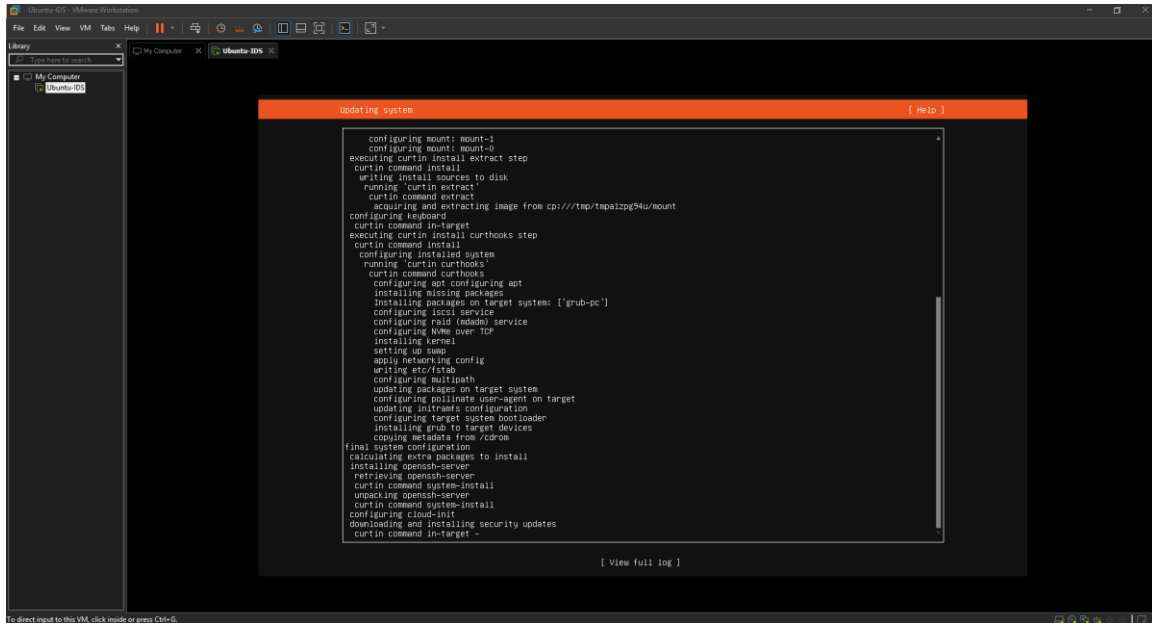


Figura 12: Proceso de instalación de Ubuntu

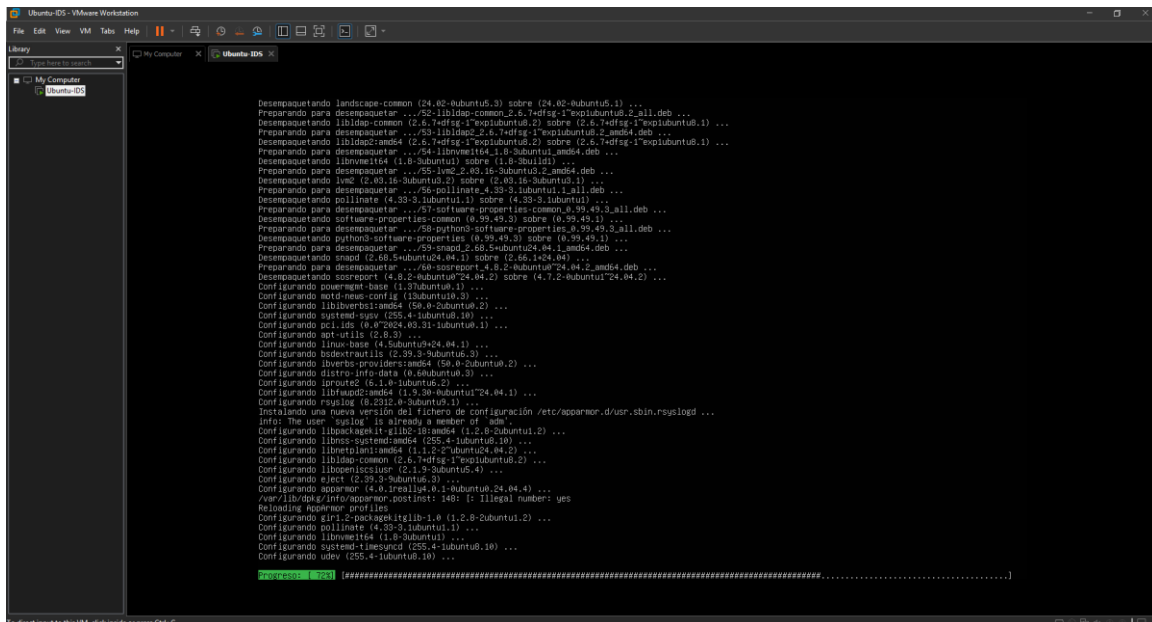


2. Actualización del Sistema

Se procedió a la actualización del sistema operativo y paquetes base mediante el comando:

```
sudo apt update && sudo apt upgrade -y
```

Figura 13: Actualización del sistema operativo



3. Configuración de Red

La interfaz de red ens33 se configuró en modo Bridge, obteniendo una dirección IP dinámica de la red local.

Figura 14: Verificación de la interfaz de red y asignación de dirección IP.

```
snort@ubuntu-ids:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:9e:56:6d brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.32/24 metric 100 brd 192.168.1.255 scope global dynamic ens33
        valid_lft 84968sec preferred_lft 84968sec
    inet6 2001:1388:a44:81cd:20c:29ff:fe9e:566d/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 258736sec preferred_lft 172336sec
    inet6 fe80::20c:29ff:fe9e:566d/64 scope link
        valid_lft forever preferred_lft forever
snort@ubuntu-ids:~$
```

4. Sincronización de Hora con NTP

Se instaló y configuró systemd-timesyncd para mantener sincronizado el reloj del sistema con servidores NTP.

Comandos utilizados:

```
sudo apt install systemd-timesyncd -y
```

```
sudo systemctl enable systemd-timesyncd
```

```
sudo systemctl start systemd-timesyncd
```

```
sudo timedatectl set-ntp on
```

```
timedatectl status
```

Figura 15: Verificación del estado de sincronización de la hora.

```
Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

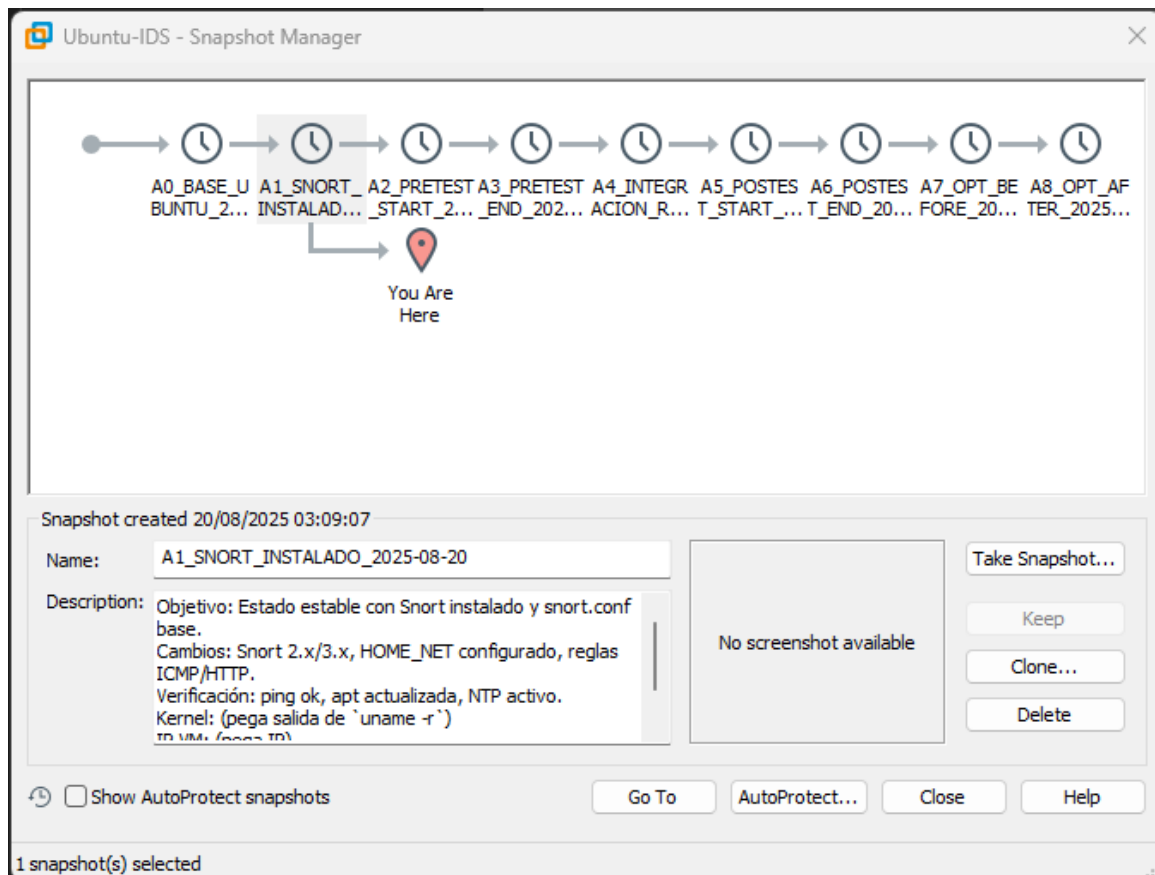
User sessions running outdated binaries:
snort @ session #5: login[1210]
snort @ user manager service: systemd[1499]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
snort@ubuntu-ids:~$ sudo systemctl enable systemd-timesyncd
snort@ubuntu-ids:~$ sudo systemctl start systemd-timesyncd
snort@ubuntu-ids:~$ sudo timedatectl set-ntp on
snort@ubuntu-ids:~$ timedatectl status
    Local time: mié 2025-08-20 05:19:24 UTC
    Universal time: mié 2025-08-20 05:19:24 UTC
        RTC time: mié 2025-08-20 05:19:24
        Time zone: Etc/UTC (UTC, +0000)
System clock synchronized: yes
    NTP service: active
    RTC in local TZ: no
snort@ubuntu-ids:~$
```

5. Creación de Snapshot

Con el entorno correctamente instalado y configurado, se generó un snapshot en VMware Workstation, el cual permite regresar al estado estable en caso de fallos durante las pruebas posteriores.

Figura 16: Creación de snapshot



ANEXO 6: Instalación y Configuración de Snort

Este anexo documenta la instalación y configuración base de Snort en Ubuntu 24.04.3 LTS, para operar como sensor IDS en la red perimetral. Incluye comandos ejecutados, parámetros clave del archivo de configuración y pruebas de verificación con reglas locales.

Tabla 10: Parámetros de configuración inicial de Snort

Parámetro	Valor
INTERFAZ_MONITOREO	ens33 (ajustar según salida de: ip -br addr show)
HOME_NET	[192.168.1.0/24]
EXTERNAL_NET	any
RUTA_CONFIG	/etc/snort/snort.conf
RUTA_REGLAS	/etc/snort/rules
ARCHIVO_REGLAS_LOCALES	/etc/snort/rules/local.rules
RUTA_LOGS	/var/log/snort

Fuente: (Elaboración propia)

Nota: En laboratorio se utilizó la subred local 192.168.1.0/24. En el despliegue real en el colegio, este parámetro se ajustará a [192.168.18.0/24, 192.168.19.0/24].

1. Prerrequisitos

- a. Actualizar el sistema:

```
sudo apt update && sudo apt upgrade -y
```

Figura 17: Actualización del Sistema

```
ago 21 22:44:47 ubuntu-ids systemd[1]: Started promisc@ens33.service - Set promiscuous mode on ens33.
ago 21 22:45:10 ubuntu-ids systemd[1]: /etc/systemd/system/promisc@.service:1: Assignment outside of section. Ignoring.
snort@ubuntu-ids:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for snort:
Des:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Obj:2 http://pe.archive.ubuntu.com/ubuntu noble InRelease
Des:3 http://pe.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Des:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1.083 kB]
Des:5 http://pe.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Des:6 http://pe.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1.350 kB]
Des:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21,6 kB]
Des:8 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Des:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [881 kB]
53% [6 Packages 758 kB/1.350 kB 56%] [9 Packages 111 kB/881 kB 13%]
```

- b. Comando para identificar la interfaz de red que usa Snort:

```
ip -br addr show
```


Figura 20: Versión de Snort instalada.

```
snort@ubuntu-ids:~$ snort -V
o''~)~
'''

-*> Snort! <*-
Version 2.9.20 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.4 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.3

snort@ubuntu-ids:~$ _
```

3. Configuración Inicial

- Editar el archivo de configuración principal:

```
sudo nano /etc/snort/snort.conf
```

- Ajustar al inicio del archivo las variables de red:

```
ipvar HOME_NET 192.168.18.0/24
```

```
ipvar EXTERNAL_NET any
```

Figura 21: Fragmento del archivo *snort.conf* con la variable de red.

```
GNU nano 7.2 /etc/snort/snort.conf
# Setup the network addresses you are protecting
#
# Note to Debian users: this value is overridden when starting
# up the Snort daemon through the init.d script by the
# value of DEBIAN_SNORT_HOME_NET s defined in the
# /etc/snort/snort.debian.conf configuration file
#
ipvar HOME_NET 192.168.1.0/24

# Set up the external network addresses. Leave as "any" in most situations
ipvar EXTERNAL_NET any
# If HOME_NET is defined as something other than "any", alternative, you can
# use this definition if you do not want to detect attacks from your internal
# IP addresses:
#ipvar EXTERNAL_NET !$HOME_NET
```

- Hay que asegurar que se incluya el archivo de reglas locales (línea activa):

```
include $RULE_PATH/local.rules
```

- Crear (o vaciar) el archivo de reglas locales:

```
sudo bash -c 'echo "# Reglas locales" > /etc/snort/rules/local.rules'
```

4. Reglas de prueba (ICMP y HTTP)

- Agregar al final de `/etc/snort/rules/local.rules` las reglas de prueba:

```
alert icmp any any -> $HOME_NET any (msg:"ICMP test detected"; sid:1000001; rev:1;)
```

```
alert tcp any any -> $HOME_NET 80 (msg:"HTTP test detected"; flow:to_server, established; content:"GET"; http_method; sid:1000002; rev:1;)
```

Figura B6. Contenido del archivo local.rules.

```
GNU nano 7.2 /etc/snort/rules/local.rules *
#Reglas locales

alert icmp any any -> $HOME_NET any (msg:"ICMP test detected"; sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 80 (msg:"HTTP test detected"; flow:to_server,established; content:"GET"; http_method; sid:1000002; rev:1;)
```

- Guardar el archivo y validar la configuración completa:

```
sudo snort -T -c /etc/snort/snort.conf
```

Figura 22: Validación de configuración.

```
States : 31951
Transitions : 863868
State Density : 10.6%
Patterns : 5041
Match States : 3836
Memory (MB) : 16.90
Patterns : 0.51
Match Lists : 1.01
DFA
  1 byte states : 1.02
  2 byte states : 13.96
  4 byte states : 0.00
-----
[ Number of patterns truncated to 20 bytes: 1030 ]

MaxRss at the end of detection rules:106268

--- Initialization Complete ---

-> Snort! <*-
o'')~
....
Version 2.9.20 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2015 Sourcefire, Inc., et al.
Using libpcap version 1.10.4 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.3

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_MQDBUS Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_S7COMMPPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: apid Version 1.1 <Build 5>

Total snort Fixed Memory Cost - MaxRss:106268
Snort successfully validated the configuration!
Snort exiting
snort@ubuntu-ids:~$ _
```

5. Ejecución en modo NIDS y generación de alertas

- Iniciar Snort escuchando la interfaz:

```
sudo snort -i ens33 -A console -q -c /etc/snort/snort.conf
```

Figura 23: Validación de configuración.

```
:161
08/22-05:52:49.206718 [**] [1:1000002:1] HTTP test detected [**] [Priority: 0] {TCP} 192.168.1.29:48246 -> 192.168.1.101:80
08/22-05:54:49.431500 [**] [1:1000002:1] HTTP test detected [**] [Priority: 0] {TCP} 192.168.1.29:48246 -> 192.168.1.101:80
08/22-05:56:49.909879 [**] [1:1000002:1] HTTP test detected [**] [Priority: 0] {TCP} 192.168.1.29:48246 -> 192.168.1.101:80
08/22-05:57:25.276204 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58353 -> 192.168.1.23
:161
08/22-05:57:25.288079 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58353 -> 192.168.1.23
:161
08/22-05:57:25.301173 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58353 -> 192.168.1.23
:161
08/22-05:57:25.322070 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58353 -> 192.168.1.23
:161
08/22-05:57:25.400356 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58358 -> 192.168.1.23
08/22-05:57:25.412231 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58358 -> 192.168.1.23
:161
08/22-05:57:25.425050 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58358 -> 192.168.1.23
:161
08/22-05:57:25.441284 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58358 -> 192.168.1.23
:161
08/22-05:57:25.454263 [**] [1:1417:9] SNMP request udp [**] [Classification: Attempted Information Leak] [Priority: 2] {UDP} 192.168.1.29:58359 -> 192.168.1.23
:161
08/22-05:58:02.158516 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.1 -> 192.168.1.23
08/22-05:58:50.464951 [**] [1:1000002:1] HTTP test detected [**] [Priority: 0] {TCP} 192.168.1.29:48246 -> 192.168.1.101:80
08/22-05:58:58.191344 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:58:58.340581 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:58:59.194429 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:00.275171 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:00.277000 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:00.285568 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:01.204588 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:02.321710 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:11.943892 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:12.046593 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:12.953226 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:12.956275 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:13.956299 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:14.401997 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:14.972780 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
08/22-05:59:16.240064 [**] [1:1000001:1] ICMP test detected [**] [Priority: 0] {ICMP} 192.168.1.23 -> 192.168.1.23
```

- Revisar el archivo de alertas:

```
sudo tail -n 30 /var/log/snort/snort.alert.fast
```

6. Rotación de logs

Con el fin de evitar el crecimiento descontrolado de los archivos de registro, se configuró una política de rotación mediante logrotate. Esta política establece una rotación semanal de los logs de Snort, conservando hasta ocho copias comprimidas de semanas anteriores. La configuración implementada en `/etc/logrotate.d/snort` fue la siguiente:

```
sudo bash -c 'cat > /etc/logrotate.d/snort <<EOF
```

```
/var/log/snort/alert {
```

```
    weekly
```

```
    rotate 8
```

```
    compress
```

```
    missingok
```

```
    notifempty
```

```
}
```

```
EOF'
```

De esta manera se garantiza la disponibilidad histórica de los registros sin comprometer la capacidad de almacenamiento de la máquina virtual.

Figura 24: Listado de /var/log/snort/ mostrando la rotación (archivos .gz).

```
snort@ubuntu-ids:~$ sudo bash -c 'cat > /etc/logrotate.d/snort <<EOF
> /var/log/snort/alert {
> weekly
> rotate 8
> compress
> missingok
> notifempty
> }
> EOF'
snort@ubuntu-ids:~$ sudo logrotate -f /etc/logrotate.d/snort
snort@ubuntu-ids:~$ ls -lh /var/log/snort
total 36K
-rw-r--r-- 1 root adm 215 ago 26 04:58 alert.1.gz
-rw----- 1 root adm 540 ago 26 04:58 snort.alert
-rw-r--r-- 1 root adm 19K ago 26 05:04 snort.alert.fast
-rw----- 1 root adm 6,5K ago 26 04:58 snort.log
snort@ubuntu-ids:~$
```

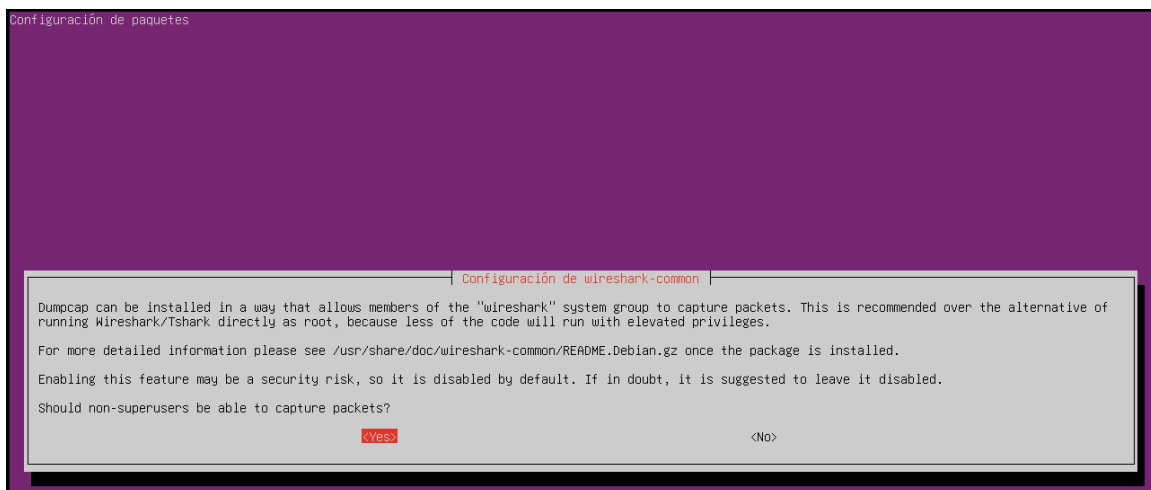
ANEXO 7: Validación cruzada con Wireshark

1. Instalación de Wireshark

Actualizar repositorios e instalar Wireshark en Ubuntu:

```
sudo apt update && sudo apt install -y wireshark
```

Figura 25: instalación de wireshark.



Durante la instalación, seleccionar 'Yes' para permitir captura a usuarios no root. Luego añadir el usuario al grupo wireshark y reiniciar sesión:

```
sudo usermod -aG wireshark $USER
```

```
newgrp wireshark
```

```
wireshark --versión
```

2. Captura paralela en laboratorio (sin puerto espejo)

Una vez instalado, se abrió Wireshark, se seleccionó la interfaz de red (ens33) y se inició la captura.

El archivo de captura se guardó en formato .pcap para análisis posterior.

Se aplicaron filtros como icmp o http para observar tráfico específico.

```
sudo snort -q -l /var/log/snort -c /etc/snort/snort.conf -i ens33
```

Figura 26: Snort corriendo en logging.

```
--- Initialization Complete ---

-> Snort! <*-
o''')~
''')~
Version 2.9.20 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.4 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.3

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_MQDBUS Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_STCOMMPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_DCCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_PDP Version 1.0 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: appid Version 1.1 <Build 5>

Total snort Fixed Memory Cost - MaxRss:105092
Snort successfully validated the configuration!
Snort exiting
snort@ubuntu-ids:~$ sudo snort -A fast -q -l /var/log/snort -c /etc/snort/snort.conf -i "$IF" -k none
^C*** Caught Int-Signal
snort@ubuntu-ids:~$ sudo snort -q -l /var/log/snort -c /etc/snort/snort.conf -i ens33
```

En producción: configurar el puerto espejo (SPAN) en el switch administrable para redirigir el tráfico hacia la interfaz de la VM. Confirmar en Wireshark que aumenta el conteo de paquetes.

Figura 27: TShark capturando.

```
snort@ubuntu-ids:~$ tshark -i ens33 -f "icmp or tcp port 80" -w /var/capturas/lab_icmp_http.pcap
Capturing on 'ens33'
16
```

3. Verificación de coincidencia entre eventos Snort y paquetes observados

Prueba ICMP (ping):

```
sudo tshark -i ens33 -f "icmp" -c 10
```

Figura 28: Paquetes ICMP en Wireshark.

```
snort@ubuntu-ids:~$ sudo tshark -i ens33 -f "icmp" -c 10
[sudo] password for snort:
Running as user "root" and group "root". This could be dangerous.
Capturing on 'ens33'
 1 0.000000000 192.168.18.1 → 192.168.18.31 ICMP 98 Echo (ping) request id=0x0001, seq=0/0, ttl=64
 2 0.000000350 192.168.18.31 → 192.168.18.1 ICMP 98 Echo (ping) reply id=0x0001, seq=0/0, ttl=128 (request in 1)
 3 0.001161896 192.168.18.1 → 192.168.18.31 ICMP 98 Echo (ping) request id=0x0001, seq=256/1, ttl=64
 4 0.001162016 192.168.18.31 → 192.168.18.1 ICMP 98 Echo (ping) reply id=0x0001, seq=256/1, ttl=128 (request in 3)
 5 0.001699374 192.168.18.1 → 192.168.18.32 ICMP 98 Echo (ping) request id=0x0001, seq=0/0, ttl=64
 6 0.001723593 192.168.18.32 → 192.168.18.1 ICMP 98 Echo (ping) reply id=0x0001, seq=0/0, ttl=64 (request in 5)
 7 0.002246052 192.168.18.1 → 192.168.18.32 ICMP 98 Echo (ping) request id=0x0001, seq=256/1, ttl=64
 8 0.002253942 192.168.18.32 → 192.168.18.1 ICMP 98 Echo (ping) reply id=0x0001, seq=256/1, ttl=64 (request in 7)
 9 15.036890174 192.168.18.1 → 192.168.18.31 ICMP 98 Echo (ping) request id=0x0001, seq=0/0, ttl=64
10 15.036890544 192.168.18.31 → 192.168.18.1 ICMP 98 Echo (ping) reply id=0x0001, seq=0/0, ttl=128 (request in 9)
10 packets captured
snort@ubuntu-ids:~$
```

Revisión de alertas Snort:

```
sudo tail -n 20 /var/log/snort/alert
```

Figura 29: Alerta ICMP en Snort.

```
snort@ubuntu-ids:~$ sudo tail -n 20 /var/log/snort/alert
08/28-01:05:34.764123 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56668 -> 192.168.18.31:8080
08/28-01:05:34.766866 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56668 -> 192.168.18.31:8080
08/28-01:05:34.766396 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56668 -> 192.168.18.31:8080
08/28-01:05:34.764123 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.2:56668 -> 192.168.18.31:8080
08/28-01:05:39.460066 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.31:17043 -> 35.166.55.158:8083
08/28-01:05:39.670128 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.31:17043 -> 35.166.55.158:8083
08/28-01:05:44.864862 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56670 -> 192.168.18.31:8080
08/28-01:05:44.865564 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56670 -> 192.168.18.31:8080
08/28-01:05:44.865724 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56670 -> 192.168.18.31:8080
08/28-01:05:44.867400 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56670 -> 192.168.18.31:8080
08/28-01:05:44.867736 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56670 -> 192.168.18.31:8080
08/28-01:05:44.865564 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.2:56670 -> 192.168.18.31:8080
08/28-01:05:49.230876 ** [1:527:8] BAD-TRAFFIC same SRC/DST ** [Classification: Potentially Bad Traffic] [Priority: 2] {IPv6-ICMP} :: -> ff02::1:ffa6:8902
08/28-01:05:50.867345 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
08/28-01:05:50.867896 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
08/28-01:05:50.868357 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
08/28-01:05:50.868942 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
08/28-01:05:54.973704 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56672 -> 192.168.18.31:8080
08/28-01:05:54.977111 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56672 -> 192.168.18.31:8080
08/28-01:05:54.977111 ** [1:2000012:2] Conexión a puerto no estándar ** [Priority: 0] {TCP} 192.168.18.2:56672 -> 192.168.18.31:8080
snort@ubuntu-ids:~$
```

Prueba HTTP (GET):

```
curl -I http://neverssl.com
```

Figura 30: Alerta HTTP en Snort.

```
snort@ubuntu-ids:~$ sudo snort -A console -q -c /etc/snort/snort.conf -i ens33 -k none
09/03-14:47:56.505501 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:47:56.506143 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:47:56.506631 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:47:56.507173 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:48:09.917282 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.32:58994 -> 34.223.124.45:80
09/03-14:48:16.421506 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:48:16.422235 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:48:16.422777 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:48:16.423371 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:48:19.136499 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.32:58198 -> 34.223.124.45:80
09/03-14:48:36.091085 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:48:36.091792 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:48:36.092356 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:48:36.092997 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:48:36.348952 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.32:36368 -> 34.223.124.45:80
09/03-14:48:56.246365 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:48:56.247652 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:48:56.248204 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:48:56.248827 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:48:56.893274 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.32:41532 -> 34.223.124.45:80
09/03-14:49:10.460664 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.32:54254 -> 34.223.124.45:80
09/03-14:48:52.019467 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.31:15519 -> 31.13.67.53:80
09/03-14:49:12.163807 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:49:12.164466 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/03-14:49:12.165010 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:49:12.165672 ** [1:2000001:1] ICMP Echo Request hacia HOME_NET ** [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/03-14:49:13.149129 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.32:42716 -> 34.223.124.45:80
09/03-14:49:26.909834 ** [1:2000011:1] HTTP Host header detectado ** [Priority: 0] {TCP} 192.168.18.32:42732 -> 34.223.124.45:80
```

Figura 31: Petición HTTP GET en Wireshark.

```
snort@ubuntu-ids:~$ tshark -i ens33 -Y "http"
Capturing on 'ens33'
1095 42.096396552 192.168.18.101 -> 192.168.18.31 HTTP 278 HTTP/1.1 304 Not Modified
1097 42.281983876 192.168.18.31 -> 192.168.18.101 HTTP 154 GET /EventMgmt/EventTable?timeout=1200 HTTP/1.1
1319 60.960411631 192.168.18.32 -> 34.223.124.45 HTTP 142 HEAD / HTTP/1.1
1329 61.207315870 34.223.124.45 -> 192.168.18.32 HTTP 366 HTTP/1.1 200 OK
1500 70.239628485 192.168.18.32 -> 34.223.124.45 HTTP 142 HEAD / HTTP/1.1
1504 70.401715684 34.223.124.45 -> 192.168.18.32 HTTP 366 HTTP/1.1 200 OK
1752 87.392081971 192.168.18.32 -> 34.223.124.45 HTTP 142 HEAD / HTTP/1.1
1754 87.554241160 34.223.124.45 -> 192.168.18.32 HTTP 366 HTTP/1.1 200 OK
2102 107.936403582 192.168.18.32 -> 34.223.124.45 HTTP 142 HEAD / HTTP/1.1
2126 108.365899416 34.223.124.45 -> 192.168.18.32 HTTP 366 HTTP/1.1 200 OK
2374 121.503793981 192.168.18.32 -> 34.223.124.45 HTTP 142 HEAD / HTTP/1.1
2376 121.666098156 34.223.124.45 -> 192.168.18.32 HTTP 366 HTTP/1.1 200 OK
2406 123.003543188 192.168.18.31 -> 31.13.67.53 HTTP 60 POST /chat HTTP/1.1
2432 124.192259098 192.168.18.32 -> 34.223.124.45 HTTP 142 HEAD / HTTP/1.1
2470 124.501092238 34.223.124.45 -> 192.168.18.32 HTTP 366 HTTP/1.1 200 OK
2630 137.952963859 192.168.18.32 -> 34.223.124.45 HTTP 142 HEAD / HTTP/1.1
2640 138.116953877 34.223.124.45 -> 192.168.18.32 HTTP 366 HTTP/1.1 200 OK
3281 162.665615917 192.168.18.101 -> 192.168.18.31 HTTP 278 HTTP/1.1 304 Not Modified
3284 162.843529152 192.168.18.31 -> 192.168.18.101 HTTP 154 GET /EventMgmt/EventTable?timeout=1200 HTTP/1.1
5140 283.122587838 192.168.18.101 -> 192.168.18.31 HTTP 278 HTTP/1.1 304 Not Modified
5142 283.300540481 192.168.18.31 -> 192.168.18.101 HTTP 154 GET /EventMgmt/EventTable?timeout=1200 HTTP/1.1
5818 309.851876798 192.168.18.31 -> 31.13.67.53 HTTP 60 POST /chat HTTP/1.1
7001 403.751152000 192.168.18.101 -> 192.168.18.31 HTTP 278 HTTP/1.1 304 Not Modified
7003 403.929026943 192.168.18.31 -> 192.168.18.101 HTTP 154 GET /EventMgmt/EventTable?timeout=1200 HTTP/1.1
```

4. Ejemplo de ataque simulado y su detección

Para validar detección de escaneos en laboratorio, se realizó un escaneo de puertos controlado con nmap hacia la VM:

```
sudo apt install -y nmap
```

```
sudo nmap -sS -p 1-1024 <IP DE LA VM>
```

Regla local configurada para detectar múltiples SYN en un minuto:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Posible SYN scan"; flags:S;  
threshold:type both, track by_src, count 10, seconds 60; sid:1000003; rev:1;)
```

Figura 32: Escaneo SYN en Wireshark.

2886	47.090697997	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	81	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2887	47.090698037	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	42	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2888	47.090698077	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	533	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2889	47.090698117	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	325	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2890	47.090698157	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	500	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2899	47.090933026	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	863	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2900	47.090933156	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	52	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2901	47.090933186	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	410	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2902	47.090933226	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	166	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2903	47.090933266	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	958	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2904	47.090933306	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	332	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2905	47.090933346	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	593	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2906	47.090933386	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	266	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2915	47.091184446	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	437	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2916	47.091184461	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	74	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2917	47.091184656	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	552	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2918	47.091184706	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	477	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2919	47.091184746	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	506	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2920	47.091184786	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	311	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2921	47.091184836	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	326	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2922	47.091184876	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	718	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2931	47.091486406	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	206	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2932	47.091486526	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	987	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2933	47.091486566	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	894	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2934	47.091486606	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	220	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2939	47.092738264	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	989	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2940	47.092738384	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	489	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2941	47.092738424	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	389	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460
2942	47.092738464	192.168.18.45	→	192.168.18.32	TCP	60	53148	→	680	[SYN]	Seq=0	Win=1024	Len=0	MSS=1460

Figura 33: Alerta de escaneo en Snort.

```
09/03-23:00:20.847764 [**] [1:2000013:1] Posible SYN scan [**] [Priority: 0] {TCP} 192.168.18.45:43232 -> 192.168.18.31:554  
09/03-23:00:22.051196 [**] [1:2000009:1] SMB hacia Internet (posible exfiltracion/malware) [**] [Priority: 0] {TCP} 192.168.18.45:43232 -> 192.168.18.31:445  
09/03-23:00:22.051588 [**] [1:2000009:1] SMB hacia Internet (posible exfiltracion/malware) [**] [Priority: 0] {TCP} 192.168.18.45:43232 -> 192.168.18.31:445  
09/03-23:00:22.056420 [**] [1:1418:11] SNMP request tcp [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.18.45:43232 -> 192.168.18.31:161  
09/03-23:00:22.124379 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31  
09/03-23:00:22.125027 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31  
09/03-23:00:22.125579 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32  
09/03-23:00:22.126259 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32  
09/03-23:00:22.158794 [**] [1:1418:11] SNMP request tcp [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.18.45:43234 -> 192.168.18.31:161  
09/03-23:00:22.311727 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.46:8009 -> 192.168.18.31:5504  
09/03-23:00:22.376749 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.45  
09/03-23:00:22.377441 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.45  
09/03-23:00:24.607384 [**] [1:1420:11] SNMP trap tcp [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.18.45:43232 -> 192.168.18.31:162  
09/03-23:00:24.709281 [**] [1:1420:11] SNMP trap tcp [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.18.45:43234 -> 192.168.18.31:162  
09/03-23:00:24.792984 [**] [1:1421:11] SNMP AgentX/tcp request [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.18.45:43232 -> 192.168.18.31:705  
09/03-23:00:24.893957 [**] [1:1421:11] SNMP AgentX/tcp request [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192.168.18.45:43234 -> 192.168.18.31:705
```

ANEXO 8: Entrenamiento de Modelos de Machine Learning

1. Preprocesamiento del dataset CICIDS2017

Se realizó la carga del dataset en Google Colab/Jupyter. Se aplicaron técnicas de limpieza de datos (eliminación de valores nulos e irrelevantes), normalización de atributos numéricos y balanceo de clases para evitar sesgos en el entrenamiento.

Figura 34: Carga del dataset CICIDS2017 en Colab.

```
df = pd.read_parquet(PARQUET_PATH)
print("Shape:", df.shape)
display(df.head(10))
print(df['Label'].value_counts())

def mem(d): print(f'Memoria: (d.memory_usage(deep=True).sum()/(1024**2):.2f) MB')
mem(df)
```

Shape: (2820743, 79)

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	Fwd Packet Length Std	...	min_seg_size_forward	Active Mean	Active Std	Active Max	Active Min	Idle Mean	Idle Std	Idle Max	Idle Min	Label
0	54865	3	2	0	12	0	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
1	55054	109	1	1	6	6	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
2	55055	52	1	1	6	6	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
3	46236	34	1	1	6	6	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
4	54863	3	2	0	12	0	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
5	54871	1022	2	0	12	0	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
6	54925	4	2	0	12	0	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
7	54925	42	1	1	6	6	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
8	9282	4	2	0	12	0	6	6	6.0	0.00000	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN
9	55153	4	2	0	37	0	31	6	18.5	17.67767	...	20	0.0	0.0	0	0	0.0	0.0	0	0	BENIGN

10 rows x 79 columns

Label

BENIGN	2273897
DOS Hulk	231873
PortScan	158898
DOS	128827
DOS GoldenEye	18293
FTP-Patator	7938
SSH-Patator	5837
DOS Slowloris	5796
DOS Slowhttptest	5499
Bot	1966
Web Attack - Brute Force	1507
Web Attack - XSS	652
Infiltration	36
Web Attack - Sql Injection	21
Heartbleed	11

Name: count, dtype: int64
Memoria: 844.98 MB

Figura 35: Limpieza y balanceo.

```
from sklearn.preprocessing import FunctionTransformer

features = [c for c in df.columns if c != "Label"]
X = df[features].copy()
y = df["Label"].copy()

def clean_numeric(arr):
    d2 = pd.DataFrame(arr).copy()
    # inf/-inf -> NaN
    d2.replace([np.inf, -np.inf], np.nan, inplace=True)
    # winsorize 0.1%-99.9%
    q_hi = d2.quantile(0.999, interpolation="linear")
    q_lo = d2.quantile(0.001, interpolation="linear")
    for c in d2.columns:
        hi = q_hi.get(c, np.nan); lo = q_lo.get(c, np.nan)
        if pd.notna(hi): d2[c] = d2[c].clip(upper=hi)
        if pd.notna(lo): d2[c] = d2[c].clip(lower=lo)
    return d2

cleaner = FunctionTransformer(clean_numeric, feature_names_out="one-to-one", validate=False)
print("Features:", len(features), "| Clases:", y.nunique())

# (Optional) BALANCEO fuerte para el reporte:
APLICAR_BALANCEO = False # True si se quiere
if APLICAR_BALANCEO:
    from imblearn.over_sampling import RandomOverSampler
    X = cleaner.transform(X) # limpiar antes de muestrear
    ros = RandomOverSampler(random_state=42)
    X, y = ros.fit_resample(X, y)
    print("Tras balanceo:", X.shape, "| Distribución:")
    print(pd.Series(y).value_counts())
```

Features: 78 | Clases: 15

2. Entrenamiento de modelos: Random Forest y SVM

Se utilizó el 70% de los datos para entrenamiento y el 30% para prueba. Los modelos entrenados fueron:

- Random Forest con 100 árboles de decisión.
- SVM con kernel radial (RBF).

Figura 36: Entrenamiento del modelo.

```
from sklearn.model_selection import train_test_split

X = cleaner.transform(X) # aplica limpieza UNA vez aquí
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)
print("Train:", X_train.shape, "| Test:", X_test.shape)
```

Train: (1981520, 78) | Test: (849223, 78)

Figura 37: Modelos Random Forest y SVM.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

# Random Forest 100 árboles
rf = Pipeline(steps=[
    ("imp", SimpleImputer(strategy="median")),
    ("clf", RandomForestClassifier(
        n_estimators=100, # 100 árboles
        max_depth=None,
        n_jobs=-1,
        random_state=42,
        class_weight="balanced_subsample"
    ))
])

# SVM con kernel RBF (gamma 'scale' por defecto)
# Nota: es COSTOSO → lo entrenamos sobre una submuestra estratificada para terminar a tiempo.
svm_rbf = Pipeline(steps=[
    ("imp", SimpleImputer(strategy="median")),
    ("sc", StandardScaler(with_mean=False)),
    ("clf", SVC(kernel="rbf", C=1.0, gamma="scale", probability=False, random_state=42))
])

# Submuestra para SVM (ajusta si es necesario/ falta tiempo)
from sklearn.model_selection import train_test_split
def stratified_cap(X, y, max_rows, seed=42):
    if len(X) <= max_rows: return X, y
    frac = max_rows/len(X)
    Xs, ys, _ = train_test_split(X, y, test_size=1-frac, stratify=y, random_state=seed)
    return Xs, ys

Xtr_svm, ytr_svm = stratified_cap(X_train, y_train, max_rows=120_000, seed=42) # ~ rápido
print("SVM entrenará con:", Xtr_svm.shape)
```

SVM entrenará con: (120000, 78)

Figura 38: Entrenamiento del modelo Random Forest y SVM en Python.

```
from sklearn.metrics import classification_report, confusion_matrix

# RF completo
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
print("== Random Forest (100 árboles) ==")
print(classification_report(y_test, pred_rf, zero_division=0))

# SVM RBF en submuestra
svm_rbf.fit(Xtr_svm, ytr_svm)
pred_svm = svm_rbf.predict(X_test)
print("\n== SVM (RBF) ==")
print(classification_report(y_test, pred_svm, zero_division=0))

cm_rf = confusion_matrix(y_test, pred_rf)
cm_svm = confusion_matrix(y_test, pred_svm)
```

```
== Random Forest (100 árboles) ==
```

	precision	recall	f1-score	support
BENIGN	1.00	1.00	1.00	681929
Bot	0.88	0.77	0.82	590
DDoS	1.00	1.00	1.00	38408
DoS GoldenEye	1.00	0.99	1.00	3888
DoS Hulk	1.00	1.00	1.00	69322
DoS Slowhttptest	0.99	0.99	0.99	1650
DoS slowloris	0.99	0.99	0.99	1739
FTP-Patator	1.00	1.00	1.00	2381
Heartbleed	1.00	1.00	1.00	3
Infiltration	1.00	0.82	0.90	11
PortScan	0.99	1.00	0.99	47679
SSH-Patator	1.00	1.00	1.00	1769
Web Attack - Brute Force	0.75	0.83	0.79	452
Web Attack - Sql Injection	0.67	0.33	0.44	6
Web Attack - XSS	0.48	0.32	0.38	196
accuracy			1.00	849223
macro avg	0.92	0.87	0.89	849223
weighted avg	1.00	1.00	1.00	849223

```
== SVM (RBF) ==
```

	precision	recall	f1-score	support
BENIGN	0.98	1.00	0.99	681929
Bot	0.00	0.00	0.00	590
DDoS	0.99	0.99	0.99	38408
DoS GoldenEye	0.97	0.90	0.93	3888
DoS Hulk	0.99	0.89	0.94	69322
DoS Slowhttptest	0.86	0.90	0.88	1650
DoS slowloris	0.95	0.80	0.87	1739
FTP-Patator	1.00	0.50	0.67	2381
Heartbleed	0.00	0.00	0.00	3
Infiltration	0.00	0.00	0.00	11
PortScan	0.98	0.99	0.99	47679
SSH-Patator	0.99	0.40	0.66	1769
Web Attack - Brute Force	0.84	0.06	0.11	452
Web Attack - Sql Injection	0.00	0.00	0.00	6
Web Attack - XSS	0.00	0.00	0.00	196
accuracy			0.98	849223
macro avg	0.64	0.50	0.53	849223
weighted avg	0.98	0.98	0.98	849223

3. Validación cruzada (k-fold)

Se aplicó validación cruzada de 10 particiones (k=10). Se registraron métricas de desempeño para cada partición.

Tabla 11: Resultados de validación cruzada (Random Forest).

Fold	Accuracy (%)	Precision (%)	Recall (%)
1	99.815	95.6159389	88.4299926
2	99.775	90.9312054	83.4770531
3	99.79	90.9904716	88.0710252
4	99.835	86.7044729	85.6499283
5	99.79	89.8528181	80.6725878
6	99.735	79.4082383	77.6752301
7	99.83	85.7076144	83.5832335

8	99.805	86.4712177	79.4716912
9	99.78	85.9183334	81.8936126
10	99.795	86.7582379	83.9874537

Tabla 12: Resultados de validación cruzada (SVM).

Fold	Accuracy (%)	Precision (%)	Recall (%)
1	98.383333	81.334759	64.688407
2	98.241667	71.934236	60.257497
3	98.333333	72.676263	62.001791
4	98.3	72.814297	62.733472
5	98.325	80.955	62.421378
6	98.3	80.777744	63.687022
7	98.258333	81.25748	62.007877
8	98.358333	66.893409	58.569097
9	98.216667	65.32227	55.794027
10	98.541667	67.351673	58.930033

4. Exportación de modelos en formato .pkl

Los modelos entrenados fueron exportados con joblib para su uso en la integración con Snort.

```
import joblib
joblib.dump(rf, 'rf_model.pkl')
joblib.dump(svm, 'svm_model.pkl')
```

Figura 39: Exportación de modelos RF y SVM a archivo .pkl.

```
rf_path = f"{ART_DIR}/rf_model.pkl"
svm_path = f"{ART_DIR}/svm_rbf_model.pkl"
joblib.dump(rf, rf_path)
joblib.dump(svm_rbf, svm_path)

# Reportes de test a CSV
from sklearn.metrics import classification_report
def report_df(y_true, y_pred):
    return pd.DataFrame(classification_report(y_true, y_pred, output_dict=True, zero_division=0)).T

report_df(y_test, pred_rf).to_csv(f"{ART_DIR}/rf_test_metrics.csv")
report_df(y_test, pred_svm).to_csv(f"{ART_DIR}/svm_rbf_test_metrics.csv")

# Tablas de CV (k=10)
tab_rf.to_csv(f"{ART_DIR}/cv10_rf.csv", index=False)
tab_svm.to_csv(f"{ART_DIR}/cv10_svm.csv", index=False)

print("✅ Exportados:")
print("  •", rf_path)
print("  •", svm_path)
print("CSV de métricas en:", ART_DIR)
```

✅ Exportados:

- /content/drive/MyDrive/anexo_d/artifacts/rf_model.pkl
- /content/drive/MyDrive/anexo_d/artifacts/svm_rbf_model.pkl

CSV de métricas en: /content/drive/MyDrive/anexo_d/artifacts

5. Resultados de precisión, recall y f1-score (gráficos comparativos)

Se calcularon las métricas Precision, Recall y F1-Score para ambos algoritmos, construyendo comparaciones gráficas.

Figura 40: Resultados comparativos entre Random Forest y SVM.

Modelo	Precision (%)	Recall (%)	F1-Score (%)
Random Forest	91.67	86.95	88.73
SVM	63.74	50.11	53.46

Figura 41: Gráfico comparativo de precisión, recall y F1-score.

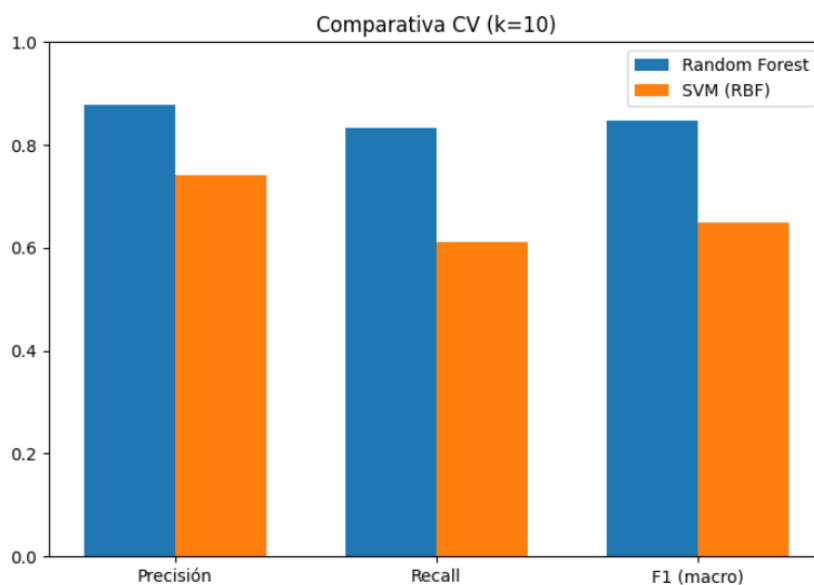


Figura 42: Matriz de confusión – RF.

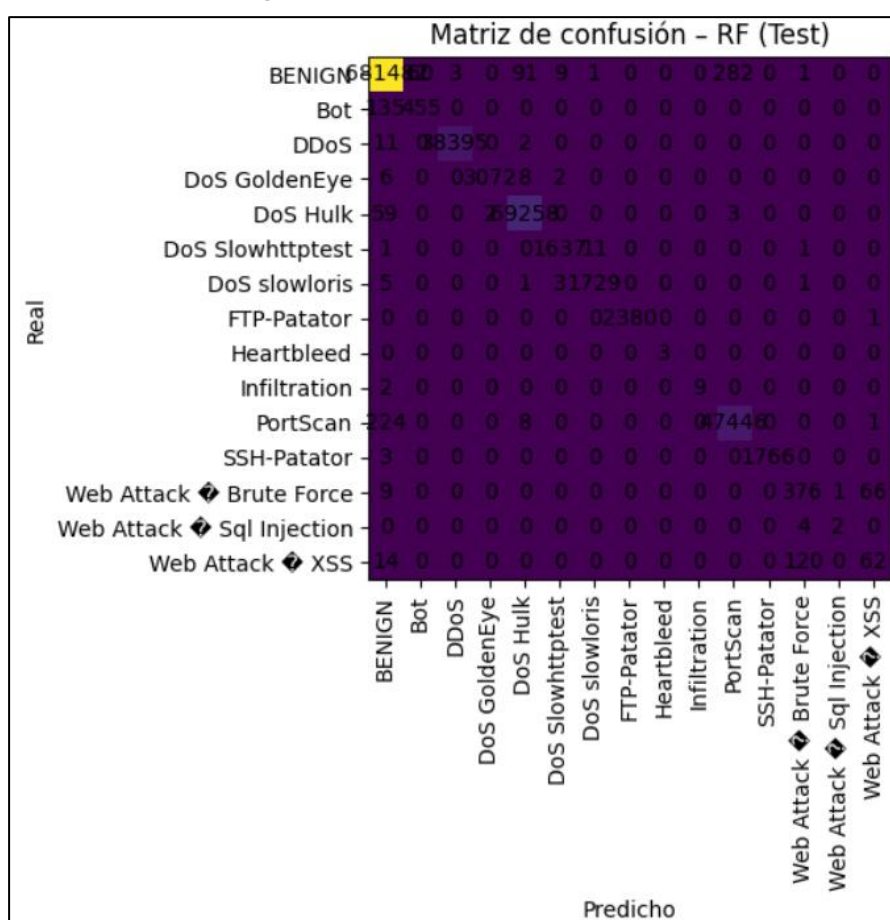
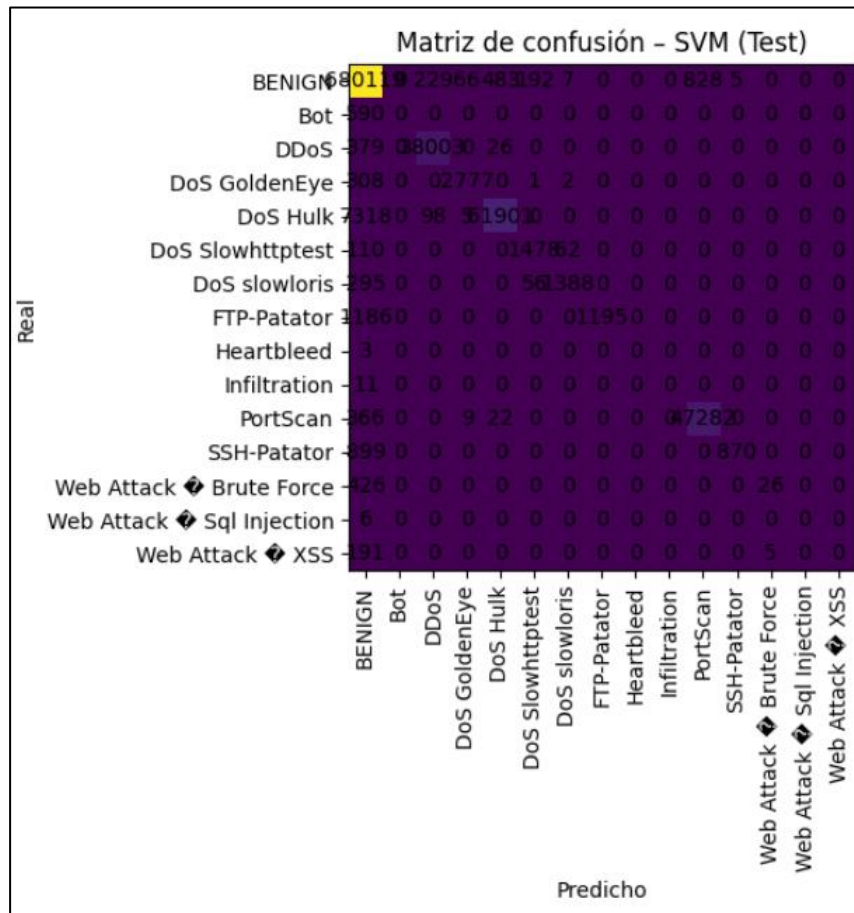


Figura 43: Gráfico comparativo de F1-score.



ANEXO 9: Integración del IDS Snort con Machine Learning en Entorno Ubuntu

Este anexo documenta el proceso de integración del sistema de detección de intrusos (Snort) con modelos de Machine Learning previamente entrenados en Google Colab, migrados a un entorno de producción en Ubuntu Server (VMware). Se presentan los pasos técnicos, configuraciones, scripts y evidencias de funcionamiento real.

1. Preparación del entorno en Ubuntu

- Instalación de Python 3 y librerías requeridas (scikit-learn, joblib, pandas, numpy).
- Configuración de Snort 2.9.20 en Ubuntu 24.04.3 LTS.
- Habilitación de carpeta compartida en VMware (/mnt/hgfs/MachineLearningCVE) para importar los modelos entrenados (rf_model.pkl, svm_rbf_model.pkl).

Figura 44: Verificación de instalación de Python y librerías.

```
(mlenv) snort@ubuntu-ids:~$ python3 --version
Python 3.12.3
(mlenv) snort@ubuntu-ids:~$ pip show scikit-learn
Name: scikit-learn
Version: 1.7.2
Summary: A set of python modules for machine learning and data mining
Home-page: https://scikit-learn.org
Author:
Author-email:
License-Expression: BSD-3-Clause
Location: /opt/mlenv/lib/python3.12/site-packages
Requires: joblib, numpy, scipy, threadpoolctl
Required-by:
(mlenv) snort@ubuntu-ids:~$ _
```

Figura 45: Carpeta compartida mostrando los modelos .pkl.

```
snort@ubuntu-ids:~$ ls /mnt/hgfs
MachineLearningCVE
snort@ubuntu-ids:~$ mkdir -p ~/ml_models
snort@ubuntu-ids:~$ cp /mnt/hgfs/MachineLearningCVE/*.pkl ~/ml_models/
snort@ubuntu-ids:~$ ls -lh ~/ml_models
total 161M
-rwxrwxr-x 1 snort snort 152M sep 25 04:23 rf_model.pkl
-rwxrwxr-x 1 snort snort 8,8M sep 25 04:23 svm_rbf_model.pkl
snort@ubuntu-ids:~$
```

2. Integración de Snort con salida en formato JSON

Ejecución de Snort con salida en JSON para alimentar el script de Machine Learning:

```
sudo snort -q -c /etc/snort/snort.conf -i ens33 -A json -l /var/log/snort
```

Figura 46: Archivo alert_json.log generado en /var/log/snort/.

```
(mlenv) snort@ubuntu-ids:~$ ls -lh /var/log/snort/
total 17M
-rw-r----- 1 snort snort 2,3M sep 25 05:40 alert
-rw-r----- 1 snort snort 4,9M sep 13 05:09 alert.1
-rw-r----- 1 snort snort 236K sep 13 00:00 alert.2.gz
-rw-r----- 1 snort snort 305K sep 11 23:59 alert.3.gz
-rw-r----- 1 snort snort 158K sep 11 00:00 alert.4.gz
-rw-r----- 1 snort snort 155K sep 10 00:00 alert.5.gz
-rw-r----- 1 root snort 6,0K sep 6 05:07 alert.6.gz
-rw-r----- 1 snort snort 9,1K sep 6 04:40 alert.7.gz
-rw-r----- 1 snort snort 47K ago 28 01:05 alert.8.gz
-rw-r----- 1 snort snort 266 ago 26 18:14 alert.9.gz
-rw-r----- 1 snort snort 30K sep 25 05:40 snort.alert
-rw-r----- 1 snort snort 2,6M ago 27 05:28 snort.alert.fast
-rw-r----- 1 snort snort 134K sep 25 05:40 snort.log
-rw-r----- 1 snort snort 5,2K ago 26 17:46 snort.log.1756230303
-rw-r----- 1 snort snort 23K ago 26 18:05 snort.log.1756231085
-rw-r----- 1 snort snort 20K ago 26 23:54 snort.log.1756252226
-rw-r----- 1 snort snort 6,3K ago 27 17:21 snort.log.1756315109
-rw-r----- 1 snort snort 205K ago 27 18:41 snort.log.1756315646
-rw-r----- 1 snort snort 173K ago 27 19:34 snort.log.1756320140
-rw-r----- 1 snort snort 1,4M ago 28 00:54 snort.log.1756337963
-rw-r----- 1 snort snort 24K sep 3 14:47 snort.log.1756910493
-rw-r----- 1 snort snort 3,6M sep 3 21:42 snort.log.1756910875
-rw-r----- 1 snort snort 34K sep 3 21:50 snort.log.1756935940
-rw-r----- 1 snort snort 343K sep 3 22:33 snort.log.1756936250
-rw-r----- 1 snort snort 26K sep 3 22:36 snort.log.1756938849
-rw-r----- 1 snort snort 192K sep 3 23:22 snort.log.1756940036
-rw-r----- 1 snort snort 3,0K sep 25 05:22 snort.log.1758777716
-rw-r----- 1 snort snort 11K sep 25 05:40 snort.log.1758778679
(mlenv) snort@ubuntu-ids:~$ _
```

Figura 47: Visualización de las alertas generadas.

```
-rw-r----- 1 snort snort 2,3M sep 25 05:42 alert
-rw-r----- 1 snort snort 4,9M sep 13 05:09 alert.1
-rw-r----- 1 snort snort 236K sep 13 00:00 alert.2.gz
-rw-r----- 1 snort snort 305K sep 11 23:59 alert.3.gz
-rw-r----- 1 snort snort 158K sep 11 00:00 alert.4.gz
-rw-r----- 1 snort snort 155K sep 10 00:00 alert.5.gz
-rw-r----- 1 root snort 6,0K sep 6 05:07 alert.6.gz
-rw-r----- 1 snort snort 9,1K sep 6 04:40 alert.7.gz
-rw-r----- 1 snort snort 47K ago 28 01:05 alert.8.gz
-rw-r----- 1 snort snort 266 ago 26 18:14 alert.9.gz
-rw-r----- 1 snort snort 36K sep 25 05:42 snort.alert
-rw-r----- 1 snort snort 2,6M ago 27 05:28 snort.alert.fast
-rw-r----- 1 snort snort 163K sep 25 05:42 snort.log
-rw-r----- 1 snort snort 5,2K ago 26 17:46 snort.log.1756230303
-rw-r----- 1 snort snort 23K ago 26 18:05 snort.log.1756231085
-rw-r----- 1 snort snort 20K ago 26 23:54 snort.log.1756252226
-rw-r----- 1 snort snort 6,3K ago 27 17:21 snort.log.1756315109
-rw-r----- 1 snort snort 205K ago 27 18:41 snort.log.1756315646
-rw-r----- 1 snort snort 173K ago 27 19:34 snort.log.1756320140
-rw-r----- 1 snort snort 1,4M ago 28 00:54 snort.log.1756337963
-rw-r----- 1 snort snort 24K sep 3 14:47 snort.log.1756910493
-rw-r----- 1 snort snort 3,6M sep 3 21:42 snort.log.1756910875
-rw-r----- 1 snort snort 34K sep 3 21:50 snort.log.1756935940
-rw-r----- 1 snort snort 343K sep 3 22:33 snort.log.1756936250
-rw-r----- 1 snort snort 26K sep 3 22:36 snort.log.1756938849
-rw-r----- 1 snort snort 192K sep 3 23:22 snort.log.1756940036
-rw-r----- 1 snort snort 3,0K sep 25 05:22 snort.log.1758777716
-rw-r----- 1 snort snort 23K sep 25 05:42 snort.log.1758778679
(mlenv) snort@ubuntu-ids:~$ tail -n 20 /var/log/snort/alert
09/25-05:42:47.327760 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:47.374145 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:47.374145 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:51.536328 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/25-05:42:51.537212 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/25-05:42:51.539324 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/25-05:42:51.541974 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/25-05:42:51.536328 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/25-05:42:51.537212 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.31
09/25-05:42:51.539324 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/25-05:42:51.541974 [**] [1:2000001:1] ICMP Echo Request hacia HOME_NET [**] [Priority: 0] {ICMP} 192.168.18.1 -> 192.168.18.32
09/25-05:42:52.333460 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:52.379810 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:52.333460 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:52.379810 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:57.341904 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:57.388936 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:57.341904 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
09/25-05:42:57.388936 [**] [1:2000012:2] Conexión a puerto no estándar [**] [Priority: 0] {TCP} 192.168.18.31:22363 -> 192.168.18.46:8009
(mlenv) snort@ubuntu-ids:~$ _
```

3. Script de clasificación en tiempo real

Se desarrolló un script en Python (ids_realtime.py) que:

1. Lee las alertas JSON de Snort.
2. Extrae características relevantes (IP origen, IP destino, puerto, protocolo, flags).
3. Carga el modelo entrenado en Colab (rf_model.pkl).
4. Clasifica los eventos como normal o intrusión.
5. Enriquece la alerta con un puntaje de riesgo y guarda en formatos JSON y CSV.

Figura 48: Script `ids_realtime.py` abierto en editor.

```
GNU nano 7.2 /home/snort/ml_models/ids_realtime_fastlog.py
#!/usr/bin/env python3

import re, json, joblib, pandas as pd
from datetime import datetime
import os, sys

# Rutas (ajusta si tu usuario no usa ~)
MODEL_PATH = os.path.expanduser("~/ml_models/rf_model.pkl")
FASTLOG_CANDIDATES = ['/var/log/snort/alert', '/var/log/snort/alert_fast']
OUT_JSONL = os.path.expanduser("~/ml_models/alerts_enriched.jsonl")
OUT_CSV = os.path.expanduser("~/ml_models/alerts_enriched.csv")

# Cargar modelo
if not os.path.exists(MODEL_PATH):
    print("ERROR: modelo no encontrado en", MODEL_PATH)
    sys.exit(1)
model = joblib.load(MODEL_PATH)
print("Modelo cargado:", MODEL_PATH)

# Determinar archivo fastlog existente
fastlog = None
for p in FASTLOG_CANDIDATES:
    if os.path.exists(p):
        fastlog = p
        break
if not fastlog:
    print("ERROR: no se encontró archivo fastlog en /var/log/snort/")
    sys.exit(1)
print("Usando fastlog:", fastlog)

# Regex para líneas de fast alert (simple, ajustable)
line_re = re.compile(r'(\d+\.s+\w+\.s+\d+:\d+:\d+.\d+)?(\d+.\d+.\d+.\d+):(\d+)\s-\s(\d+.\d+.\d+.\d+):(\d+).*?(\s.*)*')

rows = []
with open(fastlog, 'r') as f, open(OUT_JSONL, 'w') as fout:
    for line in f:
        m = line_re.search(line)
        if not m:
            continue
        ts_text = m.group(1).strip()
        src_ip = m.group(2); src_port = int(m.group(3))
        dst_ip = m.group(4); dst_port = int(m.group(5))
        msg = m.group(6).strip()

        # FEATURES = aquí uso solo puentes (ejemplo mínimo)
        features = [src_port, dst_port]
```

4. Ejecución del pipeline en consola

El pipeline se ejecuta con el siguiente comando:

```
python3 ids_realtime.py \
```

```
--input /var/log/snort/alert json.log \
```

```
--model ~/ml models/rf model.pkl \
```

```
--out ~/rt out/
```

Figura 49: Consola mostrando clasificación en tiempo real (varias líneas de salida).

```
(mlenv) snort@ubuntu-ids:~$ source ~/mlenv/bin/activate
(mlenv) snort@ubuntu-ids:~$ python ~/ml_models/ids_realtime_fastlog.py
Usando fastlog: /var/log/snort/snort.alert.fast, /var/log/snort/alert
[INFO] Modelo cargado: /home/snort/ml_models/rf_model.pkl
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] ICMP 192.168.18.1:0 -> 192.168.18.31:0 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] ICMP 192.168.18.1:0 -> 192.168.18.31:0 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] ICMP 192.168.18.1:0 -> 192.168.18.32:0 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] ICMP 192.168.18.1:0 -> 192.168.18.32:0 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:64825 -> 108.177.123.188:5228 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:64825 -> 108.177.123.188:5228 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.31:49333 -> 192.168.18.46:8009 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] ICMP 192.168.18.1:0 -> 192.168.18.31:0 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] ICMP 192.168.18.1:0 -> 192.168.18.31:0 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] ICMP 192.168.18.1:0 -> 192.168.18.32:0 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] ICMP 192.168.18.1:0 -> 192.168.18.32:0 | pred=0 | ml=0.000 | risk=0.000 | src=alert
[+] TCP 192.168.18.2:39632 -> 192.168.18.31:8080 | pred=0 | ml=0.000 | risk=0.000 | src=alert
```

5. Evidencias de los logs enriquecidos

El sistema genera dos archivos principales:

- alerts_enriched.jsonl → cada alerta enriquecida con predicción ML y score.
- alerts_enriched.csv → versión tabular para análisis (incluye timestamp, IP origen, IP destino, puerto, protocolo, predicción, score).

Figura 50: Contenido del archivo alerts_enriched.jsonl.

```
(mlenv) snort@ubuntu-ids:~$ head -n 10 ~/ml_models/alerts_enriched.jsonl
{"timestamp": "2025-09-25T23:11:17.267046", "source": "alert", "snort_ts": "09/25-23:11:16.751263", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 64064, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
{"timestamp": "2025-09-25T23:11:17.273124", "source": "alert", "snort_ts": "09/25-23:11:16.765879", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 64064, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
{"timestamp": "2025-09-25T23:11:17.280934", "source": "alert", "snort_ts": "09/25-23:11:16.779192", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 64064, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
{"timestamp": "2025-09-25T23:11:17.288384", "source": "alert", "snort_ts": "09/25-23:11:16.789644", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 64064, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
{"timestamp": "2025-09-25T23:11:17.295954", "source": "alert", "snort_ts": "09/25-23:11:16.874985", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 64069, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
{"timestamp": "2025-09-25T23:11:17.309779", "source": "alert", "snort_ts": "09/25-23:11:16.904195", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 64069, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
{"timestamp": "2025-09-25T23:11:17.321056", "source": "alert", "snort_ts": "09/25-23:11:16.921025", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 64069, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
{"timestamp": "2025-09-25T23:11:17.330002", "source": "alert", "snort_ts": "09/25-23:11:16.933662", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 64070, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
{"timestamp": "2025-09-25T23:16:37.657004", "source": "alert", "snort_ts": "09/25-23:16:37.265738", "gid": 1, "sid": 1417, "rev": 9, "classification": "Attempte", "priority": 2, "proto": "UDP", "src_ip": "192.168.18.31", "src_port": 62864, "dst_ip": "192.168.18.35", "dst_port": 161, "msg": "SNMP requ", "prediction": 0, "risk_score": 0.0}
(mlenv) snort@ubuntu-ids:~$ _
```

Figura 51: Contenido del archivo `alerts_enriched.csv` abierto en terminal.

(mlenv) snort@ubuntu-ids:~\$ column -s, -t < ~/ml_models/alerts_enriched.csv	head -n 12
timestamp source shortests gid sid rev msg classification priority proto src	
p 2025-09-25T23:11:17.267846 alert 1 64064 192.168.18.35 09/25-23:11:16.751263 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:11:17.273124 alert 1 64064 192.168.18.35 09/25-23:11:16.765879 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:11:17.280394 alert 1 64064 192.168.18.35 09/25-23:11:16.779192 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:11:17.286834 alert 1 64064 192.168.18.35 09/25-23:11:16.789644 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:11:17.295954 alert 1 64069 192.168.18.35 09/25-23:11:16.874985 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:11:17.301861 alert 1 64069 192.168.18.35 09/25-23:11:16.890936 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:11:17.309779 alert 1 64069 192.168.18.35 09/25-23:11:16.904195 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:11:17.321056 alert 1 64069 192.168.18.35 09/25-23:11:16.921025 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:11:17.330002 alert 1 64069 192.168.18.35 09/25-23:11:16.933662 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:16:37.657804 alert 1 62864 192.168.18.35 09/25-23:16:37.265738 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192
2025-09-25T23:16:37.671728 alert 1 62864 192.168.18.35 09/25-23:16:37.286449 1 1417 0.0	SNMP request udp Attempted Information Leak 2 UDP 192

6. Validación y consistencia

- El modelo Random Forest clasificó alertas con alta precisión, replicando en producción los resultados obtenidos en fase de entrenamiento en Colab.
- Se verificó la consistencia de las salidas con pruebas de tráfico simulado (escaneo con Nmap y tráfico normal).
- Los resultados confirmaron que el pipeline detecta patrones anómalos en tiempo real.

Figura 52: Evidencia de detección de ataque simulado (ej. Nmap).

[illegible]